

جدول المحتويات لوثيقة متطلبات النظام - (SRS) مشروع Hands

رقم الصفحة	العنوان / الفصل
3	مقدمة المشروع
4	الوصف العام للنظام
5 - 16	حالات الاستخدام (Use Cases)
17 - 73	المعمارية البرمجية والوحدات التفصيلية للأنظمة الخلفية
17 - 24	-المصادقة والتفويض (Authentication and Authorization)
24 - 39	-نموذج طلب الخدمة وإدارة الخدمات (Service Request and Service Management Model)
39 - 49	-نظام الشكاوي والبلاغات وإدارة الحظر
49 - 53	-نظام المراقبة والتدقيق (Monitoring and Audit System)
53 - 55	-نظام النسخ الاحتياطي
55 - 58	-إدارة ملفات المستخدمين (User Profile Management)
58 - 61	-نظام التحقق من مزودي الخدمات (Provider Verification System)
61 - 65	-إدارة المحفظة الإلكترونية (Electronic Wallet Management)
65 - 70	-نظام التقارير وتصدير الملفات
70 - 73	-لوحة التحكم والإحصائيات والرسوم البيانية
73 - 77	دراسة تطبيقات مشابهة
77 - 80	متطلبات الواجهة الخارجية والمتطلبات غير الوظيفية

الفصل الأول: مقدمة المشروع

1. المقدمة

1.1 الهدف من الوثيقة:

تهدف هذه الوثيقة إلى تحديد وتوصيف المتطلبات الوظيفية (Functional) وغير الوظيفية (Non-Functional) لنظام Hands بشكل هندسي دقيق ومتكامل. يعتبر هذا النظام منصة رائدة تهدف إلى ربط المستخدمين الباحثين عن خدمات متنوعة (منزلية، تقنية، تعليمية، صحية، تجميلية وغيرها) مع مقدمي خدمات محترفين وموثقين في المجتمع المحلي السوري.

تُعدّ هذه الوثيقة المرجع البرمجي والتحليلي الأساسي لجميع أعضاء فريق التطوير لضمان بناء نظام متكامل ومطابق لأفضل المعايير الهندسية.

1.2 اصطلاحات وتعريف:

- **المستخدم (User/Customer):** العميل أو طالب الخدمة الذي يقوم بإنشاء حساب على التطبيق لتصفح الخدمات وإرسال الطلبات وتلقي العروض وتقييم النتائج.
- **مقدم الخدمة (Service Provider):** الفني أو المهني الذي يمتلك مهارات معينة ويقوم بالاشتراك في المنصة لتلقي طلبات العملاء وإرسال عروض الأسعار والتفاوض وتقديم خدماته مقابل أجر مالي.
- **المشرف (Admin):** موظف إداري يمتلك صلاحيات الإشراف على المحتوى، مراجعة الشكاوى، وتعديل الخدمات والأسئلة وحل النزاعات.
- **المدير العام (Super Admin):** مالك النظام الذي يمتلك الصلاحيات المطلقة لإدارة حسابات المشرفين، محاسبي الكاشير، وإدارة الصلاحيات الشاملة للنظام.
- **أمين الصندوق / الكاشير (Cashier):** المسؤول المالي عن مراجعة طلبات سحب الأموال المقدمة من مقدمي الخدمات وتدقيق التحويلات المالية.

1.3 نطاق المشروع:

يقدم نظام Hands حلاً تقنياً متكاملاً يغطي دورة حياة تقديم الخدمة بالكامل:

1. يتيح للعميل اختيار نوع الخدمة عبر نماذج أسئلة ديناميكية تجمع متطلبات العميل بدقة.
2. يسمح لمقدمي الخدمات بإرسال عروض أسعار تنافسية ضمن مجالات تخصصهم.
3. يوفر قناة اتصال آمنة ومباشرة (محادثة نصية فورية) للاتفاق على التفاصيل والسعر النهائي.
4. يدمج نظام دفع إلكتروني متكامل عبر بوابة ShamCash يضمن الحقوق المالية لكلا الطرفين.
5. يوفر محفظة مالية ذكية ونظام كاشير معقد لإجراء عمليات السحب الآمن لمقدمي الخدمات.
6. يدعم المنصة بنظام إدارة بلاغات وشكاوى وتذاكر فنية لضمان تجربة مستخدم خالية من العوائق.

1.4 المراجع الدراسية والتحليلية:

- دراسات استقصائية ومقابلات شخصية مع شريحة واسعة من أصحاب المهن اليدوية والتقنية في سوريا لمعرفة احتياجاتهم.
- دراسة تحليلية لتطبيقات ومواقع عالمية وعربية رائدة في هذا المجال (مثل Mostaqbal، Khamsat، TaskRabbit، Upwork) واستنتاج نقاط القوة والضعف لتلافيها.

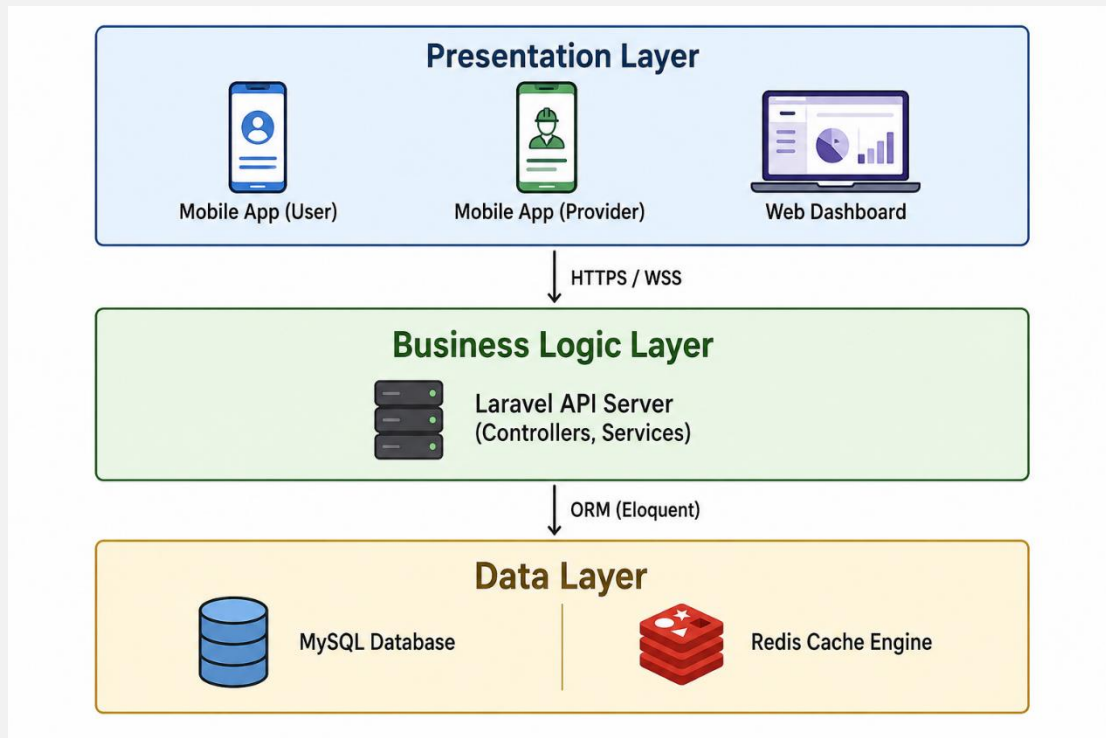
- معايير هندسة البرمجيات القياسية الصادرة عن معهد مهندسي الكهرباء والإلكترونيات (IEEE 830-1998) والخاصة بصياغة وثائق مواصفات متطلبات البرمجيات (SRS).

الفصل الثاني: الوصف العام للنظام

2.1 منظور المنتج والمعمارية العامة:

يعتمد نظام Hands على معمارية خادم-عميل تفاعلية وموزعة متمثلة بـ **Client-Server Architecture**. تتكون البنية من ثلاثة مستويات رئيسية:

1. **واجهة العرض (Presentation Layer):** وتتمثل في تطبيقات الهواتف المحمولة الذكية لكل من العميل ومقدم الخدمة، إضافة إلى لوحة تحكم الويب للمشرفين والمديرين.
2. **منطق الأعمال (Business Logic Layer):** وهو خادم الواجهة الخلفية (Backend API Server) المبني باستخدام إطار العمل **Laravel (PHP)** والذي يدير جميع العمليات والمنطق البرمجي والتحقق القوي من المدخلات. وتم بناءه اعتماداً على مبدأ الطبقات الثلاث (controller-service-repository).
3. **طبقة البيانات (Data Layer):** وتتمثل في قاعدة بيانات **MySQL** لتخزين البيانات العلائقية ومستودعات التخزين السحابي للمرفقات والمستندات.



2.2 الافتراضات والاعتماديات الخارجية:

- توفر اتصال إنترنت مستقر ومستمر لدى جميع الأطراف لاستقبال الرسائل والإشعارات في الوقت الفعلي (Real-Time).
- توفر البريد الإلكتروني بصورة دائمة لإتمام عمليات التحقق من الحسابات واسترجاع كلمات المرور بنجاح.
- موثوقية واستقرار بوابة الدفع الخارجية ShamCash لضمان سلامة العمليات المالية وتجنب حالات تعليق المعاملات.

الفصل الثالث: USECASE

الخاصية (Field)	الوصف (Description)
معرف حالة الاستخدام	UC-01
اسم حالة الاستخدام	تسجيل حساب جديد (Register New User)
الفاعل الرئيسي	المستخدم ومقدم الخدمة
الفاعلون الثانويون	(Email Service Provider) نظام البريد الإلكتروني
الوصف المختصر	تمكّن هذه العملية الزائر من إنشاء حساب جديد في النظام كمطلب أساسي للاستفادة من الخدمات، سواء كمستفيد (User) أو كمقدم خدمة (Provider) عبر تأكيد بريده الإلكتروني.
الشروط المسبقة	1. أن يكون النظام متاحاً وقاعدة البيانات متصلة. 2. ألا يكون البريد الإلكتروني المدخل مسجلاً مسبقاً في النظام.
الشروط اللاحقة	1. email_verified_at (يتم إنشاء سجل مستخدم جديد بحالة غير نشطة مؤقتاً = null). 2. يتم إرسال رمز تحقق صالح ومؤقت إلى بريد المستخدم.
المسار الأساسي (Basic Flow)	1. يقوم الفاعل بإرسال طلب التسجيل متضمناً (الاسم، البريد الإلكتروني، كلمة المرور وتأكيدها، والدور المطلوب). 2. يتحقق النظام من صحة البيانات والقيود (الاسم نصي أحرف فقط، البريد الإلكتروني فريد، وكلمة المرور قوية ومعقدة). 3. يقوم النظام بتوليد رمز تحقق عشوائي من 6 أرقام. 4. يقوم النظام بإرسال البريد الإلكتروني للمستخدم محتوياً على الرمز. 5. يقوم النظام بتشفير كلمة المرور لحفظها الآمن في قاعدة البيانات. 6. (0) يتم حفظ سجل المستخدم الجديد مع تعيين عدّد المحاولات الفاشلة إلى 6. 7. مع رسالة توجيهية لتفعيل الحساب (HTTP 201) يرجع النظام استجابة بنجاح العملية.
المسارات البديلة والاستثناءات	[Validation Error - خطأ في التحقق من البيانات] في حال كانت أي من المدخلات غير مطابقة للشروط، يرفض النظام إكمال العملية ويعيد رسالة خطأ تفصيلية للمستخدم مع رمز الحالة (HTTP 422).
(Alternative / Exception Flows)	إذا حاول الفاعل إرسال طلب [Unauthorized Role - محاولة تلاعب بالصلاحيات] يتخطى النظام فحص التحقق (role = admin) يحتوي على دور مدير نظام. يمنع العملية فوراً (Exception) البرمجي الأولي ليبرمي استثناء أمنياً مباشراً. في حال تعذر إرسال البريد الإلكتروني، يتم إيقاف العملية وإعلام [فشل خادم البريد] المستخدم بوجود مشكلة مؤقتة في الإرسال.

الخاصية (Field)	الوصف (Description)
معرف حالة الاستخدام	UC-02
الاستخدام	

اسم حالة الاستخدام	(Verify Email) توثيق البريد الإلكتروني
الفاعل الرئيسي	(Unverified User) المستخدم غير الموثق
الفاعلون الثانويون	لا يوجد (نظام قاعدة البيانات فقط)
الوصف المختصر	تتيح هذه العملية للمستخدم تفعيل حسابه وتوثيق بريده الإلكتروني عن طريق إدخال رمز التحقق المكون من 6 أرقام والذي تم إرساله إلى بريده أثناء التسجيل.
الشروط المسبقة	أن يكون للمستخدم حساب مسجل مسبقاً في النظام بحالة غير موثقة. 1. (email_verified_at = null). أن يكون المستخدم قد استلم رمز التحقق المكون من 6 أرقام على بريده الإلكتروني. 2.
الشروط اللاحقة	يتم تحديث حالة الحساب إلى موثق عبر تخزين تاريخ ووقت التوثيق الحسابي الحالي. 1. email_verified_at. لمنع (email_verification_code = null) يتم تصفير رمز التحقق وجعله فارغاً. 2. إعادة استخدامه.
المسار الأساسي	يقوم الفاعل بإدخال البريد الإلكتروني ورمز التحقق المستلم (المكون من 6 أرقام). 1. يتحقق النظام من صحة البيانات والقيود (البريد الإلكتروني بصيغة صحيحة، والرمز. 2. يتكون من 6 أرقام بالضبط). يبحث النظام في قاعدة البيانات عن مستخدم يطابق البريد الإلكتروني ورمز التحقق معاً. 3. يجد النظام السجل المطابق بنجاح. 4. يقوم النظام بتحديث السجل: يضع تاريخ التوثيق الحالي ويسمح الرمز القديم. 5. مع رسالة تؤكد توثيق الحساب بنجاح (HTTP 200) يعيد النظام استجابة بنجاح العملية. 6.
المسارات البديلة والاستثناءات	في حال ترك الحقول فارغة أو [Validation Error - خطأ في التحقق من المدخلات] (HTTP 422) إدخال رمز بطول مختلف عن 6 أرقام، يرفض النظام الطلب ويعيد رسالة خطأ.
(Alternative / Exception Flows)	في حال عدم العثور على مستخدم: [Invalid Code - رمز التحقق غير صحيح أو منتهي] يطابق البريد الإلكتروني والرمز معاً (إما لأن الرمز خاطئ أو البريد غير مسجل)، يرفض (HTTP 422) النظام العملية ويعيد رسالة خطأ: "رمز غير صحيح" مع رمز الحالة.

(Field) الخاصية	(Description) الوصف
معرف حالة الاستخدام	UC-03
اسم حالة الاستخدام	(User Login) تسجيل الدخول إلى النظام
الفاعل الرئيسي	(Registered User / Provider) المستخدم المسجل
الفاعلون الثانويون	(Email Service Provider) نظام البريد الإلكتروني
الوصف المختصر	تمكّن هذه العملية المستخدم (سواء كان مستفيداً أو مقدم خدمة) من الدخول إلى حسابه في التطبيق بعد التحقق من صحة بيانات اعتماده وحالة حسابه الأمنية.
الشروط المسبقة	أن يكون للمستخدم حساب مسجل مسبقاً في قاعدة البيانات. 1. ألا يكون الحساب مقفلاً بشكل مؤقت بسبب محاولات فاشلة سابقة. 2.
الشروط اللاحقة	يتم تصفير عداد المحاولات الفاشلة وإلغاء أي قفل زمني. 1. (Role) وإرجاعه للمستخدم مصحوباً بدوره (JWT Token) يتم توليد رمز وصول آمن. 2. ومؤشر لتغيير كلمة المرور إن لزم الأمر.
المسار الأساسي	يقوم الفاعل بإدخال البريد الإلكتروني وكلمة المرور الخاصة به. 1. (Validation). يتحقق النظام من إدخال البيانات بالشكل الصحيح. 2. يبحث النظام عن المستخدم في قاعدة البيانات عبر بريده الإلكتروني. 3. يتحقق النظام من أن الحساب ليس في حالة قفل مؤقت. 4. يتحقق النظام من مطابقة كلمة المرور المدخلة مع المشفرة في قاعدة البيانات وتكون صحيحة. 5. ليس (email_verified_at) يتحقق النظام من أن البريد الإلكتروني موثق ومفعّل. 6. (فارغاً).
(Basic Flow)	

	7. وإلغاء فترات (failed_attempts = 0) يقوم النظام بتصفير عداد المحاولات الفاشلة . 8. يتحقق النظام مما إذا كان قد مضى 90 يوماً أو أكثر على آخر تغيير لكلمة المرور ليحدد (must_change_password). 9. لإثبات الهوية JWT يقوم النظام بتوليد رمز الـ . 10. ونوع Token تحتوي على الـ (HTTP 200) يعيد النظام استجابة بنجاح العملية . . وقيمة مؤشر تغيير كلمة المرور (Role) الحساب
المسارات البديلة والاستثناءات (Alternative / Exception Flows)	في حال إرسال حقول فارغة أو بريد: [Validation Error - خطأ في التحقق من المدخلات] (HTTP 422) غير صالح، يعيد النظام خطأً . إذا لم يتم العثور على البريد أو: [Invalid Credentials - بيانات الاعتماد غير صحيحة] كانت كلمة المرور خاطئة، يقوم النظام بزيادة عداد المحاولات الفاشلة بمقدار (1)، ويرمي "Invalid credentials" برسالة (HTTP 401) استثناءً يترجم إلى استجابة غير مصرح بها . عند وصول عداد المحاولات الفاشلة: [Account Locking - قفل الحساب لتكرار الخطأ] دقيقة 15 محاولات، يقوم النظام بتصفير العداد وقفل الحساب تلقائياً لمدة 5 إلى ، ويرسل بريداً إلكترونياً تنبيهاً للمستخدم (locked_until = now + 15 mins) (HTTP 401) ، ويرفض الدخول بـ (Security Alert Mail) . إذا حاول الفاعل تسجيل الدخول والحساب لا يزال في: [محاولة دخول لحساب مقفل زمنياً] فترة القفل (خلال الـ 15 دقيقة)، يرفض النظام الطلب مباشرة دون فحص كلمة المرور ويعيد "Account locked temporarily" (HTTP 401) رسالة . إذا كانت البيانات صحيحة ولكن: [Email Not Verified - الحساب غير موثق البريد] "Email" المستخدم لم يوثق بريده الإلكتروني بعد، يرفض النظام تسجيل الدخول ويعيد رسالة "not verified" (HTTP 401).

(Field) الخاصية	(Description) الوصف
معرف حالة الاستخدام	UC-04
اسم حالة الاستخدام	(Request Password Reset Code) طلب رمز إعادة تعيين كلمة المرور
الفاعل الرئيسي	(User / Provider) المستخدم
الفاعل الثانويون	(Email Service Provider) نظام البريد الإلكتروني
الوصف المختصر	تتيح هذه العملية للمستخدم الذي نسي كلمة المرور الخاصة به طلب رمز تحقق مؤقت وعشوائي يتم إرساله إلى بريده الإلكتروني المسجل لتبدأ عملية استعادة الحساب . أن يكون للنظام اتصال بخادم البريد الإلكتروني وخادم قاعدة البيانات . أن يكون البريد الإلكتروني المدخل مسجلاً وموجوداً بالفعل في جدول المستخدمين . (users).
الشروط المسبقة	1. أن يكون البريد الإلكتروني المدخل مسجلاً وموجوداً بالفعل في جدول المستخدمين . 2. أن يكون البريد الإلكتروني المدخل مسجلاً وموجوداً بالفعل في جدول المستخدمين . (users).
الشروط اللاحقة	يتم توليد وتحديث/إدراج رمز إعادة التعيين والوقت الحالي في جدول password_reset_tokens. يتم إرسال رسالة بريد إلكتروني تحتوي على الرمز للمستخدم .
المسار الأساسي (Basic Flow)	يقوم الفاعل بطلب إعادة تعيين كلمة المرور وإدخال بريده الإلكتروني المسجل . يتحقق النظام من صحة المدخلات (البريد الإلكتروني بصيغة صحيحة وموجود في قاعدة البيانات) . يقوم النظام بتوليد رمز عشوائي مكون من 6 أرقام . في حال وجود رمز سابق: password_reset_tokens يقوم النظام بفحص جدول لهذا البريد يتم تحديثه، وفي حال عدم وجوده يتم إدراج سجل جديد بالرمز والوقت الحالي . يقوم النظام بإرسال رسالة بريد إلكتروني إلى الفاعل تحتوي على الرمز الجديد للتغيير . مع رسالة تأكيد: "تم إرسال رمز (HTTP 200) يعيد النظام استجابة بنجاح العملية . "إعادة تعيين كلمة المرور
المسارات البديلة والاستثناءات	في حال ترك حقل البريد فارغاً، أو: [Validation Error - خطأ في التحقق من البيانات] كتابته بصيغة غير صحيحة، أو إدخال بريد غير مسجل في النظام، يرفض النظام الطلب (HTTP 422) ويعيد رسالة خطأ تفصيلية .

(Alternative / Exception Flows)	في حال تعذر الاتصال بخادم البريد الإلكتروني بعد حفظ: [فشل إرسال البريد الإلكتروني] الرمز في قاعدة البيانات، تتوقف العملية ويُفضل معالجة الفشل لضمان عدم تضليل (HTTP 500 بإرجاع خطأ في الخادم المضيف) المستخدم.
---------------------------------	--

(Field) الخاصية	(Description) الوصف
معرف حالة الاستخدام	UC-05
اسم حالة الاستخدام	(Reset Password) إعادة تعيين كلمة المرور
الفاعل الرئيسي	المستخدم (User / Provider)
الفاعل الثانويون	لا يوجد (نظام قاعدة البيانات فقط)
الوصف المختصر	تتيح هذه العملية للمستخدم إدخال كلمة مرور جديدة وتأكيدا وتطبيقها على حسابه، وذلك بعد إثبات ملكيته للحساب عن طريق الرمز المكون من 6 أرقام والمطابق للبريد الإلكتروني. وتم توليد السجل في (UC-04) أن يكون المستخدم قد طلب رمز إعادة تعيين مسبقاً. 1.
الشروط المسبقة	جدول الرموز. أن يكون البريد الإلكتروني مسجلاً في قاعدة بيانات النظام. 2.
الشروط اللاحقة	يتم تشفير وتحديث كلمة المرور الجديدة في حساب المستخدم. 1. يتم تصفير محاولات تسجيل الدخول الفاشلة وإلغاء أي قفل مؤقت على الحساب. 2. يتم تحديث مؤشر تاريخ تغيير كلمة المرور للوقت الحالي لحساب دورة الـ 90 يوماً. 3. الأمنية. لمنع إعادة password_reset_tokens يتم حذف رمز التحقق المستخدم من جدول. 4.
المسار الأساسي (Basic Flow)	يقوم الفاعل بإدخال البريد الإلكتروني، رمز التحقق (6 أرقام)، كلمة المرور الجديدة، 1. وتأكيد كلمة المرور. يتحقق النظام من صحة البيانات والقيود (البريد الإلكتروني موجود ومسجل، الرمز. 2. مكون من 6 أرقام، وكلمة المرور الجديدة تحقق الشروط الأمنية: 8 أحرف، حروف كبيرة وصغيرة، أرقام، رموز، ومطابقة للتأكيد) عن سجل يطابق البريد password_reset_tokens يبحث النظام في جدول. 3. الإلكتروني والرمز معاً. يتحقق النظام من صلاحية الرمز الزمنية (بحيث لا يكون قد مضى على توليده أكثر من 10 دقائق). 4. يقوم النظام بتشفير كلمة المرور الجديدة وحفظها في جدول المستخدمين، وتحديث قيم failed_attempts = 0 و locked_until = null و password_changed_at = now(). 5. password_reset_tokens يقوم النظام بحذف السجل الخاص بالرمز من جدول. 6. مع رسالة تأكيد: "تم تغيير كلمة (HTTP 200) يعيد النظام استجابة بنجاح العملية. 7. المرور بنجاح".
المسارات البديلة والاستثناءات (Alternative / Exception Flows)	في حال كانت كلمة المرور ضعيفة: [Validation Error - خطأ في التحقق من المدخلات] أو غير متطابقة مع حق التأكيد، أو البريد غير مسجل، يرفض النظام الطلب ويعيد رسالة (HTTP 422) خطأ تفصيلية. في حال عدم العثور على سجل يجمع البريد: [Invalid Code - رمز التحقق غير صحيح] والرمز معاً، يوقف النظام العملية ويرمي استثناءً يترجم إلى استجابة خطأ برسالة: "رمز (لديك Exception Handler حسب إعدادات الـ HTTP 422/401) غير صحيح". إذا وجد النظام الرمز والبريد متطابقين، ولكن: [Code Expired - انتهاء صلاحية الرمز] تبين أنه مضى على توليد الرمز أكثر من 10 دقائق، يتم إيقاف العملية فوراً ويرمي استثناءً برسالة: "انتهت صلاحية الرمز".

الخاصية (Field)	الوصف (Description)
معرف حالة الاستخدام	UC-06
اسم حالة الاستخدام	(Create Service Request) إنشاء طلب خدمة جديد
الفاعل الرئيسي	(User / Client) المستخدم المستفيد
الفاعلون الثانويون	(File Storage System) نظام التخزين
الوصف المختصر	تتيح هذه العملية للمستخدم تقديم طلب خدمة جديد في قسم فرعي محدد، والإجابة على الأسئلة الديناميكية المخصصة لهذا القسم، مع إمكانية إرفاق صور كإجابات إذا كان نوع السؤال يتطلب ذلك.
الشروط المسبقة	1. (Authenticated User) أن يكون المستخدم مسجلاً ومسجلاً دخوله إلى النظام بنجاح. 2. معرفاً مسبقاً في النظام ويحتوي (category_id) أن يكون القسم الفرعي للخدمة. على الأسئلة الخاصة به
الشروط اللاحقة	1. يتم حفظ طلب الخدمة الجديد في قاعدة البيانات وتعيين حالته الافتراضية ك قيد (status = pending). 2. (public/requests) يتم رفع وحفظ الصور المرفقة داخل مجلد التخزين العام. 3. لضمان تحديث (available_requests) يتم تفريغ الكاش الخاص بالطلبات المتاحة. البيانات لدى مقدمي الخدمات.
المسار الأساسي (Basic Flow)	1. يقوم الفاعل باختيار القسم الفرعي للخدمة وإرسال الإجابات على الأسئلة المطروحة مع ملفات الصور إن وجدت. 2. يتحقق النظام من صحة البيانات الأساسية للطلب (وجود القسم، مصفوفة الإجابات، والتحقق من صيغ وحجم الصور بشكل عام). 3. (in_progress أو pending) يتحقق النظام من عدم وجود طلب مفتوح (بحالة لنفس المستخدم وفي نفس القسم). 4. "يتأكد النظام من أن المعرف يخص "قسماً فرعياً" وليس "قسماً رئيسياً". 5. يجلب النظام الأسئلة المرتبطة بالقسم ويبدأ بالتحقق منها واحداً تلو الآخر (is_required). أ. يتأكد من إجابة الأسئلة المطلوبة ، يقوم بحفظ الملف/الملفات في (type == 'image') ب. إذا كان نوع السؤال صورة (Path) التخزين ويستبدل الملف بمسار الحفظ الإشاري. ج. إذا كان نوعاً آخر، يتم التحقق من صحة نوع الإجابة عبر الدالة المخصصة لذلك الخاص بالطلبات المتاحة لإنعاش النظام Cache يقوم النظام بحذف الـ 6. 7. يقوم النظام بإنشاء وحفظ سجل الطلب الجديد وتخزين الإجابات بصيغتها النهائية. 8. يعيد النظام استجابة بنجاح العملية مع بيانات الطلب المنشأ.
المسارات البديلة والاستثناءات (Alternative / Exception Flows)	في حال نقص البيانات الأساسية أو عدم [Validation Error - خطأ في التحقق الأولي] ، يعيد (والصيغ المدعومة KB الحجم الأقصى 2048) مطابقة ملفات الصور للشروط (HTTP 422) النظام خطأ. إذا تبين أن المستخدم لديه [Duplicate Open Request - وجود طلب نشط مسبقاً] طلب في الانتظار أو قيد التنفيذ لنفس الخدمة، يوقف النظام العملية ويعيد رسالة: "لديك طلب مفتوح بالفعل لهذه الخدمة". إذا حاول الفاعل تقديم الطلب لقسم رئيسي: [اختيار قسم رئيسي بدلاً من الفرعي] ("must be مباشرة، يرفض النظام العملية ويعيد رسالة تفيد بوجوب اختيار قسم فرعي subcat"). في حال تخطي سؤال محدد كإلزامي من لوحة التحكم، يتوقف: [عدم إجابة سؤال إلزامي] "النظام ويعيد رسالة تفيد بأن: "السؤال {اسم السؤال} مطلوب".

في حال كان السؤال يتطلب صورة كإجابة ولم :[فقر إرفاق صور لسؤال من نوع صورة] يقوم المستخدم بإرفاق ملف صالح، يرمي النظام استثناءً يوضح أن "الصورة مطلوبة لهذا السؤال".

(Field) الخاصية	(Description) الوصف
معرف حالة الاستخدام	UC-07
اسم حالة الاستخدام	(Rate Service Offer) تقييم عرض الخدمة ومقدم الخدمة
الفاعل الرئيسي	(User / Client) المستخدم المستفيد
الفاعل الثانويون	لا يوجد (نظام قاعدة البيانات فقط)
الوصف المختصر	تتيح هذه العملية للمستخدم (صاحب الطلب) وضع تقييم رقمي وكتابة مراجعة نصية لعرض الخدمة المنفذ بعد إتمام عملية الدفع، مما يساهم في تحديث التقييم العام لمقدم الخدمة وإغلاق العرض رسمياً.
الشروط المسبقة	(Authenticated) أن يكون المستخدم مسجلاً ومسجلاً دخوله إلى النظام بنجاح. 2. موجوداً بالفعل في قاعدة البيانات (offerId) أن يكون عرض الخدمة. 3. (status = 'waiting_for_rating') أن تكون الخدمة قد دُفعت وتحولت حالتها إلى بانتظار التقييم.
الشروط اللاحقة	1. يتم إنشاء سجل تقييم جديد يربط المستخدم بمقدم الخدمة وبالعرض الفعلي. 2. في النظام (Provider) يتم إعادة حساب وتحديث متوسط التقييم العام لمقدم الخدمة. 3. (status = 'closed') يتم تحديث حالة عرض الخدمة لتصبح مغلقة نهائياً.
المسار الأساسي (Basic Flow)	1. وقيمة التقييم (offerId) يقوم الفاعل بإرسال طلب التقييم متضمناً معرف العرض. 2. إن وجد (Review) (عدد النجوم)، والتعليق النصي. 3. يتحقق النظام من صحة البيانات والقيود (التقييم الرقمي مطلوب وهو رقم صحيح بين 1 و 5، والتعليق اختياري بحد أقصى 300 حرف). 4. يجلب النظام بيانات عرض الخدمة المستهدف. 5. "يتحقق النظام من أن حالة العرض الحالية هي "بانتظار التقييم (waiting_for_rating)." (waiting_for_rating). 6. يتحقق النظام من أن المستخدم الذي يطلب التقييم هو نفسه صاحب طلب الخدمة. 7. الأصلي المرتبط بهذا العرض. 8. يتأكد النظام من عدم وجود تقييم مسبق تم تقديمه لهذا العرض تحديداً. 9. يقوم النظام بإنشاء سجل التقييم الجديد وحفظه (يحتوي على معرف العرض، والنص). 10. يقوم النظام باستدعاء الدالة المخصصة لتحديث متوسط التقييم الخاص بمقدم الخدمة. 11. في حسابه بناءً على التقييم الجديد. 12. (closed) يقوم النظام بتحديث حالة العرض إلى مغلق. 13. (HTTP 200) يعيد النظام استجابة بنجاح العملية مع بيانات التقييم التي تم حفظها.
المسارات البديلة والاستثناءات (Alternative / Exception Flows)	في حال كان التقييم خارج النطاق: [Validation Error - خطأ في التحقق من المدخلات] (أقل من 1 أو أكثر من 5) أو نص المراجعة أطول من 300 حرف، يرفض النظام الطلب (HTTP 422) ويعيد خطأ. إذا حاول الفاعل تقييم عرض لم يصل لحالة: [الخدمة غير مدفوعة أو غير جاهزة للتقييم] "Service not paid yet": الانتظار بعد (لم يتم الدفع مثلاً)، يوقف النظام العملية ويرمي استثناءً برسالة "Service not paid yet". إذا تبين أن معرف الفاعل (المستخدم): [Unauthorized - محاولة تقييم غير مصرح بها] (الحالي) لا يطابق معرف صاحب الطلب الأصلي المرتبط بهذا العرض، يتم إيقاف العملية "Unauthorized": فوراً ويرمي استثناءً برسالة "Unauthorized". إذا وجد النظام تقييماً مسجلاً في قاعدة البيانات: [Already Rated - العرض مقيم مسبقاً] "Already rated": لنفس معرف العرض، يتم منع تكرار التقييم ويرمي استثناءً برسالة "Already rated".

(Field) الخاصية	(Description) الوصف
معرف حالة الاستخدام	UC-08
اسم حالة الاستخدام	(Create Service Offer) تقديم عرض خدمة
الفاعل الرئيسي	(Provider) مقدم الخدمة
الفاعلون الثانويون	لا يوجد (نظام قاعدة البيانات فقط)
الوصف المختصر	تتيح هذه العملية لمقدم الخدمة تصفح طلبات الخدمات المتاحة وتقديم عرض سعر يشتمل على حد أدنى وحد أعلى للتكلفة المتوقعة مع رسالة اختيارية توضيحية للمستفيد.
الشروط المسبقة	(Authenticated). أن يكون مقدم الخدمة قد سجل دخوله للحساب بنجاح. 2. موجوداً ونشطاً في النظام (service_request_id) أن يكون طلب الخدمة المستهدف. 3. ألا يكون مقدم الخدمة قد أرسل عرضاً سابقاً لنفس الطلب.
الشروط اللاحقة	1. يتم إنشاء سجل عرض سعر جديد وحفظه برقم طلب الخدمة ومُعرّف مقدم الخدمة. 2. يتم حذف وتحديث الكاش المرتبط بعروض مقدم الخدمة المعني لإنعاش البيانات. المحدثة لديه
المسار الأساسي	1. يقوم الفاعل (مقدم الخدمة) باختيار طلب خدمة معين وإدخال تفاصيل العرض: الحد (Message). الأدنى للسعر، الحد الأعلى للسعر، ورسالة توضيحية. 2. يتحقق النظام من صحة البيانات والقيود المفروضة على المدخلات. أ. وجود طلب الخدمة الأصلي في قاعدة البيانات أقل من أو يساوي min_price. كون الأسعار مدخلة بقيم رقمية منطقية (بحيث يكون max_price). 3. (provider) يتحقق النظام من دور الفاعل (التحقق من أن صلاحية الحساب هي). 4. يفحص النظام السجلات للتأكد من عدم وجود عرض سابق تم تقديمه بواسطة هذا الفاعل على نفس الطلب. 5. يقوم النظام باستدعاء دالة لتفريغ الكاش الخاص بعروض مقدم الخدمة الحالي (clearOfferCache). 6. يتم حفظ سجل العرض الجديد متضمناً الأسعار المقترحة والرسالة في قاعدة البيانات برتبة افتراضية أو حالة أولية. محملة ببيانات العرض الذي تم (HTTP 200) يعيد النظام استجابة بنجاح العملية. 7. إنشاءه.
(Basic Flow)	
المسارات البديلة والاستثناءات	في حال نقص البيانات الأساسية، [Validation Error - خطأ في التحقق من المدخلات] أو إذا كان الحد الأدنى للسعر أعلى من الحد الأعلى، أو تخطت الأسعار القيمة القصوى (HTTP 422) المسموحة في التحقق، يرفض النظام الطلب ويعيد خطأ. أو (Client) إذا حاول مستخدم عادي [Invalid Role - صلاحية غير صالحة للعملية] ، يوقف النظام العملية فوراً ويرمي API دور آخر غير مقدم الخدمة استدعاء هذا الـ "استثناء برسالة: "فقط مقدمي الخدمة يمكنهم إرسال عروض إذا وجد النظام أن مقدم الخدمة يمتلك بالفعل [Duplicate Offer - تقديم عرض مكرر] عرضاً مسجلاً لنفس طلب الخدمة، يتم إيقاف العملية ويرمي استثناء يمنع التكرار برسالة: "لقد قمت بإرسال عرض مسبقاً لهذا الطلب
(Alternative / Exception Flows)	

الخاصية (Field)	الوصف (Description)
معرف حالة الاستخدام	UC-09
اسم حالة الاستخدام	(View and Filter Received Offers) استعراض وتصفية العروض المستلمة
الفاعل الرئيسي	(User / Client) المستخدم المستفيد
الفاعل الثانويون	لا يوجد
الوصف المختصر	تتيح هذه العملية للمستخدم استعراض قائمة العروض القادمة من مقدمي الخدمات والخاصة بطلباته النشطة، مع إمكانية تصفيتها وترتيبها حسب السعر، التقييم، أو المسافة الجغرافية. ووقت الوصول المتوقع لبيته أو موقعه.
الشروط المسبقة	1. (Authenticated User) أن يكون المستخدم مسجلاً ومسجلاً دخوله للنظام. 2. أن يمتلك المستخدم طلبات خدمة منشأة مسبقاً، وقام مقدمو خدمات بإرسال عروض عليها.
الشروط اللاحقة	يعيد النظام قائمة مرقمة (10 عروض لكل صفحة) تحتوي على تفاصيل العرض، بيانات مقدم الخدمة (الملف الشخصي، سابقة الأعمال، والتقييمات)، مضافاً إليها المسافة الجغرافية بالدقائق (\$ETA\$) المقدرة والوقت المتوقع للوصول.
المسار الأساسي	يقوم الفاعل بطلب استعراض العروض المقدمة له، مع إمكانية تمرير محددات تصفية (مثل: الحد الأدنى والأعلى للسعر، التقييم، طلب الترتيب، أو التحقق من هوية مقدم الخدمة). يتحقق النظام من صحة صياغة المدخلات الاختيارية (أن تكون المدخلات رقمية والتقييم بين 1 و 5). 3. (lat, lng) يجلب النظام إحداثيات موقع المستخدم الحالي. 4. (status = pending) يبني النظام استعلاماً لقاعدة البيانات لجلب العروض قيد الانتظار. والمرتبطة بطلبات هذا المستخدم حصراً ('pending'). يقوم النظام بحساب المسافة الجغرافية المباشرة: إذا توفرت إحداثيات الفاعل الجغرافية. ديناميكياً لكل مقدم خدمة بناءً (ETA) (بالكيلومترات) والوقت المتوقع للوصول بالدقائق على موقعه المسجل. 6. يطبق النظام الفلاتر الممررة: (min_price, max_price) أ. تصفية العروض بناءً على نطاق الأسعار. (is_verified) ب. تصفية مقدمي الخدمة الموثقين فقط. ج. تصفية مقدمي الخدمة الحاصلين على تقييم أعلى أو يساوي التقييم المطلوب. حسب المسافة تصاعدياً، السعر (sort_by) يطبق النظام آلية الترتيب المحددة. 7. (تصاعدي/تنازلي)، أو التقييم تنازلياً. (وفي حال عدم تحديد نوع الترتيب وتوفر الموقع، يتم (الترتيب حسب المسافة تصاعدياً كوضع افتراضي). يقسم النظام النتائج لصفحات (كل صفحة تحتوي على 10 سجلات). 8. يدمج النظام بيانات السجلات مع العلاقات الكاملة لمقدم الخدمة (الملف، التقييمات، سابقة الأعمال والطلب الأصلي). 9. يعيد النظام النتائج بصيغة دائرية منسقة ومقربة رقمياً للمسافة والوقت المتوقع. 10. في حال تمرير قيم غير رقمية: [Validation Error - خطأ في التحقق من المدخلات] (HTTP 422) للفلاتر أو قيم تقييم خارج نطاق (1-5)، يرفض النظام الطلب ويعيد خطأ في حال طلب الفاعل ترتيب النتائج بناءً على: [محاولة ترتيب بالمسافة بدون تحديد الموقع] ولكن حسابه لا يحتوي على إحداثيات موقع مسجلة (sort_by = distance) ""المسافة فارغة"، يتم إيقاف الاستعلام ويرمي النظام استثناءً يمنع العملية برسالة: (lat أو lng) (HTTP 400/500) ""يجب تحديد موقعك لاستخدام الترتيب حسب المسافة
المسارات البديلة والاستثناءات	
(Alternative / Exception Flows)	

الخاصية (Field)	الوصف (Description)
معرف حالة الاستخدام	UC-10
اسم حالة الاستخدام	بدء عملية دفع عرض الخدمة (Initiate Offer Payment)
الفاعل الرئيسي	المستخدم (User / Client)
الفاعلون الثانويون	نظام الدفع أو خادم بوابات التحويل
الوصف المختصر	تمكن هذه العملية المستخدم من إنشاء طلب دفع لعرض خدمة تمت الموافقة عليه، حيث يقوم النظام بالتحقق من القيمة المدخلة وتوليد رمز مرجعي فريد يلتزم المستخدم بكتابته عند إجراء التحويل المالي الخارجي (Reference Number).
الشروط المسبقة	1. (Authenticated User) أن يكون المستخدم مسجلاً ومسجلاً دخوله للنظام. 2. أن يكون عرض الخدمة المستهدف متواجد في النظام وحالته الحالية هي "بانتظار الدفع" (status = 'awaiting_payment').
الشروط اللاحقة	1. (status) "يتم إنشاء سجل دفع جديد في قاعدة البيانات بالحالة الافتراضية "قيد الانتظار" (status = 'pending'). 2. يتم توليد كود مرجعي فريد وإرجاعه للمستخدم لإتمام عملية التحويل بناءً عليه.
المسار الأساسي	1. المبلغ المراد (offer) يقوم الفاعل بطلب دفع العرض، ويرسل كلاً من: معرف العرض ، ورقم الحساب البنكي أو المحفظة المحوّل منها (amount) دفعه (account_number). 2. يتحقق النظام من صحة البيانات الأساسية (أن يكون المبلغ قيمته 1 أو أكثر ورقم الحساب رقمياً). 3. يجلب النظام بيانات عرض الخدمة، ويتأكد من أن حالة العرض الحالية هي "بانتظار الدفع" (awaiting_payment). 4. يطابق النظام بين المبلغ المرسل من الفاعل والسعر النهائي المتفق عليه والمخزن في العرض (final_price). 5. يقوم النظام بتوليد رمز مرجعي تلقائي وفريد للعملية بالصيغة الفنية: PAY-{offer_id}-{random_number}. 6. يقوم النظام بإنشاء سجل جديد في جدول المدفوعات برقم العرض، ورقم الطلب، وتعيين الحالة كـ (SYP) ومعرفات المستخدم ومقدم الخدمة، ومبلغ الدفع بالعملة المحلية ، "قيد الانتظار". 7. مصحوبة بالرقم المرجعي والمبلغ (HTTP 200) يعيد النظام استجابة بنجاح العملية. "في الملاحظة reference ورسالة توجيهية للمستخدم: "قم بالتحويل واكتب".
المسارات البديلة والاستثناءات	في حال ترك حقل المبلغ أو رقم [Validation Error - خطأ في التحقق من المدخلات] (HTTP 422) الحساب فارغاً أو إدخال قيم غير رقمية، يرفض النظام الطلب ويعيد خطأ إذا كانت حالة العرض الحالية ليست: [Offer Not Ready - العرض غير جاهز للدفع] كأن يكون العرض ملغياً، أو مسودّة، أو تم دفعه مسبقاً، يتوقف (awaiting_payment "Offer not ready for payment": النظام ويرمي استثناء يمنع العملية برسالة إذا قام الفاعل بإدخال مبلغ لا يطابق تماماً: [Incorrect Amount - عدم مطابقة المبلغ] السعر النهائي المعتمد في جدول العروض، يتوقف النظام فوراً ويرمي استثناء لمنع "التلاعب المالي برسالة: "ادخل المبلغ الصحيح".
(Alternative / Exception Flows)	

الخاصية (Field)	الوصف (Description)
معرف حالة الاستخدام	UC-11
اسم حالة الاستخدام	(Verify Payment) التحقق من ترحيل الدفع
الفاعل الرئيسي	أو المستخدم عند طلب التحديث / (Automated System) النظام
الفاعل الثانويون	بوابة الدفع الخارجية (ShamCash API)
الوصف المختصر	للتأكد من وصول الحوالة المالية الخاصة ShamCash يقوم النظام بالاتصال ببوابة دفع بالمستفيد ومطابقة بياناتها، وفي حال نجاحها يتم تفعيل رصيد مقدم الخدمة، وتغيير حالة العرض، وإغلاق المحادثة الجارية بين الطرفين.
الشروط المسبقة	(status = 'pending') "أن تكون عملية الدفع منشأة مسبقاً في النظام بحالة "قيد الانتظار". الخاص Token الرابط والـ ShamCash أن تتوفر بيانات الاتصال البرمجية ببوابة. (بالحساب البنكي للتطبيق).
الشروط اللاحقة	(status = 'paid'). تتحول حالة الدفع إلى مدفوع. (status = 'waiting_for_rating'). تتحول حالة عرض الخدمة إلى بانتظار التقييم. يتم زيادة الرصيد المالي في محفظة مقدم الخدمة بمقدار قيمة الطلب (wallet_balance). الخاصة بطلب الخدمة نهائياً (Chat Conversation) يتم إغلاق نافذة المحادثة.
المسار الأساسي (Basic Flow)	يتم استدعاء معرف الدفع المستهدف للتحقق منه. (pending) "يتأكد النظام من أن حالة الدفع الحالية لا تزال "قيد الانتظار". مستخدماً ShamCash إلى خوادم بوابة (HTTP Request) يقوم النظام بإرسال طلب. لإحضار قائمة بأخر 50 عملية تحويل مالي مسجلة API Token الـ يمر النظام عبر قائمة العمليات المستلمة من البوابة للبحث عن عملية مطابقة للشروط. التالية: أ. أن يتطابق مبلغ التحويل في البوابة مع المبلغ المسجل في النظام بالليرة السورية (amount_syp). في البوابة على الرقم المرجعي الفريد للنظام (note) ب. أن تحتوي ملاحظة التحويل (reference). ج. أن يكون وقت حدوث التحويل في البوابة لاحقاً لوقت إنشاء طلب الدفع في التطبيق. paid. عند العثور على العملية المطابقة بنجاح، يقوم النظام بتحديث حالة الدفع إلى. waiting_for_rating. يتم تحديث حالة عرض الخدمة المرتبط إلى. يقوم النظام بزيادة رصيد محفظة مقدم الخدمة بقيمة المبلغ المدفوع مباشرة. لإغلاق الغرفة والمحادثة الخاصة (ChatService) يستدعي النظام خدمة المحادثات. بهذا الطلب. مع تفاصيل سجل الدفع (HTTP 200) يعيد النظام استجابة بنجاح التوثيق والإغلاق. الجديد.
المسارات البديلة والاستثناءات (Alternative / Exception Flows)	pending إذا تبين عند فحص حالة الدفع في البداية أنها ليست :[العملية معالجة مسبقاً] Payment "تم التحقق منها وتأكيد مسبقاً"، يوقف النظام الفحص ويعيد رسالة (HTTP 200) already processed". في حال فحص النظام لجميع الحركات الـ 50 الأخيرة :[عدم العثور على الحوالة المالية] ولم يجد أي حركة تطابق المبلغ والمرجع المحدد للطلب، ShamCash المستلمة من بوابة تتوقف العملية ويعيد النظام رسالة خطأ: "لم يتم العثور على عملية الدفع" مع رمز الحالة (HTTP 400).

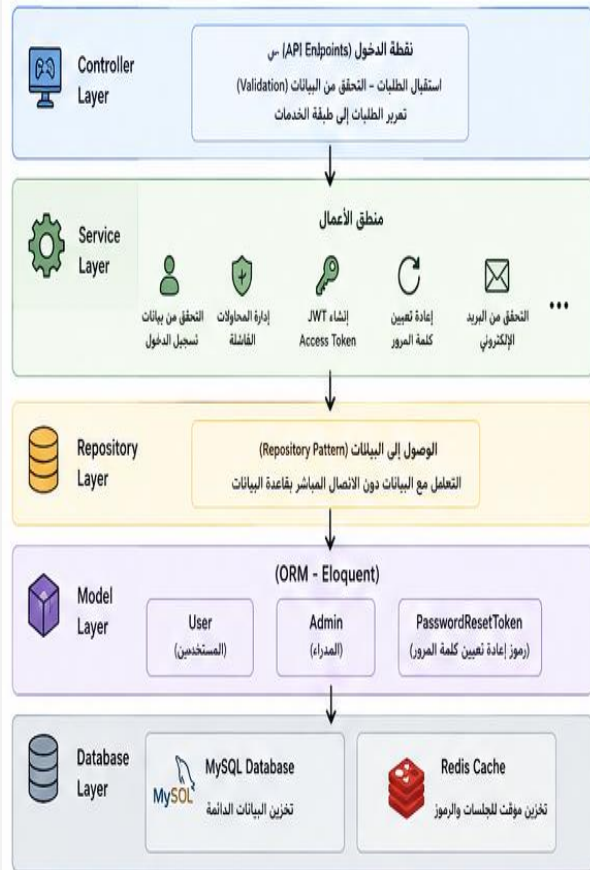
الخاصية (Field)	الوصف (Description)
معرف حالة الاستخدام	UC-12
اسم حالة الاستخدام	(Smart Provider Recommendation) اقتراح مقدمي الخدمات الذكي
الفاعل الرئيسي	(User / Client) المستخدم المستفيد
الفاعلون الثانويون	لا يوجد (محرك الحسابات الجغرافية بالنظام)
الوصف المختصر	يقوم النظام بترشيح أفضل 10 مقدمي خدمات للمستخدم في تصنيف معين بناءً على خوارزمية ذكية توازن بين جودة تقييمهم، خبرتهم (عدد العمليات المنفذة)، ومدى قربهم الجغرافي من موقع المستخدم الحالي.
الشروط المسبقة	1. (Authenticated) أن يكون المستخدم مسجلاً ومسجلاً دخوله. 2. (lat, lng) أن يمتلك المستخدم إحداثيات موقع دقيقة ومسجلة بحسابه. 3. موجوداً ونشطاً في النظام (category_id) أن يكون تصنيف الخدمة المطلوب.
الشروط اللاحقة	يعيد النظام قائمة مرتبة تنازلياً بأعلى مقدمي الخدمات كفاءة وقرباً، مع تخزين النتيجة مؤقتاً في ذاكرة الكاش السريعة لمدة 10 دقائق لتسريع الاستجابة اللاحقة.
المسار الأساسي (Basic Flow)	يرسل الفاعل طلباً للحصول على توصية بمقدمي خدمات في تصنيف معين (category_id). يُتحقق النظام من صحة ووجود معرف التصنيف في قاعدة البيانات. يفحص النظام إحداثيات المستخدم الحالية، وإذا لم تكن متوفرة يوقف العملية. يبحث النظام في ذاكرة الكاش عن مفتاح خاص بـ (المستخدم الحالي + التصنيف المستهدف + رقم الصفحة الحالية). يتم جلب القائمة مباشرة وإرجاعها (Cache Hit): في حال وجود البيانات في الكاش. للمستفيد فوراً واختصار بقية الخطوات. يقوم النظام بالاستعلام المباشر (Cache Miss): في حال عدم وجود البيانات في الكاش. ويبدأ بتطبيق المعادلة الحسابية التالية لجميع مقدمي الخدمات المدرجين تحت هذا (provider): $Score = (rating_avg \times 0.5) + (jobs_count \times 0.2) - (distance \times 0.3)$ بناءً على (ETA\$) يقوم النظام بحساب المسافة الجغرافية والوقت المتوقع للوصول. معادلة هافرسين الجغرافية. من خلال عدّ عروض الأسعار المسجلة لهذا المقدم jobs_count يتم حساب حقل (paid). "وحالتها" مدفوعة. (score) يرتب النظام النتائج تنازلياً بناءً على قيمة النقاط الإجمالية المستخرجة. يحفظ النظام النتيجة الجديدة في الكاش بمدة صلاحية مقدارها 10 دقائق (600 ثانية). يعيد النظام القائمة بنظام الترتيب (بحد أقصى 10 سجلات في الصفحة) مضافاً إليها 11. ETA\$. إجمالي النقاط والمسافة والد.
المسارات البديلة والاستثناءات (Alternative / Exception Flows)	إذا لم يتم إرسال معرف التصنيف: [Validation Error - خطأ في التحقق من المدخلات] (HTTP 422) أو كان غير موجود بقاعدة البيانات، يعيد النظام خطأ. إذا حاول المستخدم طلب التوصية الذكية وحسابه: [الموقع الجغرافي للمستخدم غير متوفر] قيم فارغة، يتم إيقاف خوارزمية (lat lng) لا تحتوي على قيم خطوط الطول والعرض. (HTTP 400) "Location required": الترشيح فوراً ويرمي النظام استثناءً برسالة.

(Field) الخاصية	(Description) الوصف
معرف حالة الاستخدام	UC-13
اسم حالة الاستخدام	(Update Support Ticket Status) تحديث حالة التذكرة / الشكوى
الفاعل الرئيسي	(Support Agent / Admin) موظف الدعم الفني / المسؤول
الفاعل الثانويون	لا يوجد
الوصف المختصر	تتيح هذه العملية لموظف الدعم الفني تغيير حالة تذكرة الدعم الحالية (مثال: من قيد الانتظار إلى قيد المعالجة أو مغلقة) مع ضمان عدم تضارب التعديلات في حال محاولة موظفين آخرين تعديل نفس التذكرة في نفس اللحظة.
الشروط المسبقة	1. (Admin / Support Agent) أن يكون المستخدم مسجلاً دخوله للنظام ويملك الصلاحيات الكافية. 2. متواجدة بالفعل في قاعدة البيانات (ticketId) أن تكون التذكرة المستهدفة. 3. من قبل واجهة المستخدم (version) إرسال رقم النسخة الحالية للتذكرة.
الشروط اللاحقة	1. للتذكرة في قاعدة البيانات (status) تحديث حقول الحالة. 2. (version = version + 1) زيادة قيمة حقول النسخة بمقدار واحد تلقائياً. 3. مسح كاش الرسائل المرتبطة بهذه التذكرة لضمان إنعاش البيانات.
المسار الأساسي (Basic Flow)	1. يقوم الفاعل بإدخال الحالة الجديدة للتذكرة، ويرسل النظام طلب التحديث متضمناً معرف (version)، ورقم النسخة المقروءة حالياً (status)، الحالة الجديدة (ticketId) التذكرة يتحقق النظام من صحة البيانات الممررة عبر حزمة التحقق المخصصة. (UpdateTicketStatusRequest). 3. يحاول النظام تحديث سجل التذكرة بشروطين: أ. تطابق معرف التذكرة ب. تطابق رقم النسخة المرسل مع رقم النسخة المخزن حالياً في قاعدة البيانات عند نجاح المطابقة، يقوم النظام بتغيير الحالة وزيادة رقم النسخة برمجياً بمقدار 1 عبر 4. (DB::raw('version + 1')). استعلام مباشر يقوم النظام بمسح وحذف ذاكرة الكاش المخصصة لرسائل هذه التذكرة عبر استدعاء 5. المفتاح التلقائي الخاص بها 6. (JSON (HTTP 200) يجلب النظام السجل المحدث للتذكرة ويعيده في الاستجابة بصيغة
المسارات البديلة والاستثناءات (Alternative / Exception Flows)	إذا حاول موظف الدعم الفني: [Concurrency Conflict - خطأ تضارب البيانات والنسخ] في قاعدة version تعديل تذكرة تم تعديلها بالفعل من قبل زميل آخر (مما يعني أن رقم الـ الممرر في الطلب)، يفشل استعلام التحديث version البيانات قد ازداد ولم يعد يطابق الـ ويرمي النظام استثناءً برسالة: "تم تعديل الشكوى من قبل شخص آخر" لمنع تضارب البيانات وتخريب العمل.

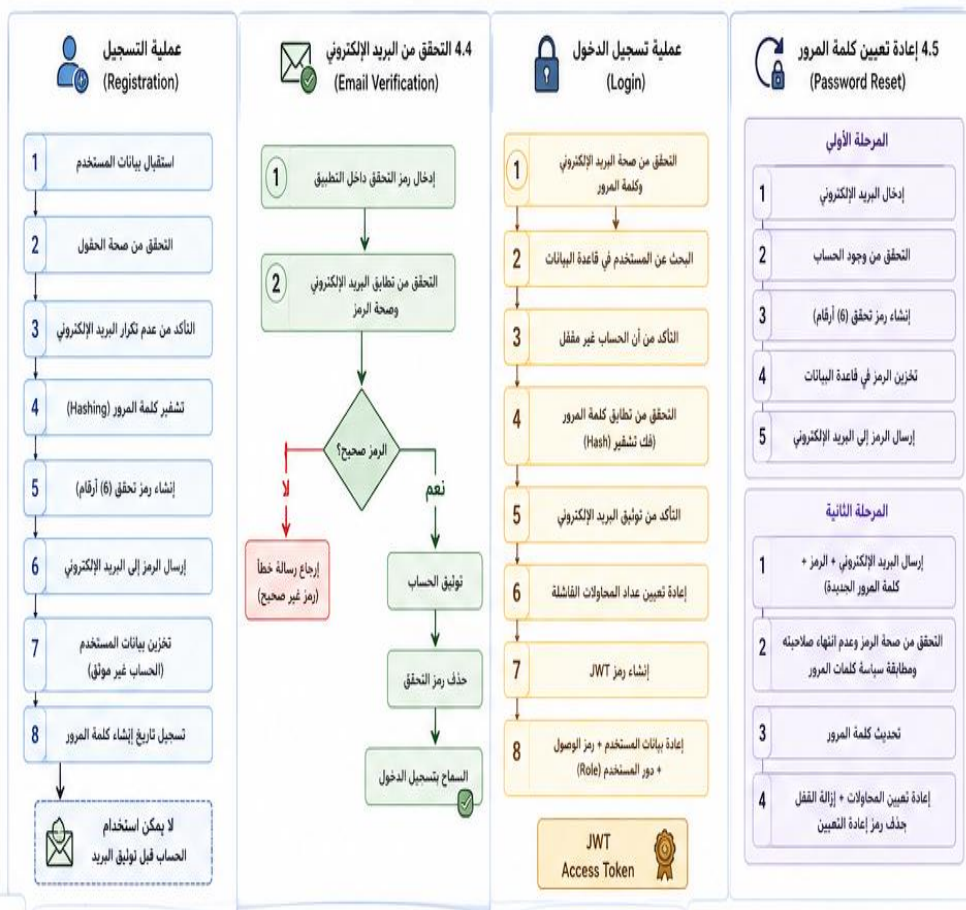
الفصل الرابع: المعمارية البرمجية والوحدات التفصيلية للأنظمة الخلفية (Backend)

4. Authentication and Authorization Module

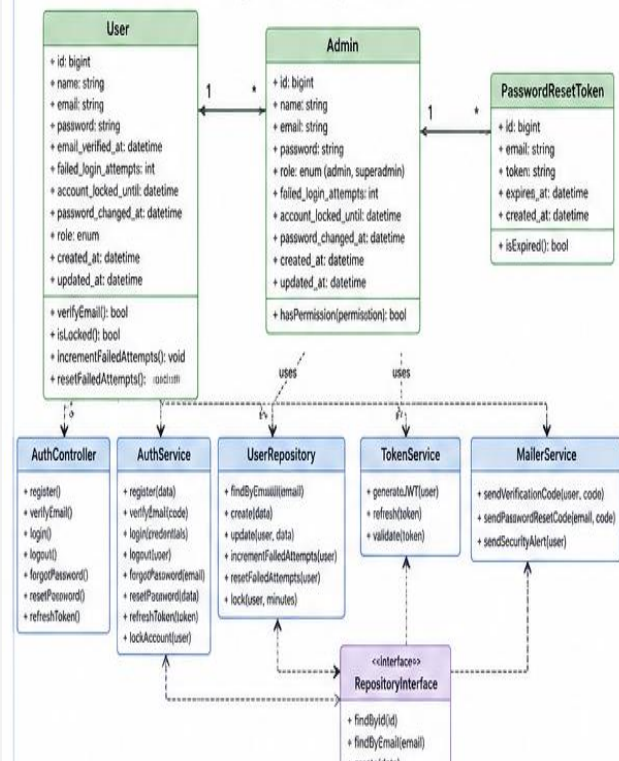
4.2 (Authentication Module Architecture)



4.3-4.7 العمليات الرئيسية في وحدة المصادقة



مخطط الفئات (Class Diagram) لوحدة المصادقة

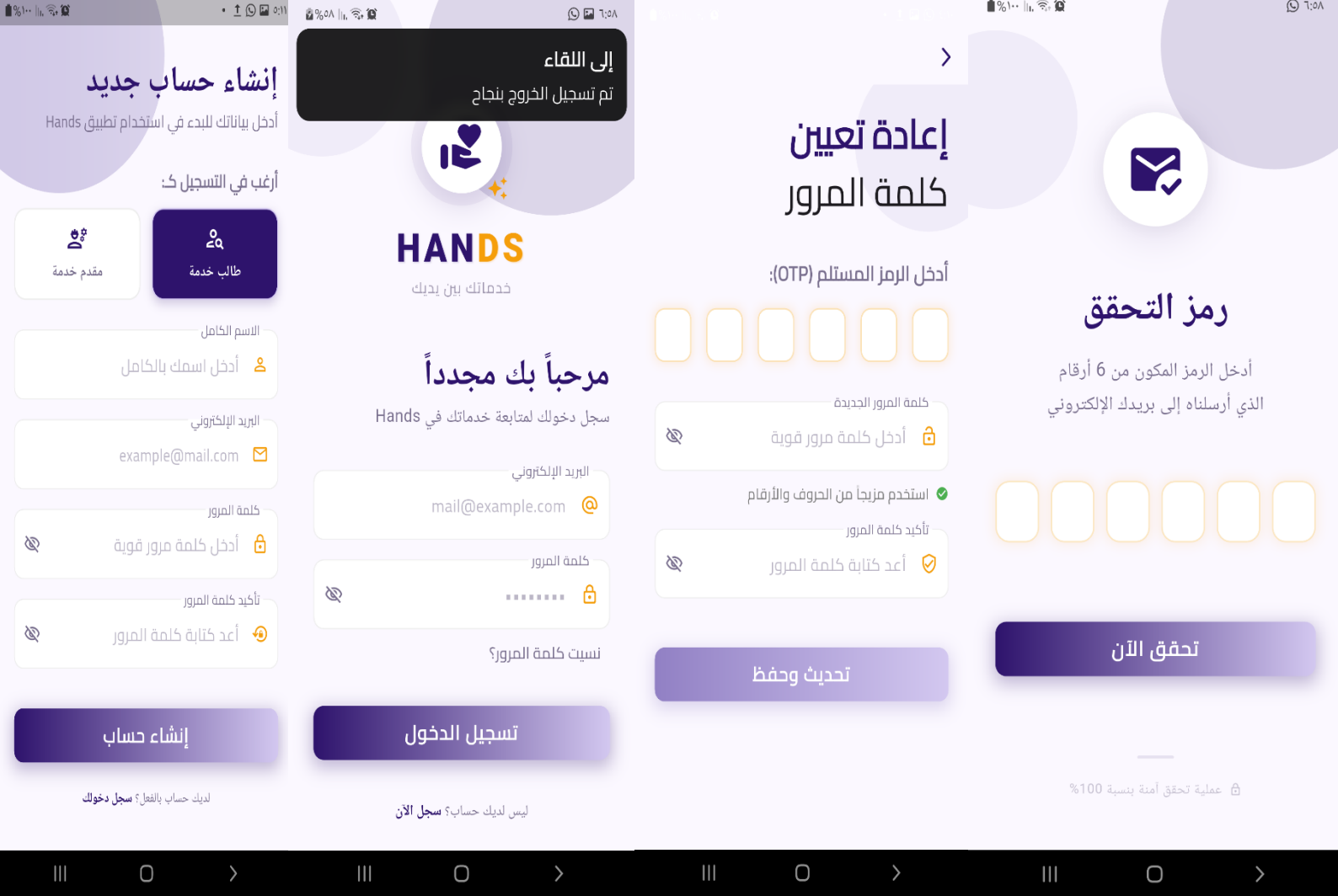


الميزات الأمنية الرئيسية 4.8-4.13



4.14 مبادئ التصميم المعتمدة





4.1 Introduction

تُعد وحدة المصادقة وإدارة الهوية (Authentication and Authorization) من أهم الوحدات الأساسية في النظام، حيث تتولى مسؤولية التحقق من هوية المستخدم قبل السماح له بالوصول إلى الخدمات المختلفة داخل التطبيق. تم تصميم هذه الوحدة وفق مبدأ الفصل بين المسؤوليات (Separation of Concerns)، بحيث يتم فصل منطق الأعمال (Business Logic) عن طبقة التحكم (Controller Layer)، مما يجعل النظام أكثر قابلية للصيانة والتطوير مستقبلاً.

تدعم الوحدة عدة أنواع من المستخدمين داخل النظام، وهم:

- المستخدم العادي (User)
- مقدم الخدمة (Service Provider)
- أمين الصندوق (Cashier)
- مدير النظام (Administrator) (admin+superadmin)

وقد تم تخصيص آلية مصادقة مستقلة لكل نوع مستخدم مع المحافظة على هيكلية موحدة لعملية تسجيل الدخول.

4.2 Architecture

تم بناء نظام المصادقة باستخدام بنية متعددة الطبقات (Layered Architecture)، وتتكون من الطبقات التالية:

Controller Layer

تمثل نقطة الدخول لجميع طلبات واجهات البرمجة (API Endpoints) ، حيث تستقبل الطلبات القادمة من التطبيق، وتتحقق أولاً من صحة البيانات المدخلة (Validation) ، ثم تقوم بتمريرها إلى طبقة الخدمات (Service Layer) دون احتواء أي منطق أعمال.

Service Layer

تحتوي هذه الطبقة على منطق الأعمال الكامل الخاص بعمليات المصادقة، وتشمل:

- التحقق من بيانات تسجيل الدخول.
- إنشاء رمز الوصول (JWT Access Token).
- التحقق من حالة الحساب.
- إدارة عدد محاولات تسجيل الدخول الفاشلة.
- إعادة تعيين كلمة المرور.
- التحقق من البريد الإلكتروني.
- تحديث كلمات المرور.
- إنشاء رموز التحقق وإرسالها عبر البريد الإلكتروني.

إن فصل منطق الأعمال داخل طبقة مستقلة يجعل النظام أكثر قابلية للاختبار (Testability) وإعادة الاستخدام (Reusability).

Repository Layer

تم استخدام Repository Pattern للوصول إلى بيانات المستخدمين بدلاً من التعامل المباشر مع قاعدة البيانات داخل طبقة الخدمات، الأمر الذي يقلل من الترابط (Coupling) بين مكونات النظام ويزيد من مرونته.

Model Layer

تمثل طبقة النماذج الكيانات المخزنة داخل قاعدة البيانات، مثل:

- المستخدمين
- المدراء
- رموز إعادة تعيين كلمة المرور

وتتولى هذه الطبقة التعامل مع قاعدة البيانات باستخدام ORM الخاص بإطار Laravel.

4.3 Registration Process

عند إنشاء حساب جديد، يمر النظام بالمراحل التالية:

1. استقبال بيانات المستخدم.
2. التحقق من صحة جميع الحقول.
3. التأكد من عدم تكرار البريد الإلكتروني.
4. تشفير كلمة المرور باستخدام خوارزمية Hashing.
5. إنشاء رمز تحقق مكون من ستة أرقام.
6. إرسال رمز التحقق إلى البريد الإلكتروني.
7. تخزين بيانات المستخدم مع اعتبار الحساب غير موثق.
8. تسجيل تاريخ إنشاء كلمة المرور لاستخدامه لاحقاً في سياسة تغيير كلمة المرور.

وبذلك لا يمكن استخدام الحساب قبل إتمام عملية توثيق البريد الإلكتروني.

4.4 Email Verification

بعد إنشاء الحساب يقوم النظام بإرسال رمز تحقق (Verification Code) إلى البريد الإلكتروني للمستخدم.

يقوم المستخدم بإدخال الرمز داخل التطبيق، ثم يتحقق النظام من:

- تطابق البريد الإلكتروني.
- صحة رمز التحقق.

وعند نجاح العملية يتم:

- توثيق الحساب.
- حذف رمز التحقق.
- السماح للمستخدم بتسجيل الدخول.

أما في حال إدخال رمز غير صحيح فيتم إرجاع رسالة خطأ مناسبة دون تعديل أي بيانات.

4.5 Login Process

تعتمد عملية تسجيل الدخول على مجموعة من الخطوات الأمنية المتتالية:

1. التحقق من صحة البريد الإلكتروني وكلمة المرور.
2. البحث عن المستخدم داخل قاعدة البيانات.
3. التأكد من أن الحساب غير مقفل.
4. التحقق من تطابق كلمة المرور بعد فك تشفير الـ Hash.
5. التأكد من توثيق البريد الإلكتروني.
6. إعادة تعيين عداد المحاولات الفاشلة عند نجاح تسجيل الدخول.
7. إنشاء رمز JWT.
8. إعادة بيانات المستخدم بالإضافة إلى رمز الوصول.

كما يقوم النظام بإرجاع دور المستخدم (Role) بعد نجاح عملية المصادقة، وذلك لتحديد الصلاحيات المناسبة داخل التطبيق.

4.6 Password Policy

تم اعتماد سياسة قوية لكلمات المرور لضمان رفع مستوى الأمان داخل النظام.

وتشترط كلمة المرور أن:

- لا يقل طولها عن ثمانية أحرف.
- تحتوي على حرف كبير.
- تحتوي على حرف صغير.
- تحتوي على رقم واحد على الأقل.
- تحتوي على رمز خاص.

كما يتم تخزين كلمات المرور بعد تشفيرها باستخدام خوارزمية Hashing ، ولا يتم حفظها بصيغتها النصية داخل قاعدة البيانات.

4.7 Password Reset

في حال نسيان كلمة المرور، يوفر النظام آلية آمنة لإعادة تعيينها.

تتكون العملية من مرحلتين:

المرحلة الأولى

يقوم المستخدم بإدخال بريده الإلكتروني.

يتحقق النظام من وجود الحساب ثم:

- يولد رمز تحقق مكوّن من ستة أرقام.
- يخزن الرمز داخل قاعدة البيانات.
- يرسل الرمز إلى البريد الإلكتروني.

المرحلة الثانية

يقوم المستخدم بإرسال:

- البريد الإلكتروني.
- رمز التحقق.
- كلمة المرور الجديدة.

يقوم النظام بعد ذلك بالتحقق من:

- صحة الرمز.
- عدم انتهاء صلاحيته.
- مطابقة سياسة كلمات المرور.

ثم يتم:

- تحديث كلمة المرور.
- إعادة تعيين عدد المحاولات الفاشلة.
- إزالة حالة قفل الحساب.
- حذف رمز إعادة التعيين من قاعدة البيانات.

4.8 Account Lock Mechanism

لتقليل احتمالية هجمات التخمين (Brute Force Attack)، تم تطبيق آلية قفل الحساب.

تعتمد هذه الآلية على:

- عدد محاولات تسجيل الدخول الفاشلة.
- عند تجاوز خمس محاولات متتالية يتم:
 - قفل الحساب لمدة خمس عشرة دقيقة.
 - إعادة تعيين عداد المحاولات.

كما يقوم النظام بإرسال رسالة تنبيه عبر البريد الإلكتروني للمستخدم لإعلامه بمحاولة دخول غير اعتيادية، مما يرفع من مستوى الأمان ويتيح للمستخدم اتخاذ الإجراءات المناسبة إذا لم يكن هو من حاول تسجيل الدخول.

4.9 Token Management

يعتمد النظام على JSON Web Token (JWT) في إدارة جلسات المستخدمين.

بعد نجاح عملية تسجيل الدخول يتم:

- إنشاء Access Token.
- تحديد مدة صلاحية الرمز.
- إرجاع نوع الرمز Bearer Token.

كما يدعم النظام إعادة توليد رمز جديد (Refresh Token) دون الحاجة إلى إعادة تسجيل الدخول، الأمر الذي يحسن تجربة المستخدم ويزيد من استمرارية الجلسة.

4.10 Password Expiration Policy

يتابع النظام تاريخ آخر تغيير لكلمة المرور.

وعند تجاوز تسعين يوماً منذ آخر تحديث لها، يقوم النظام بإعلام التطبيق بضرورة مطالبة المستخدم بتغيير كلمة المرور، وذلك كإجراء وقائي ينسجم مع أفضل الممارسات الأمنية.

4.11 Input Validation

تم تنفيذ طبقة تحقق كاملة قبل تنفيذ أي عملية داخل النظام، وتشمل:

- التحقق من الحقول الإلزامية.
- التحقق من صيغة البريد الإلكتروني.
- منع تكرار البريد الإلكتروني.
- التحقق من دور المستخدم.
- التحقق من صحة رموز التحقق.
- التحقق من تطابق كلمة المرور مع حقل التأكيد.
- التحقق من قوة كلمة المرور.

يساعد ذلك في منع إدخال بيانات غير صحيحة وتقليل الأخطاء البرمجية وتحسين أمن النظام.

4.12 Exception Handling

تم استخدام معالجة الاستثناءات (Exception Handling) لإدارة الأخطاء الناتجة أثناء تنفيذ عمليات المصادقة.

وتشمل الحالات التالية:

- بيانات تسجيل دخول غير صحيحة.
- حساب غير موثق.
- حساب مقفل مؤقتاً.
- انتهاء صلاحية رمز إعادة التعيين.
- رمز تحقق غير صحيح.
- انتهاء صلاحية رمز المصادقة.

ويتم إرجاع رسائل خطأ مناسبة مع أكواد HTTP القياسية لضمان سهولة تعامل تطبيق الهاتف مع الأخطاء.

4.13 Security Features

تم تضمين مجموعة من آليات الحماية داخل وحدة المصادقة، أهمها:

- تشفير كلمات المرور.
- استخدام JWT Authentication.
- التحقق من البريد الإلكتروني.
- سياسة كلمات مرور قوية.

- قفل الحساب بعد المحاولات الفاشلة.
- إرسال تنبيهات أمنية بالبريد الإلكتروني.
- صلاحية زمنية لرموز إعادة تعيين كلمة المرور.
- تحديث كلمة المرور بشكل دوري.
- التحقق من جميع البيانات المدخلة قبل تنفيذ أي عملية.

4.14 Design Principles

اعتمد تصميم الوحدة على مجموعة من مبادئ هندسة البرمجيات، من أبرزها:

- Layered Architecture.
- Separation of Concerns.
- Dependency Injection.
- Repository Pattern.
- Single Responsibility Principle (SRP).
- Loose Coupling.
- High Cohesion.
- Reusability.
- Maintainability.
- Scalability.

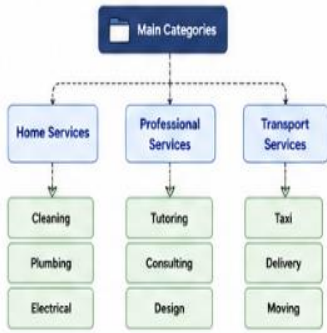
وقد ساهمت هذه المبادئ في إنتاج وحدة مصادقة مرنة، آمنة، وسهلة التطوير والتوسع مستقبلاً.

إدارة طلبات الخدمات 5. (Service Request and Service Management Module)

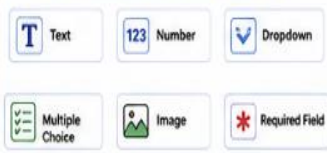
5.2 Service Lifecycle



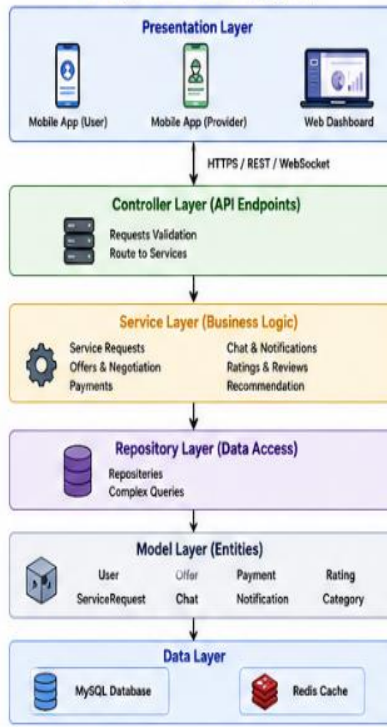
5.3 Service Categories (Hierarchical)



5.4 Dynamic Service Forms (Examples)



5.1 System Architecture (Layered)



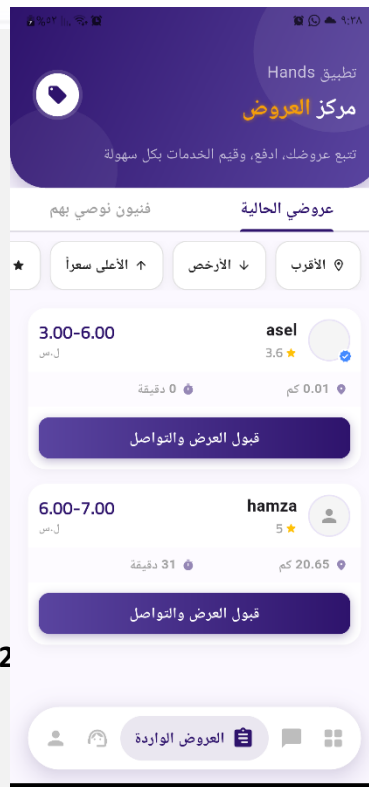
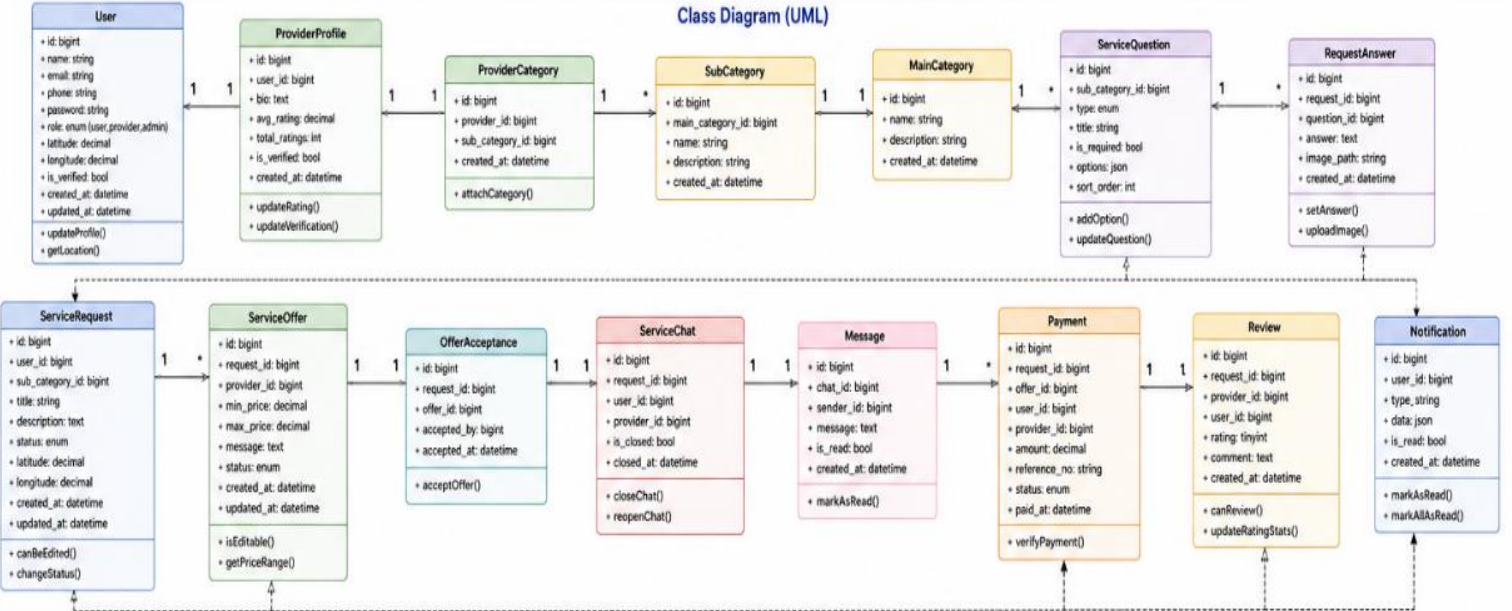
5.5 - 5.16 Key Features Overview

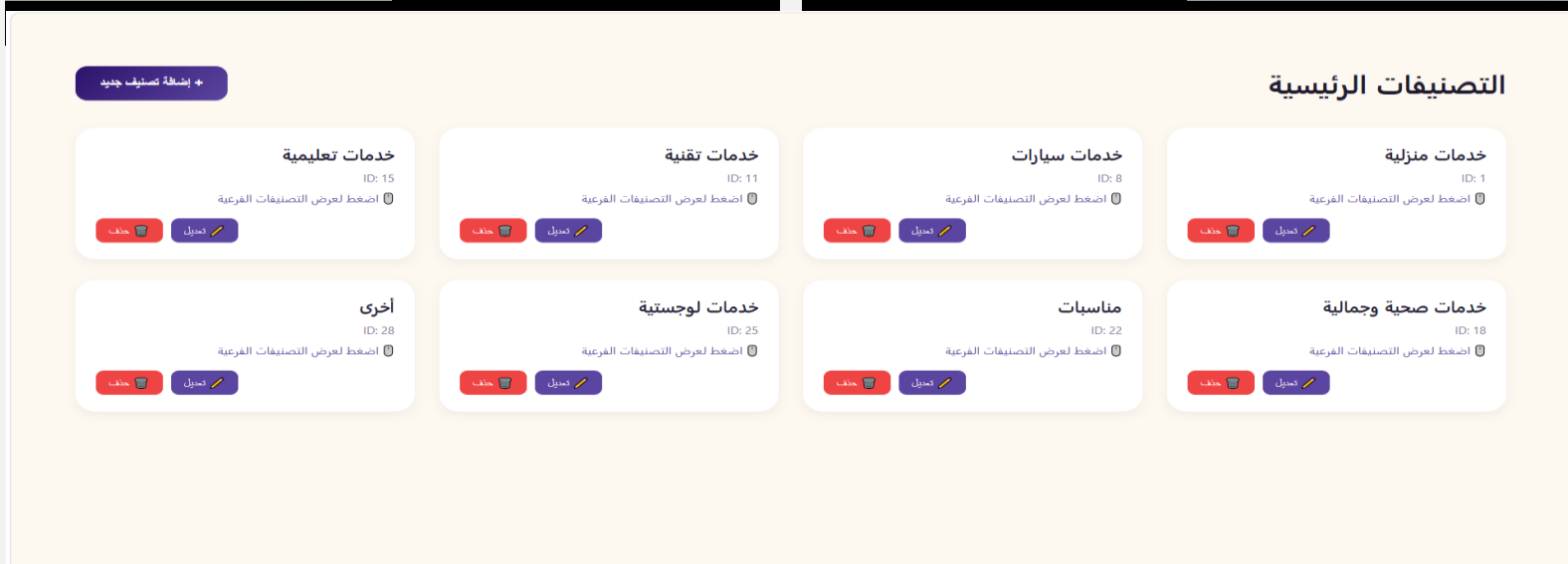
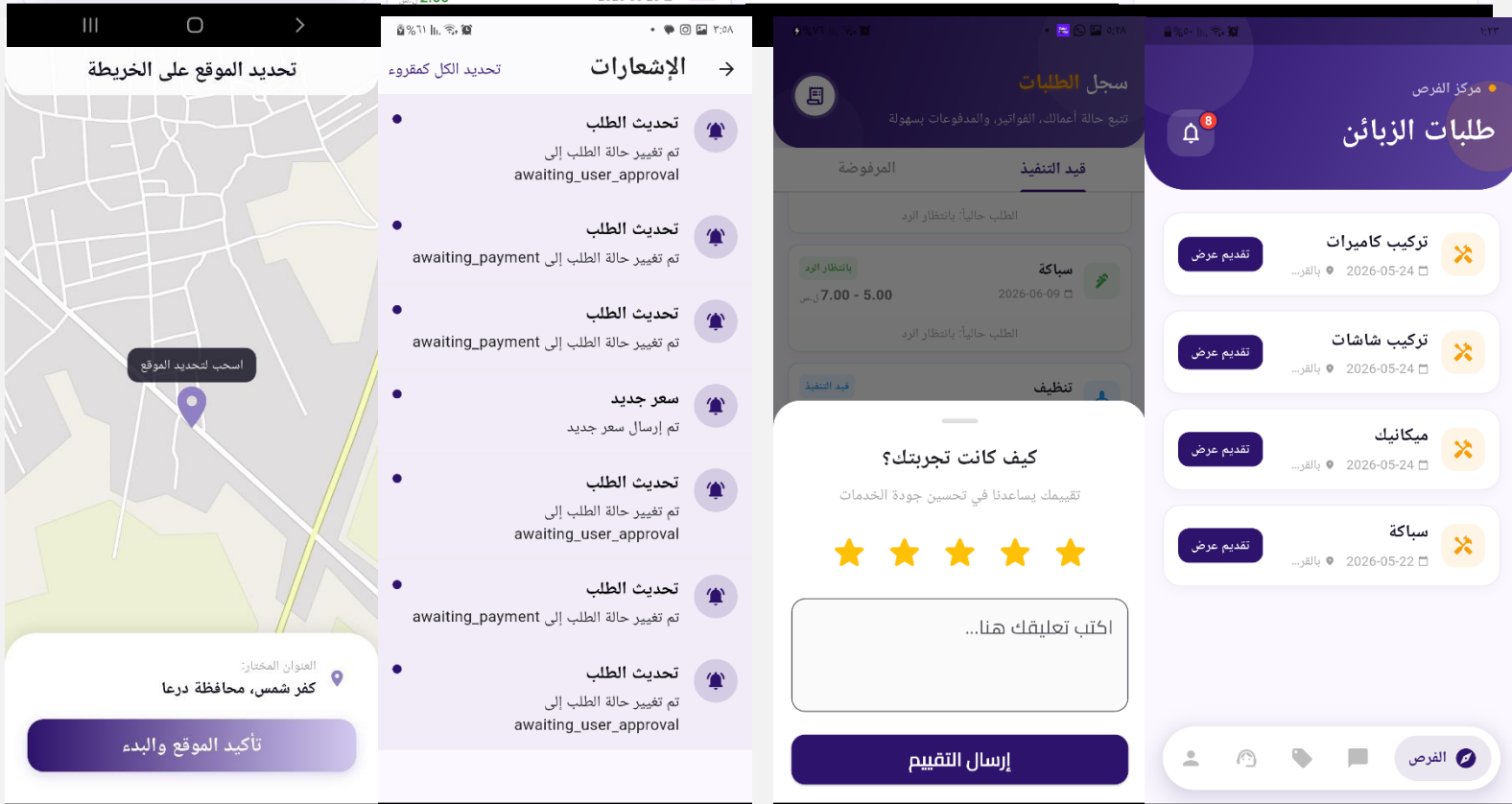
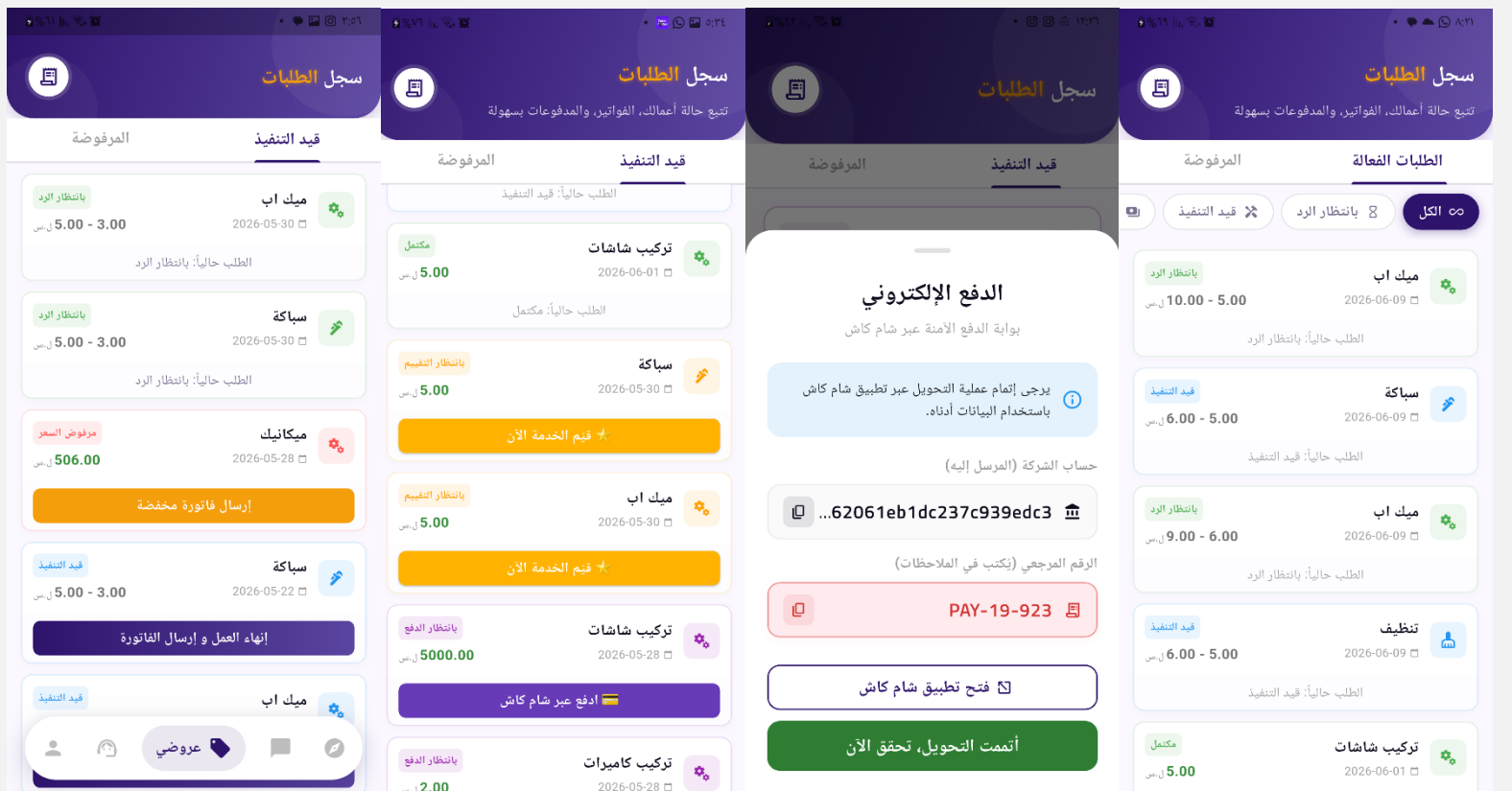


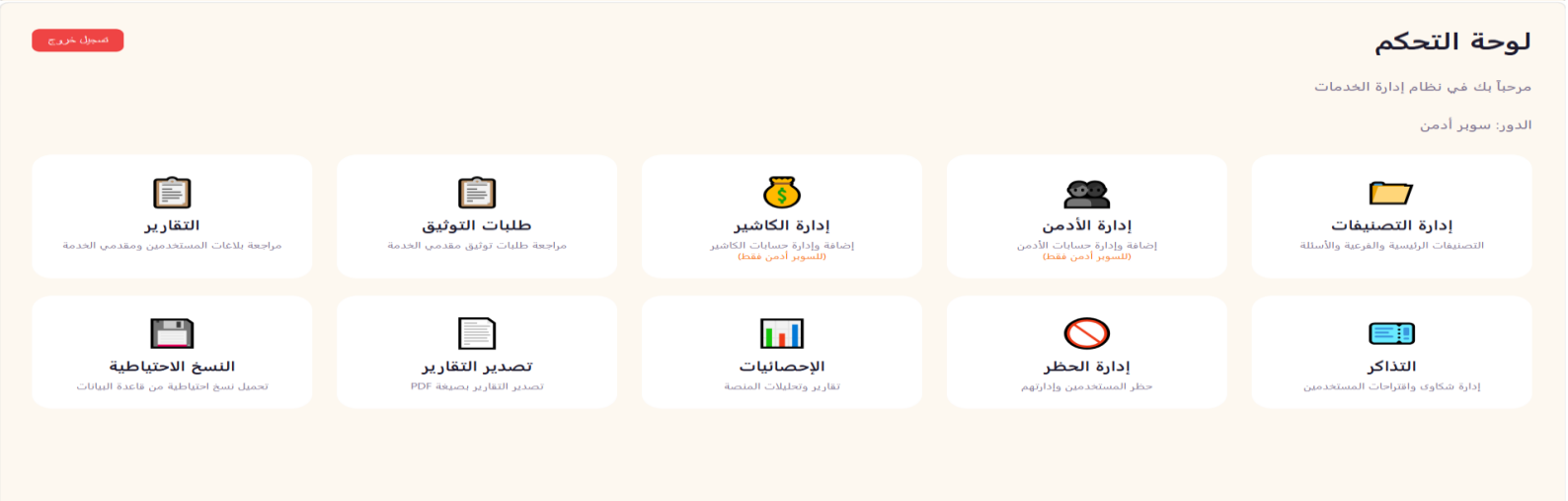
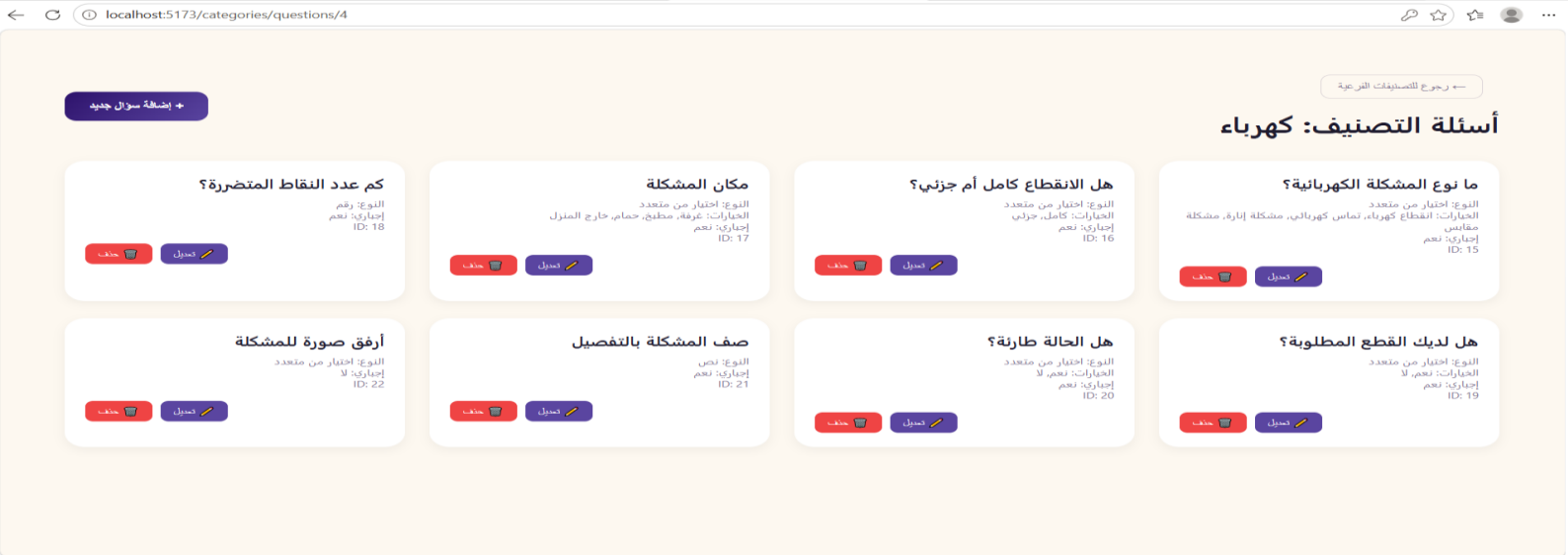
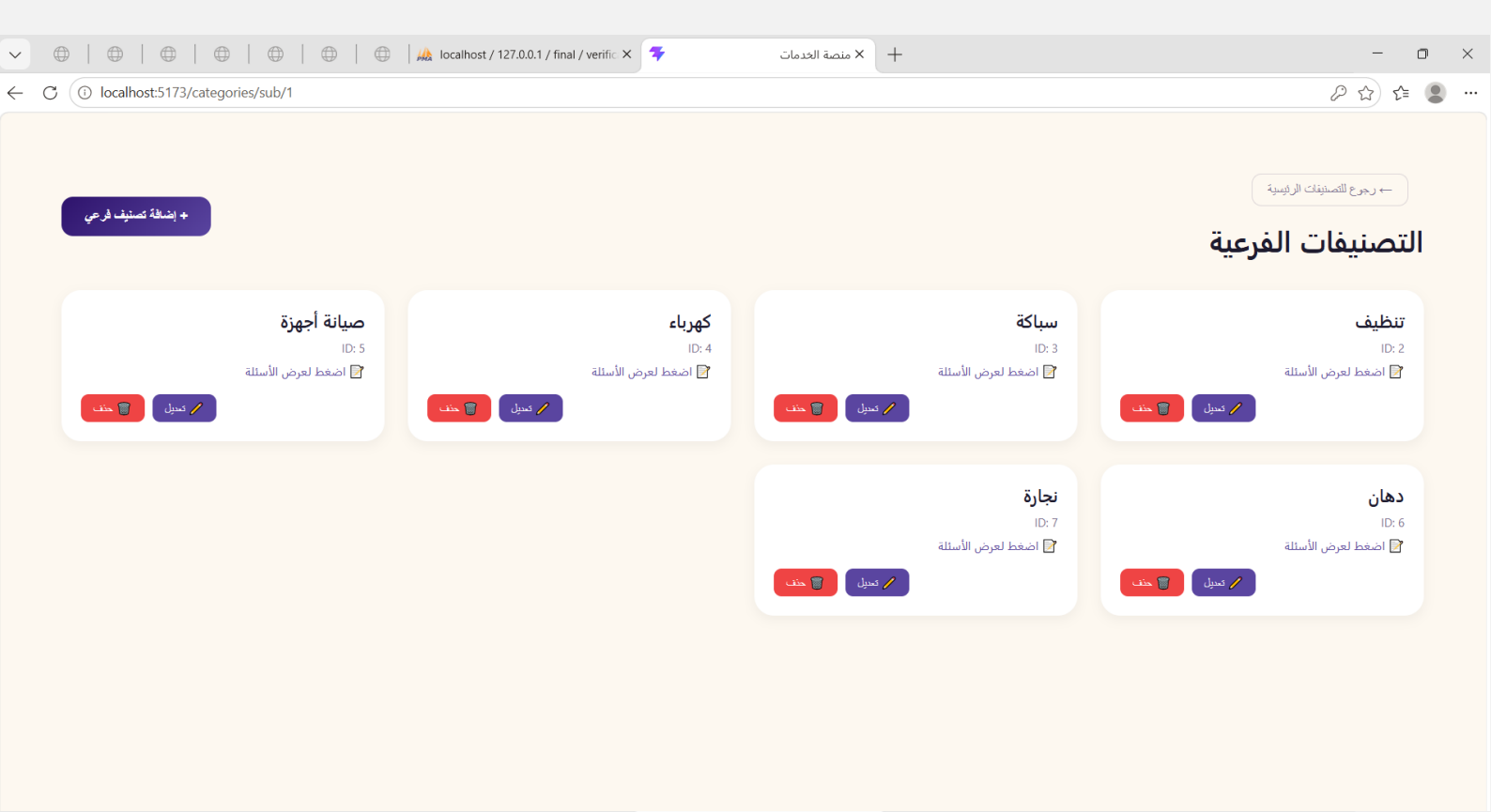
Additional Capabilities (5.17 - 5.31)



Class Diagram (UML)







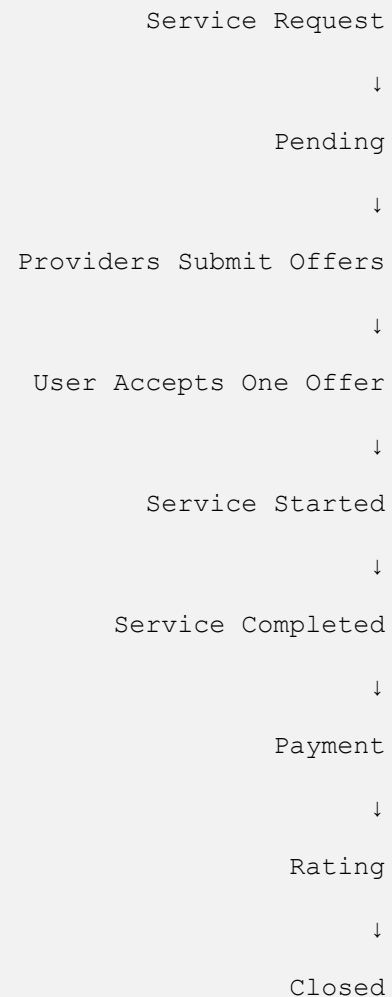
5.1 Introduction

تمثل وحدة إدارة الخدمات المحور الرئيسي للنظام، إذ توفر بيئة متكاملة تربط بين طالب الخدمة ومقدم الخدمة ضمن دورة عمل تبدأ بإنشاء الطلب وتنتهي بإتمام الخدمة وتقييمها. وقد صُممت هذه الوحدة اعتماداً على مفهوم دورة حياة الخدمة (Service Lifecycle)، بحيث تمر الخدمة بعدة حالات متتابعة تضمن تنفيذ العمليات بصورة منظمة وأمنة.

اعتمد تصميم هذه الوحدة على مبدأ فصل المسؤوليات (Separation of Concerns)، حيث تم توزيع المهام بين طبقات النظام المختلفة، مما ساهم في تحسين قابلية الصيانة، وتقليل الترابط بين المكونات، وإمكانية تطوير النظام مستقبلاً دون التأثير على الوحدات الأخرى.

5.2 Service Lifecycle

تمر الخدمة داخل النظام بعدة مراحل متتابعة، كما هو موضح في الشكل التالي:



تم تصميم هذه الدورة بحيث تمنع أي انتقال غير صحيح بين الحالات المختلفة، حيث لا يسمح النظام بالانتقال إلى مرحلة جديدة إلا بعد استيفاء شروط المرحلة السابقة.

5.3 Service Categories

يعتمد النظام على هيكلية تصنيف هرمية (Hierarchical Categories) ، حيث يتم تقسيم الخدمات إلى:

- التصنيفات الرئيسية (Main Categories)
- التصنيفات الفرعية (Sub Categories)

ولا يسمح للمستخدم بإنشاء طلب باستخدام تصنيف رئيسي، وإنما يجب اختيار تصنيف فرعي فقط، وذلك لضمان وصول الطلب إلى مقدمي الخدمات المختصين.

كما يدعم كل تصنيف مجموعة ديناميكية من الأسئلة (Dynamic Questions) ، مما يسمح بإضافة خدمات جديدة دون الحاجة إلى تعديل الكود البرمجي.

5.4 Dynamic Service Forms

بدلاً من إنشاء نموذج ثابت لكل خدمة، تم تصميم النظام بحيث يتم إنشاء نموذج الطلب بصورة ديناميكية اعتماداً على الأسئلة المرتبطة بكل تصنيف.

تدعم الأسئلة عدة أنواع من البيانات، منها:

- النصوص.
- الأرقام.
- القوائم المنسدلة.
- الاختيار المتعدد.
- الصور.

يقوم النظام بالتحقق من نوع كل إجابة قبل تخزينها، كما يتم التحقق من الأسئلة الإلزامية لضمان اكتمال بيانات الطلب.

ويُعد هذا التصميم أكثر مرونة مقارنة بالأنظمة التقليدية، حيث يمكن للمسؤول إضافة أو تعديل الأسئلة الخاصة بأي خدمة دون الحاجة إلى إعادة تطوير التطبيق.

5.5 Creating a Service Request

تمر عملية إنشاء طلب الخدمة بالمراحل التالية:

1. اختيار التصنيف الفرعي.
2. تحميل الأسئلة الخاصة بالخدمة.
3. إدخال الإجابات المطلوبة.
4. رفع الصور عند الحاجة.
5. التحقق من صحة جميع البيانات.
6. التحقق من عدم وجود طلب مفتوح لنفس الخدمة.
7. تخزين الطلب داخل قاعدة البيانات.

8. جعله متاحًا لمقدمي الخدمات المناسبين.

كما يمنع النظام إنشاء أكثر من طلب نشط لنفس المستخدم ضمن نفس التصنيف، الأمر الذي يقلل من تكرار الطلبات ويحافظ على جودة البيانات.

5.6 Image Management

يدعم النظام رفع الصور كجزء من إجابات الطلب.

وقد تم تنفيذ مجموعة من إجراءات التحقق تشمل:

- التأكد من أن الملف صورة.
- التحقق من نوع الامتداد.
- تحديد الحد الأقصى لحجم الملف.
- تخزين الصور داخل مساحة تخزين مخصصة.
- حفظ مسارات الصور داخل قاعدة البيانات.

يسمح النظام كذلك برفع أكثر من صورة للإجابة الواحدة عند الحاجة.

5.7 Provider Categories

قبل السماح لمقدم الخدمة باستقبال الطلبات، يجب عليه تحديد المجالات التي يعمل بها.

يقوم النظام بما يلي:

- التحقق من اختيار تصنيفات فرعية فقط.
- ربط مقدم الخدمة بتلك التصنيفات.
- إعادة طلب التحقق من مقدم الخدمة عند تعديل مجالات العمل.

وبذلك لا تظهر لمقدم الخدمة إلا الطلبات التي تتوافق مع تخصصه.

5.8 Service Offers

بعد ظهور الطلبات المناسبة، يستطيع مقدم الخدمة إرسال عرض يتضمن:

- الحد الأدنى للسعر.
- الحد الأعلى للسعر.
- رسالة توضيحية اختيارية.

كما يمنع النظام إرسال أكثر من عرض من نفس مقدم الخدمة على الطلب نفسه، وذلك لمنع التكرار والمحافظة على عدالة المنافسة بين مقدمي الخدمات.

5.9 Offer Acceptance

بعد استلام عدة عروض، يستطيع المستخدم اختيار العرض المناسب.

وعند قبول أحد العروض يقوم النظام تلقائيًا بما يلي:

- اعتماد العرض المختار.
- رفض جميع العروض الأخرى.
- تغيير حالة الطلب إلى Accepted.
- إنشاء محادثة مباشرة بين الطرفين.
- إرسال إشعار إلى مقدم الخدمة.

وبذلك يصبح لكل طلب مقدم خدمة واحد فقط.

5.10 Service Execution

بعد قبول العرض يستطيع مقدم الخدمة بدء تنفيذ الخدمة.

ولا يسمح النظام ببدء التنفيذ إلا إذا كانت حالة الطلب Accepted.

وعند بدء التنفيذ يتم تحديث حالة الطلب إلى In Progress ، مع إرسال إشعار إلى المستخدم.

5.11 Final Price Negotiation

يدعم النظام التفاوض على السعر النهائي بعد تنفيذ الخدمة.

فإذا كان السعر النهائي يقع ضمن المجال السعري الذي اقترحه مقدم الخدمة، ينتقل الطلب مباشرة إلى مرحلة الدفع.

أما إذا تجاوز السعر هذا المجال، فينتقل الطلب إلى مرحلة انتظار موافقة المستخدم.

يمكن للمستخدم:

- قبول السعر الجديد.
- رفض السعر.
- انتظار تعديل السعر.

وفي حالة الرفض، يستطيع مقدم الخدمة إرسال سعر أقل من السعر السابق فقط، مما يمنع زيادة السعر أثناء التفاوض.

5.12 Payment Management

بعد الاتفاق على السعر النهائي، ينتقل الطلب إلى مرحلة الدفع.

يقوم النظام بما يلي:

- إنشاء سجل دفع جديد.
- إنشاء رقم مرجعي فريد لكل عملية.
- ربط عملية الدفع بالطلب والعرض والمستخدم ومقدم الخدمة.
- وضع العملية في حالة Pending حتى يتم التحقق منها.

ويضمن ذلك إمكانية تتبع جميع عمليات الدفع بصورة دقيقة.

5.13 Payment Verification

تم دمج النظام مع خدمة الدفع الإلكتروني للتحقق من عمليات التحويل.

تعتمد عملية التحقق على:

- قيمة المبلغ.
- الرقم المرجعي.
- وقت تنفيذ العملية.

وعند نجاح عملية التحقق يقوم النظام تلقائيًا بـ:

- تحديث حالة الدفع.
- تحويل حالة العرض إلى Waiting for Rating.
- إضافة قيمة الخدمة إلى محفظة مقدم الخدمة.
- إغلاق المحادثة بين الطرفين.

وبذلك يتم ضمان اكتمال دورة الخدمة بصورة صحيحة.

5.14 Chat System

يوفر النظام نظام محادثة مباشر بين المستخدم ومقدم الخدمة.

ويتم إنشاء المحادثة تلقائيًا عند قبول العرض.

وتشمل خصائص النظام:

- إنشاء محادثة واحدة لكل طلب.
- إرسال الرسائل بصورة فورية.
- منع أي مستخدم غير مخول من الوصول للمحادثة.
- إغلاق المحادثة تلقائيًا بعد اكتمال عملية الدفع.

كما يعتمد النظام على الأحداث (Events) والبيث المباشر (Broadcasting) لتحديث الرسائل فور إرسالها.

5.15 Notifications

تم دمج نظام إشعارات متكامل لمتابعة جميع الأحداث المهمة داخل دورة حياة الخدمة.

يتم إرسال إشعارات عند:

- قبول العرض.
- بدء تنفيذ الخدمة.
- تغيير حالة الطلب.
- إرسال سعر جديد.
- تحديث حالة الدفع.

كما يدعم النظام:

- عرض جميع الإشعارات.
- عرض الإشعارات غير المقروءة.
- تعليم إشعار واحد كمقروء.
- تعليم جميع الإشعارات كمقروءة.

5.16 Rating System

بعد انتهاء عملية الدفع يستطيع المستخدم تقييم مقدم الخدمة.

يشمل نظام التقييم:

- تقييم رقمي من 1 إلى 5.
- تعليق اختياري.
- منع تكرار التقييم.
- منع التقييم قبل اكتمال الدفع.

كما يقوم النظام بعد كل تقييم بإعادة حساب:

- متوسط تقييم مقدم الخدمة.
- عدد التقييمات.

ويتم تحديث هذه البيانات بصورة تلقائية داخل ملف مقدم الخدمة.

5.17 Provider Recommendation

يتضمن النظام آلية ذكية لترشيح مقدمي الخدمات.

تعتمد عملية الترشيح على عدة عوامل، منها:

- موقع المستخدم.
- موقع مقدم الخدمة.
- التصنيف المطلوب.
- المسافة الجغرافية.
- تقييم مقدم الخدمة.

كما يدعم النظام البحث عن أقرب مقدمي الخدمات ضمن نطاق جغرافي يحدده المستخدم.

5.18 Input Validation

تم تنفيذ طبقة تحقق شاملة قبل تنفيذ جميع العمليات، وتشمل:

- التحقق من وجود التصنيف.
- التحقق من صحة الإجابات.
- التحقق من أنواع الملفات.
- التحقق من الأسعار.
- التحقق من المدى السعري.
- التحقق من صلاحيات المستخدم.
- التحقق من حالة الطلب.
- التحقق من حالة العرض.
- التحقق من وجود الموقع الجغرافي عند الحاجة.

5.19 State Management

اعتمد النظام على مفهوم إدارة حالات الطلب (State Management) ، حيث يمتلك كل طلب أو عرض حالة تشغيلية تحدد العمليات المسموح بتنفيذها.

وتشمل أهم الحالات:

- Pending
- Accepted
- Rejected
- In Progress
- Awaiting User Approval
- Awaiting Payment
- Waiting for Rating
- Price Rejected
- Closed

وقد تم منع جميع الانتقالات غير المسموح بها لضمان سلامة منطق الأعمال.

5.20 Caching Strategy

لتحسين الأداء وتقليل عدد الاستعلامات، تم استخدام آلية التخزين المؤقت (Caching) في عدة أجزاء من النظام، منها:

- التصنيفات.
- التصنيفات الفرعية.
- الطلبات القريبة.
- مقدمو الخدمات المقترحو.
- الطلبات المتاحة لمقدمي الخدمات.

كما يتم تنظيف البيانات المخزنة تلقائيًا عند حدوث أي تعديل يؤثر على النتائج.

5.21 Design Principles

اعتمد تطوير هذه الوحدة على مجموعة من مبادئ هندسة البرمجيات، من أهمها:

- Layered Architecture.
- Repository Pattern.
- Dependency Injection.
- Service Layer Pattern.
- Separation of Concerns.
- High Cohesion.
- Loose Coupling.
- State-Based Workflow.
- Event Driven Architecture.
- Reusability.
- Maintainability.
- Scalability.

وقد ساهمت هذه المبادئ في بناء وحدة خدمات مرنة، قابلة للتوسع، وتدعم إضافة خدمات جديدة دون الحاجة إلى إعادة تصميم النظام.

5.22 Repository Layer Design

اعتمد النظام على نمط **Repository Pattern** لفصل عمليات الوصول إلى قاعدة البيانات عن منطق الأعمال، حيث تم إنشاء مستودعات مستقلة لكل كيان رئيسي مثل طلبات الخدمات، والعروض، والمحادثات، والرسائل.

يوفر هذا التصميم مجموعة من المزايا، أهمها:

- تقليل الترابط بين طبقة الخدمات وقاعدة البيانات.
- سهولة استبدال أو تعديل مصدر البيانات دون التأثير على منطق الأعمال.
- إعادة استخدام الاستعلامات في أكثر من جزء داخل النظام.
- تسهيل اختبار النظام. (Unit Testing)

- تحسين قابلية الصيانة والتطوير.

كما احتوت المستودعات على الاستعلامات المعقدة الخاصة بالبحث، والتصفية، والحسابات الجغرافية، بينما بقيت طبقة الخدمات مسؤولة عن تطبيق قواعد العمل فقط.

5.23 Geographical Search

يدعم النظام البحث الجغرافي (Location-Based Search) بالاعتماد على الإحداثيات الجغرافية (Latitude and Longitude).

تم حساب المسافة بين المستخدم ومقدم الخدمة باستخدام معادلة Haversine ، والتي تعد من أشهر المعادلات المستخدمة لحساب المسافات بين نقطتين على سطح الأرض.

كما يقوم النظام بحساب الزمن التقريبي للوصول (Estimated Time of Arrival - ETA) اعتمادًا على المسافة وسرعة حركة تقديرية، الأمر الذي يساعد المستخدم في اختيار مقدم الخدمة الأنسب.

ويمكن استخدام هذه المعلومات في:

- ترتيب النتائج حسب المسافة.
- البحث ضمن نطاق جغرافي معين.
- عرض الزمن المتوقع للوصول.
- ترشيح مقدمي الخدمات الأقرب.

5.24 Smart Recommendation Algorithm

يتضمن النظام خوارزمية ذكية لترتيب مقدمي الخدمات اعتمادًا على أكثر من معيار بدلاً من الاعتماد على عامل واحد.

تعتمد الخوارزمية على دمج عدة مؤشرات، منها:

- متوسط تقييم مقدم الخدمة.
- عدد الخدمات المنجزة.
- المسافة بين المستخدم ومقدم الخدمة.
- التخصص المطلوب.

ثم يتم حساب درجة (Score) لكل مقدم خدمة، وترتيب النتائج تنازليًا بحيث يظهر أكثر مقدمي الخدمات ملائمة في بداية القائمة.

يساعد هذا الأسلوب في تحسين جودة الترشيحات ورفع احتمالية اختيار مقدم الخدمة الأنسب.

5.25 Advanced Filtering

يوفر النظام مجموعة متقدمة من خيارات البحث والتصفية عند استعراض العروض، وتشمل:

- الحد الأدنى للسعر.
- الحد الأعلى للسعر.
- تقييم مقدم الخدمة.
- حالة التحقق من مقدم الخدمة.
- المسافة الجغرافية.
- الزمن المتوقع للوصول.

كما يدعم النظام ترتيب النتائج وفق عدة معايير، مثل:

- الأقرب جغرافياً.
- الأقل سعراً.
- الأعلى سعراً.
- الأعلى تقييماً.

يسهم ذلك في منح المستخدم مرونة كبيرة عند اختيار العرض المناسب.

5.26 Unified API Response

لضمان توحيد شكل الاستجابات بين جميع واجهات البرمجة، تم تطوير طبقة مستقلة لإرجاع البيانات (API Response Wrapper).

تعتمد جميع واجهات البرمجة على بنية موحدة تتضمن:

- حالة العملية.
- الرسالة النصية.
- البيانات.
- معلومات الصفحات (Pagination Metadata) عند الحاجة.

وقد ساهم هذا التصميم في تسهيل تعامل تطبيق الهاتف المحمول مع جميع واجهات البرمجة دون الحاجة إلى معالجة كل استجابة بطريقة مختلفة.

5.27 Pagination Strategy

نظراً لاحتمالية زيادة أعداد الطلبات والعروض والرسائل والإشعارات، تم استخدام آلية Pagination بدلاً من تحميل جميع البيانات دفعة واحدة.

وتوفر هذه الآلية عدداً من المزايا، أهمها:

- تقليل حجم البيانات المنقولة عبر الشبكة.
- تحسين سرعة الاستجابة.
- تقليل استهلاك الذاكرة.
- تحسين تجربة المستخدم.

كما تتضمن الاستجابة معلومات إضافية مثل:

- الصفحة الحالية.
- العدد الكلي للصفحات.
- عدد العناصر في كل صفحة.
- العدد الإجمالي للسجلات.

5.28 Real-Time Communication

يدعم النظام الاتصال الفوري (Real-Time Communication) باستخدام نظام الأحداث (Events) وآلية البث (Broadcasting).

فعند إرسال رسالة داخل المحادثة يتم:

1. إنشاء الرسالة داخل قاعدة البيانات.
2. إطلاق حدث جديد.
3. بث الرسالة عبر قناة خاصة.
4. تحديث واجهة المستخدم مباشرة دون الحاجة إلى إعادة تحميل الصفحة.

وقد تم تخصيص قناة خاصة لكل محادثة لضمان وصول الرسائل إلى أطراف المحادثة فقط.

5.29 Real-Time Notification System

إلى جانب نظام الرسائل، تم تطوير نظام إشعارات فورية يعتمد على نفس آلية الأحداث والبث المباشر.

فعند حدوث أي تغيير مهم داخل النظام، مثل:

- قبول العرض.
- تغيير حالة الطلب.
- إرسال سعر جديد.
- إتمام عملية الدفع.

يقوم النظام تلقائيًا بإرسال إشعار فوري إلى المستخدم المعني، مع حفظ نسخة من الإشعار داخل قاعدة البيانات لضمان إمكانية الرجوع إليه لاحقًا.

5.30 Asynchronous Processing

لتقليل زمن الاستجابة وتحسين الأداء، تم تنفيذ الإشعارات باستخدام نظام الطوابير (Queue System) ، بحيث يتم إرسالها بصورة غير متزامنة (Asynchronous Processing) دون التأثير على سرعة تنفيذ الطلب الأساسي.

يسهم هذا الأسلوب في تحسين أداء النظام خصوصًا عند زيادة عدد المستخدمين أو عند إرسال عدد كبير من الإشعارات في وقت واحد.

5.31 Business Rules Enforcement

لا يقتصر النظام على التحقق من صحة البيانات المدخلة، وإنما يطبق مجموعة من قواعد العمل (Business Rules) لضمان سلامة دورة حياة الخدمة.

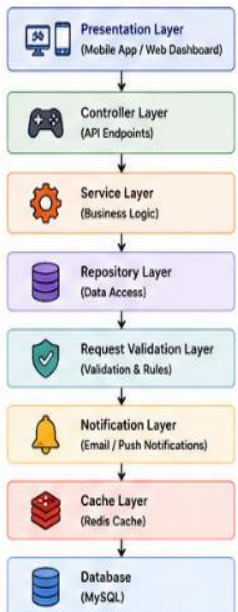
ومن أهم هذه القواعد:

- منع إنشاء أكثر من طلب نشط لنفس الخدمة.
- منع إرسال أكثر من عرض من مقدم الخدمة نفسه.
- منع قبول أكثر من عرض للطلب الواحد.
- منع الانتقال إلى حالة تشغيلية غير مسموحة.
- منع التقييم قبل إتمام عملية الدفع.
- منع تعديل السعر بطريقة تخالف سياسة التفاوض.
- منع إنشاء أكثر من محادثة لنفس الطلب.
- التحقق من ملكية المستخدم لجميع العمليات الحساسة.

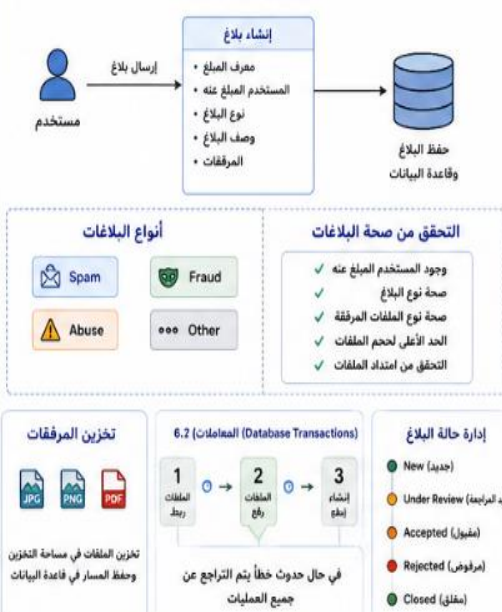
يساعد ذلك في المحافظة على اتساق البيانات ومنع حدوث حالات تشغيل غير صحيحة داخل النظام.

نظام الشكاوى والبلاغات وإدارة الحظر . 6 (Complaint, Reporting and Moderation Module)

البنية المعمارية للنظام 6



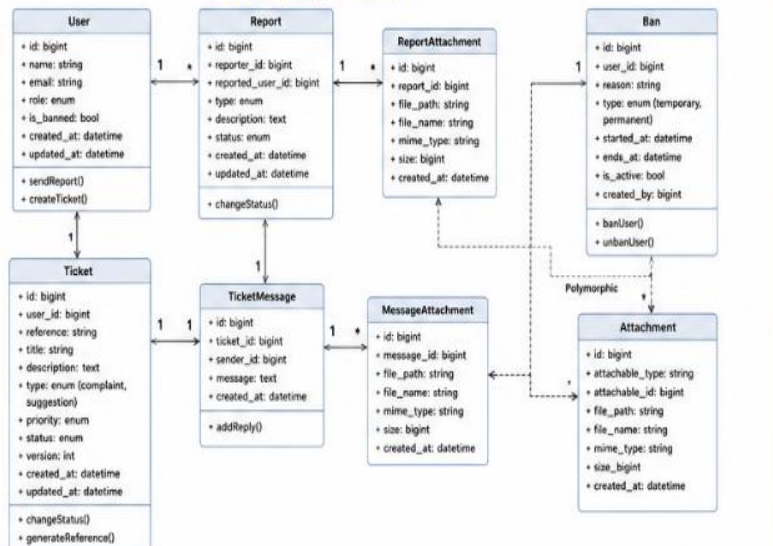
6.3 (Reporting System) البلاغات

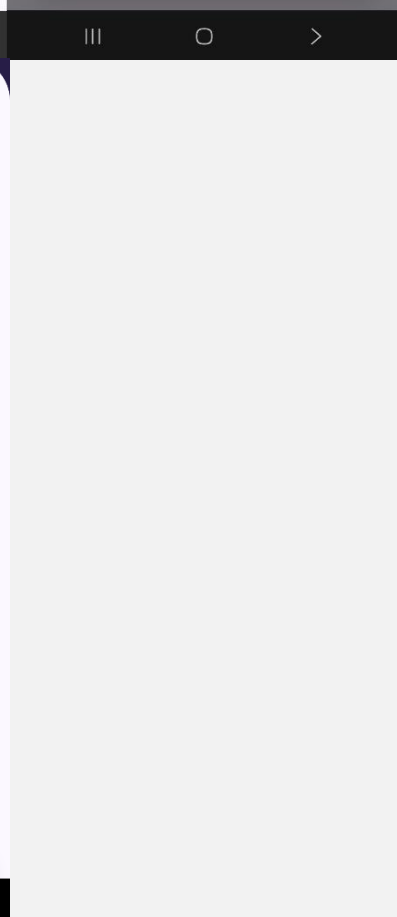
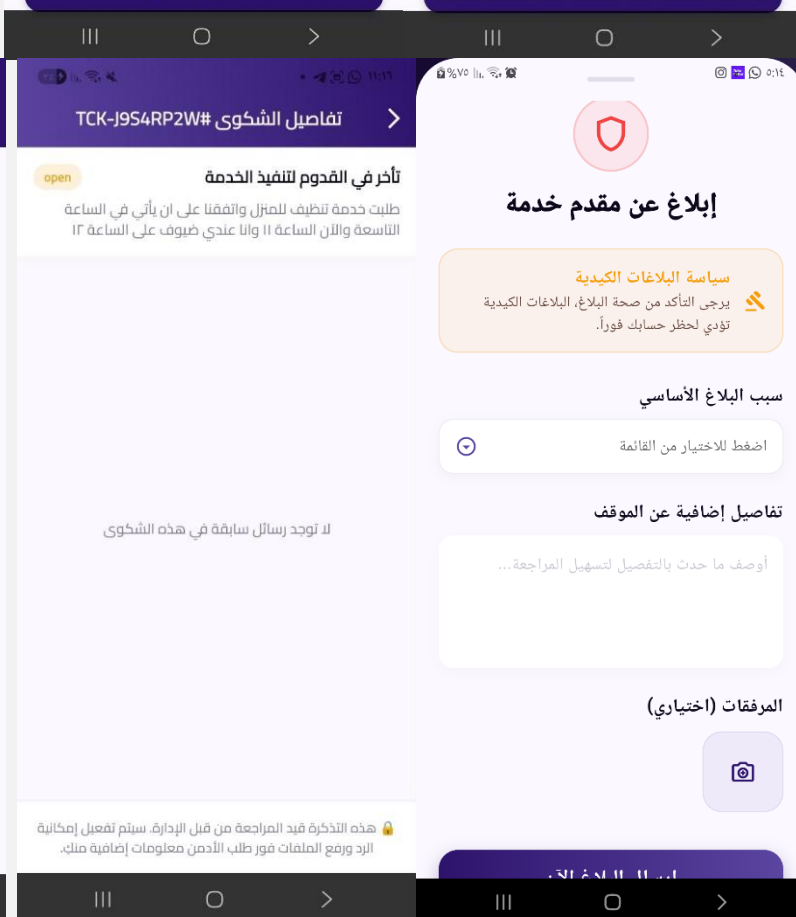
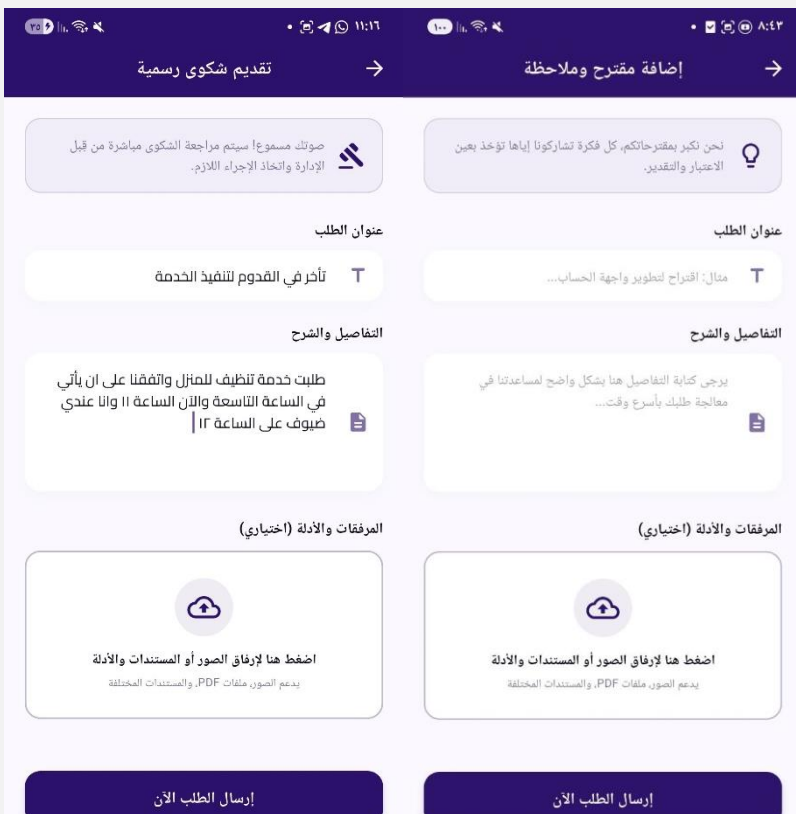


الحظر (Ban Management) 6.9 - 6.12



مخطط الفئات (Class Diagram)





localhost / 127.0.0.1 / final / users X

منصة الخدمات

localhost:5173/tickets

إدارة التذاكر

مغلقة (0)

محلولة (1)

تنتظر المستخدم (1)

قيد المراجعة (0)

معلقة (1)

مكتملة (3)

حسب الحالة

حسب الأولوية

حسب النوع

مغلقة

محلولة

تنتظر المستخدم

قيد المراجعة

معلقة

متوسطة

المرجع: TCK-JHNORCA

تحسين

ارجو التحسين...

معلقة

٢٠٢٧/٣

متوسطة

المرجع: TCK-MNJKLESM

تحسين

ارجو التحسين بأسرع وقت ممكن من فضلكم...

تنتظر المستخدم

٢٠٢٧/٣

متوسطة

المرجع: TCK-H79K7ZLU

تحسين

ارجو التحسين بأسرع وقت ممكن من فضلكم...

محلولة

٢٠٢٧/٣

localhost / 127.0.0.1 / final / users X

منصة الخدمات

localhost:5173/bans

حظر مستخدم جديد

إدارة المحظورين

nour

bayanwe34@gmail.com

معرفة المستخدم: 2

نوع الحظر: مؤقت

٢٠٢٧/٤: ينتهي فيه

معرفة الشخص الذي قام بالحظر: 1

٢٠٢٧/٣: تاريخ الحظر

هذا الحظر

localhost:5173/tickets

تحسين

المرجع: TCK-H79K7ZLU

1 رسالة

متوسطة

معلقة

محلولة

تنتظر المستخدم

قيد المراجعة

معلقة

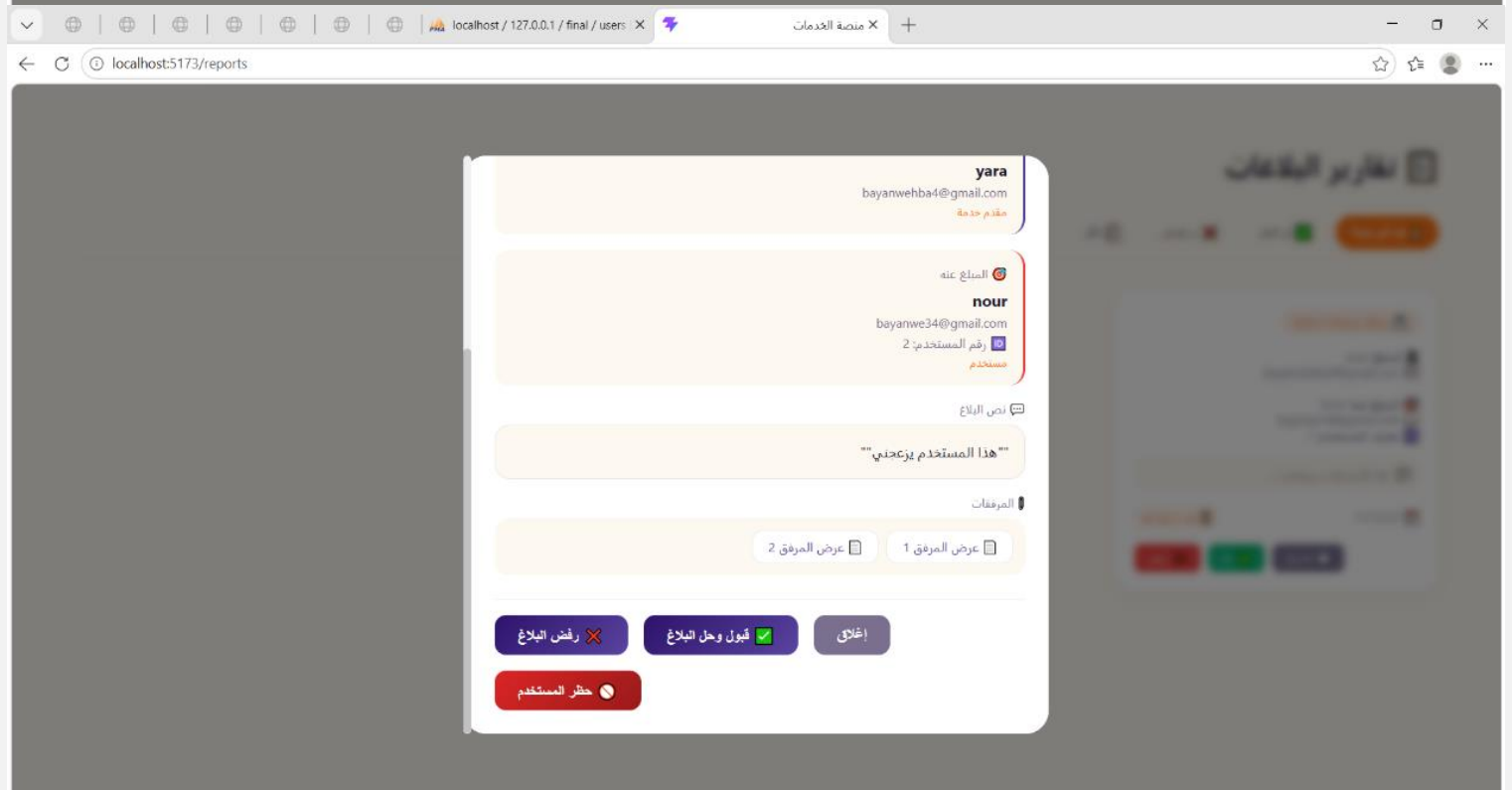
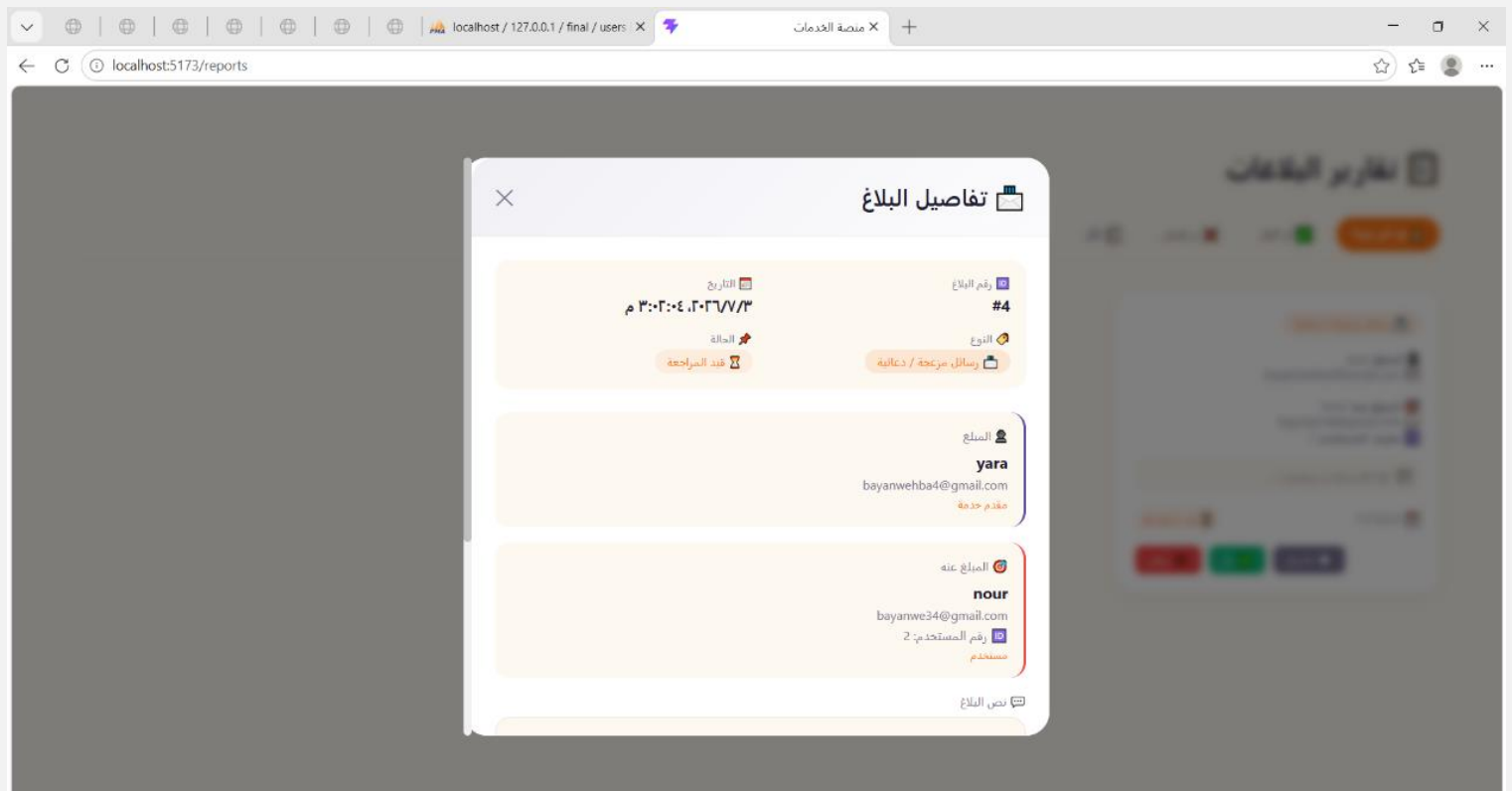
الإدارة

تم

اكتب ردك هنا...

إرسال الرد

إرفاق ملفات



6.1 مقدمة

نظرًا لأن التطبيق يربط بين مستخدمين ومقدمي خدمات بصورة مباشرة، فقد كان من الضروري توفير نظام متكامل لإدارة الشكاوى والبلاغات يضمن المحافظة على جودة المنصة ومعالجة السلوكيات غير المرغوبة.

تم تطوير هذه الوحدة بحيث تتيح للمستخدمين إرسال شكاوى أو اقتراحات إلى إدارة النظام، بالإضافة إلى إمكانية الإبلاغ عن المستخدمين المخالفين، مع توفير أدوات إدارية لمعالجة البلاغات وإدارة حالات الحظر.

6.2 البنية المعمارية للنظام

اعتمدت هذه الوحدة نفس المعمارية المستخدمة في بقية أجزاء النظام، والتي تتكون من الطبقات التالية:

- Controller Layer
- Service Layer
- Repository Layer
- Request Validation Layer
- Notification Layer
- Cache Layer

يؤدي هذا التقسيم إلى فصل مسؤوليات النظام وتحسين قابلية الصيانة وإعادة الاستخدام.

6.3 نظام البلاغات (Reporting System)

يسمح النظام لأي مستخدم بالإبلاغ عن مستخدم آخر في حال وجود سلوك مخالف.

تشمل البيانات المخزنة داخل البلاغ:

- معرف المُبلغ .
- المستخدم المُبلغ عنه .
- نوع البلاغ .
- وصف البلاغ .
- المرفقات .
- حالة البلاغ .

ويتم حفظ البلاغ داخل قاعدة البيانات مع الاحتفاظ بجميع الملفات المرفقة.

6.4 أنواع البلاغات

يدعم النظام عدة أنواع من البلاغات، وهي:

- Spam
- Fraud
- Abuse
- Other

يساعد هذا التصنيف الإدارة في فرز البلاغات وتحليلها بصورة أفضل.

6.5 التحقق من صحة البلاغات (Validation)

قبل قبول البلاغ يتم التحقق من:

- وجود المستخدم المُبلغ عنه .
- صحة نوع البلاغ .
- صحة نوع الملفات المرفقة .
- الحد الأعلى لحجم الملفات .
- التحقق من امتداد الملفات .

يساعد ذلك في منع إدخال بيانات غير صحيحة أو ملفات غير مسموحة.

6.6 تخزين المرفقات

يسمح النظام بإرفاق ملفات مع البلاغ.

وتشمل الملفات المدعومة:

- JPG
- PNG
- PDF

ويتم تخزين الملفات داخل مساحة التخزين الخاصة بالنظام، مع حفظ مسار الملف داخل قاعدة البيانات فقط.

6.7 المعاملات (Database Transactions)

تم استخدام Database Transactions أثناء إنشاء البلاغ لضمان تنفيذ جميع العمليات كوحدة واحدة.

تشمل العملية:

1. إنشاء البلاغ .
2. رفع الملفات .

3. ربط الملفات بالبلاغ .

وفي حال حدوث أي خطأ يتم التراجع عن جميع العمليات للحفاظ على سلامة البيانات.

6.8 إدارة حالة البلاغ

يمر البلاغ بعدة حالات أثناء دورة حياته، حيث يستطيع المشرف تغيير حالته بعد مراجعته. وتساعد هذه الحالات في متابعة البلاغات ومعرفة ما إذا كانت لا تزال قيد المعالجة أو تم حلها.

6.9 نظام الحظر (Ban Management)

تم تطوير نظام مستقل لإدارة المستخدمين المخالفين.

يوفر هذا النظام إمكانية:

- حظر المستخدم .
- فك الحظر .
- الحظر المؤقت .
- الحظر الدائم .

كما يتم تسجيل سبب الحظر وتاريخ بدايته ونهايته.

6.10 الحظر التلقائي

إضافة إلى الحظر اليدوي، يدعم النظام الحظر التلقائي.

فعند وصول عدد البلاغات المقبولة ضد مستخدم معين إلى حد معين، يقوم النظام تلقائيًا بإنشاء سجل حظر مؤقت دون تدخل الإدارة.

يساعد ذلك في حماية المنصة من الحسابات المسيئة بصورة استباقية.

6.11 Middleware للتحقق من الحظر

قبل تنفيذ أي طلب محمي يتم تمرير الطلب عبر Middleware مخصص.

يقوم هذا الـ Middleware بالتحقق مما إذا كان المستخدم محظورًا.

وفي حال كان الحظر ما يزال فعالاً يتم رفض الطلب مباشرة مع إعادة:

- سبب الحظر .
- تاريخ انتهاء الحظر .

وبذلك يتم منع المستخدم المحظور من استخدام النظام حتى انتهاء مدة العقوبة.

6.12 إشعارات الحظر

عند تنفيذ أي عملية حظر أو فك حظر يقوم النظام بإرسال إشعار مباشر إلى المستخدم.

وتشمل الإشعارات:

- Account Blocked
- Account Unblocked

ويساعد ذلك في إبقاء المستخدم على اطلاع بحالة حسابه.

6.13 نظام التذاكر (Support Ticket System)

تم تطوير نظام مستقل لإدارة الشكاوى والاقتراحات.

يمكن للمستخدم إنشاء تذكرة تتضمن:

- عنواناً .
- وصفاً .
- نوع التذكرة .
- الأولوية .
- المرفقات .

ويتم منح كل تذكرة رقماً مرجعياً (Reference Number) فريداً يمكن استخدامه لاحقاً للبحث عنها.

6.14 الرقم المرجعي للتذكرة

يتم توليد رقم مرجعي فريد لكل تذكرة بصورة تلقائية.

ويستخدم هذا الرقم في:

- متابعة حالة الشكاوى .
- البحث عنها .
- الرجوع إليها لاحقاً .

بدلاً من الاعتماد على معرف قاعدة البيانات.

6.15 أولويات التذاكر

يدعم النظام مستويات مختلفة من الأولوية، وهي:

- Low
- Medium
- High

ويساعد ذلك فريق الدعم على ترتيب الشكاوى حسب أهميتها.

6.16 أنواع التذاكر

يدعم النظام نوعين رئيسيين من التذاكر:

- Complaint
- Suggestion

مما يتيح الفصل بين المشاكل التقنية والاقتراحات التطويرية.

6.17 الرد على التذاكر

يستطيع كل من المستخدم والإدارة إرسال ردود داخل التذكرة.

ويحتوي الرد على:

- الرسالة النصية .
- الملفات المرفقة .
- المرسل .
- تاريخ الإرسال .

وبذلك تتحول التذكرة إلى سلسلة من المحادثات المنظمة.

6.18 المرفقات داخل الردود

يمكن إرفاق ملفات مع كل رسالة داخل التذكرة.

ويتم تخزين الملفات بصورة مستقلة وربطها برسالة محددة، مما يسمح بوجود عدة ملفات لكل رد.

6.19 إدارة حالة التذكرة

تمر التذكرة بعدة حالات أثناء دورة حياتها، مثل:

- Open
- In Progress
- Waiting User
- Resolved
- Closed

ويساعد ذلك على متابعة تقدم معالجة الشكوى.

6.20 التحكم بالتزامن (Optimistic Locking)

لتجنب تعارض التعديلات عند قيام أكثر من مسؤول بتعديل نفس التذكرة في الوقت نفسه، تم تطبيق تقنية Optimistic Locking.

يعتمد النظام على رقم إصدار (Version Number)، بحيث لا يتم قبول التعديل إلا إذا كانت نسخة البيانات لدى المستخدم مطابقة لأحدث نسخة موجودة في قاعدة البيانات.

وفي حال قام مستخدم آخر بتعديل التذكرة أولاً، يتم رفض العملية وإبلاغ المستخدم بوجود تعديل سابق.

ويعد هذا الأسلوب أكثر كفاءة من استخدام الأقفال التقليدية (Pessimistic Locking) في الأنظمة التي تكون فيها احتمالية التعارض منخفضة.

6.21 البحث والتصفية

يوفر النظام إمكانيات متقدمة للبحث داخل التذاكر باستخدام عدة معايير، منها:

- الحالة .
- النوع .
- الأولوية .
- الفترة الزمنية .

ويساعد ذلك مسؤولي النظام على الوصول السريع إلى الشكاوى المطلوبة.

6.22 التخزين المؤقت (Caching)

نظرًا لتكرار عمليات البحث والاستعلام، تم استخدام التخزين المؤقت (Cache) في عدة أجزاء من النظام، منها:

- البحث باستخدام الرقم المرجعي .
- رسائل التذاكر .
- نتائج التصفية .

كما يتم حذف البيانات المخزنة تلقائيًا عند حدوث أي تعديل لضمان إرجاع أحدث البيانات دائمًا.

6.23 إدارة المرفقات

اعتمد النظام على علاقة Polymorphic بين الملفات وكل من:

- التذاكر .
- رسائل التذاكر .

مما يسمح بإعادة استخدام نفس جدول المرفقات مع أكثر من كيان داخل النظام دون الحاجة إلى إنشاء جداول منفصلة لكل نوع.

6.24 قواعد العمل (Business Rules)

طبق النظام مجموعة من قواعد العمل لضمان صحة العمليات، من أهمها:

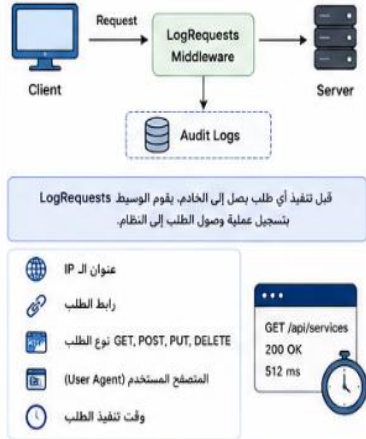
- منع إنشاء بلاغات ببيانات غير صحيحة .
- التحقق من وجود المستخدم المُبلغ عنه .
- التحقق من أنواع الملفات .
- منع الرد الفارغ داخل التذكرة .
- إنشاء رقم مرجعي فريد لكل تذكرة .
- منع الكتابة فوق تعديلات الآخرين باستخدام Optimistic Locking .
- منع المستخدم المحظور من استخدام النظام .
- تنفيذ الحظر التلقائي عند تجاوز عدد معين من البلاغات المقبولة .
- إرسال إشعارات تلقائية عند الحظر أو فك الحظر .

7. نظام المراقبة والتدقيق (Monitoring and Audit System)

تم تزويد النظام بوحدة متكاملة لمراقبة جميع العمليات المهمة التي تتم داخل المنصة، وذلك بهدف تتبع نشاط المستخدمين، ومراقبة العمليات الحساسة، وتسجيل الأخطاء والاستثناءات، مما يسهل عملية التدقيق الأمني وتحليل المشاكل ومتابعة أداء النظام.

يعتمد النظام على خدمة مركزية تدعى **SystemLogger** تقوم بتسجيل جميع الأحداث داخل جدول سجلات التدقيق (Audit Logs)، حيث يتم استدعاؤها من مختلف أجزاء النظام مثل الـ Middleware والـ Observers والخدمات المختلفة.

7.1 تسجيل طلبات النظام (Request Logging)



7.2 تسجيل الأخطاء والاستثناءات (Exception Logging)



7.3 مراقبة عمليات قواعد البيانات (Model Monitoring)

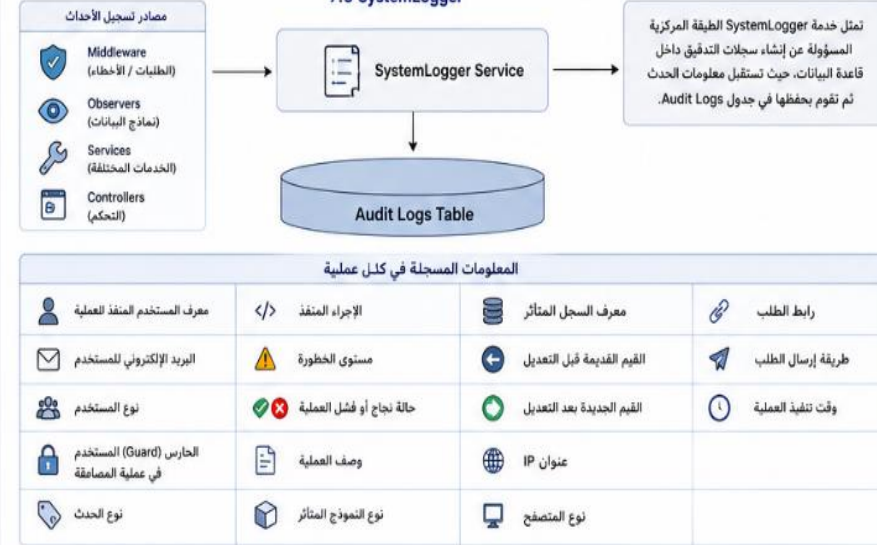


7.4 النماذج الخاضعة للمراقبة



وبذلك يتم تسجيل جميع العمليات المهمة التي تتم على هذه الكيانات بصورة تلقائية دون الحاجة إلى كتابة تعليمات تسجيل داخل كل نموذج.

7.5 SystemLogger



7.6 مستويات الخطورة (Severity Levels)



7.7 فوائد نظام المراقبة



تسجيل طلبات النظام 7.1 (Request Logging)

قبل تنفيذ أي طلب يصل إلى الخادم، يقوم الوسيط LogRequests بتسجيل عملية وصول الطلب إلى النظام.

يتم إنشاء سجل جديد يتضمن معلومات عامة عن الطلب مثل:

- عنوان الـ IP.
- رابط الطلب.
- نوع الطلب (GET، POST، PUT، DELETE).
- المتصفح المستخدم (User Agent).
- وقت تنفيذ الطلب.

يساعد ذلك في معرفة حجم استخدام النظام وتتبع جميع الطلبات الواردة.

تسجيل الأخطاء والاستثناءات 7.2 (Exception Logging)

يعتمد النظام على الوسيط LogExceptions لا يعترض جميع الاستثناءات غير المعالجة (Unhandled Exceptions).

عند حدوث أي خطأ أثناء تنفيذ أحد الطلبات يتم تسجيل المعلومات التالية:

- رسالة الخطأ .
- الملف الذي حدث فيه الخطأ .
- رقم السطر .
- مستوى الخطورة. (Critical).
- وصف الخطأ .
- حالة العملية (فشل) .

بعد تسجيل الخطأ يتم إعادة رمي الاستثناء ليتم التعامل معه بواسطة إطار العمل، وبذلك لا يتم فقدان أي معلومات تساعد في تشخيص المشكلة لاحقاً.

مراقبة عمليات قواعد البيانات 7.3 (Model Monitoring)

تم استخدام Global Observer لمراقبة جميع العمليات التي تتم على النماذج الرئيسية في النظام.

يقوم الـ Observer بتسجيل العمليات التالية:

- إنشاء سجل جديد. (Creating).
- تعديل سجل موجود. (Updating).
- حفظ القيم الجديدة بعد التعديل. (Updated).
- حذف السجل. (Deleted).

عند تنفيذ أي من هذه العمليات يتم حفظ البيانات القديمة والجديدة داخل سجل التدقيق، مما يسمح بمعرفة جميع التغييرات التي طرأت على البيانات مع مرور الزمن.

النماذج الخاضعة للمراقبة 7.4

تم ربط المراقب العام مع أهم نماذج النظام داخل AppServiceProvider، وتشمل:

- المستخدمون .
- المسؤولون .
- أمناء الصندوق .
- طلبات الخدمات .
- عروض الخدمات .
- المحادثات .
- عمليات الدفع .
- طلبات السحب .

- طلبات التوثيق .
- عمليات الحظر .

وبذلك يتم تسجيل جميع العمليات المهمة التي تتم على هذه الكيانات بصورة تلقائية دون الحاجة إلى كتابة تعليمات تسجيل داخل كل نموذج على حدة.

SystemLogger7.5

تمثل خدمة **SystemLogger** الطبقة المركزية المسؤولة عن إنشاء سجلات التدقيق داخل قاعدة البيانات، حيث تستقبل معلومات الحدث ثم تقوم بحفظها في جدول Audit Logs.

يقوم النظام بتسجيل مجموعة كبيرة من المعلومات لكل عملية، منها:

- معرف المستخدم المنفذ للعملية .
- البريد الإلكتروني للمستخدم .
- نوع المستخدم .
- الحارس (Guard) المستخدم في عملية المصادقة .
- نوع الحدث .
- الإجراء المنفذ .
- مستوى الخطورة .
- حالة نجاح أو فشل العملية .
- وصف العملية .
- نوع النموذج المتأثر .
- معرف السجل المتأثر .
- القيم القديمة قبل التعديل .
- القيم الجديدة بعد التعديل .
- عنوان IP .
- نوع المتصفح .
- رابط الطلب .
- طريقة إرسال الطلب .
- وقت تنفيذ العملية .

مستويات الخطورة (Severity Levels) 7.6

يعتمد نظام المراقبة على تصنيف الأحداث وفق مستويات مختلفة من الخطورة، بما يساعد في تسهيل تحليل السجلات، ومن أبرزها:

- **Info** للأحداث الاعتيادية .
- **Warning** للأحداث التي تستوجب الانتباه .
- **Critical** للأخطاء الحرجة والاستثناءات غير المعالجة .

يسمح هذا التصنيف لمسؤولي النظام بالتركيز على الأحداث الأكثر أهمية أثناء مراجعة سجلات التدقيق.

فوائد نظام المراقبة 7.7

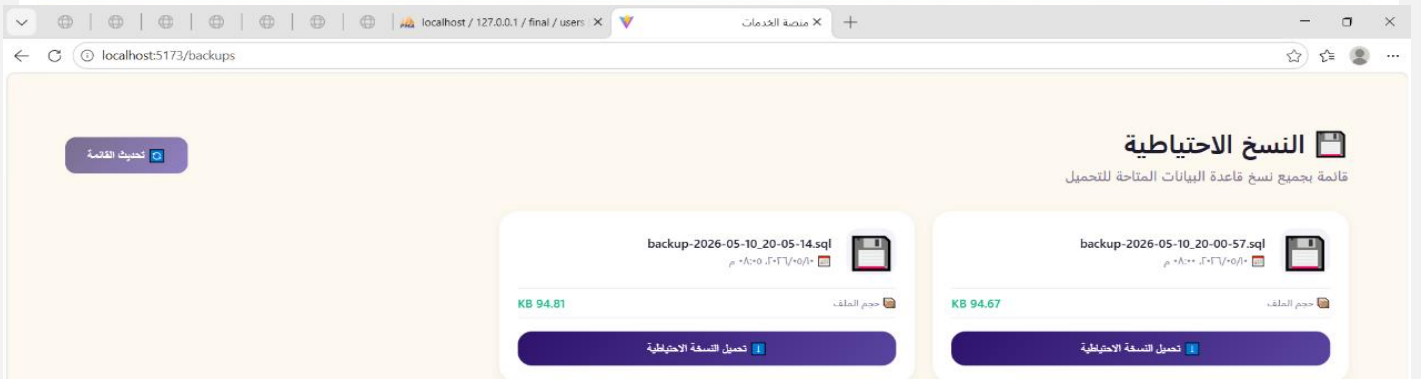
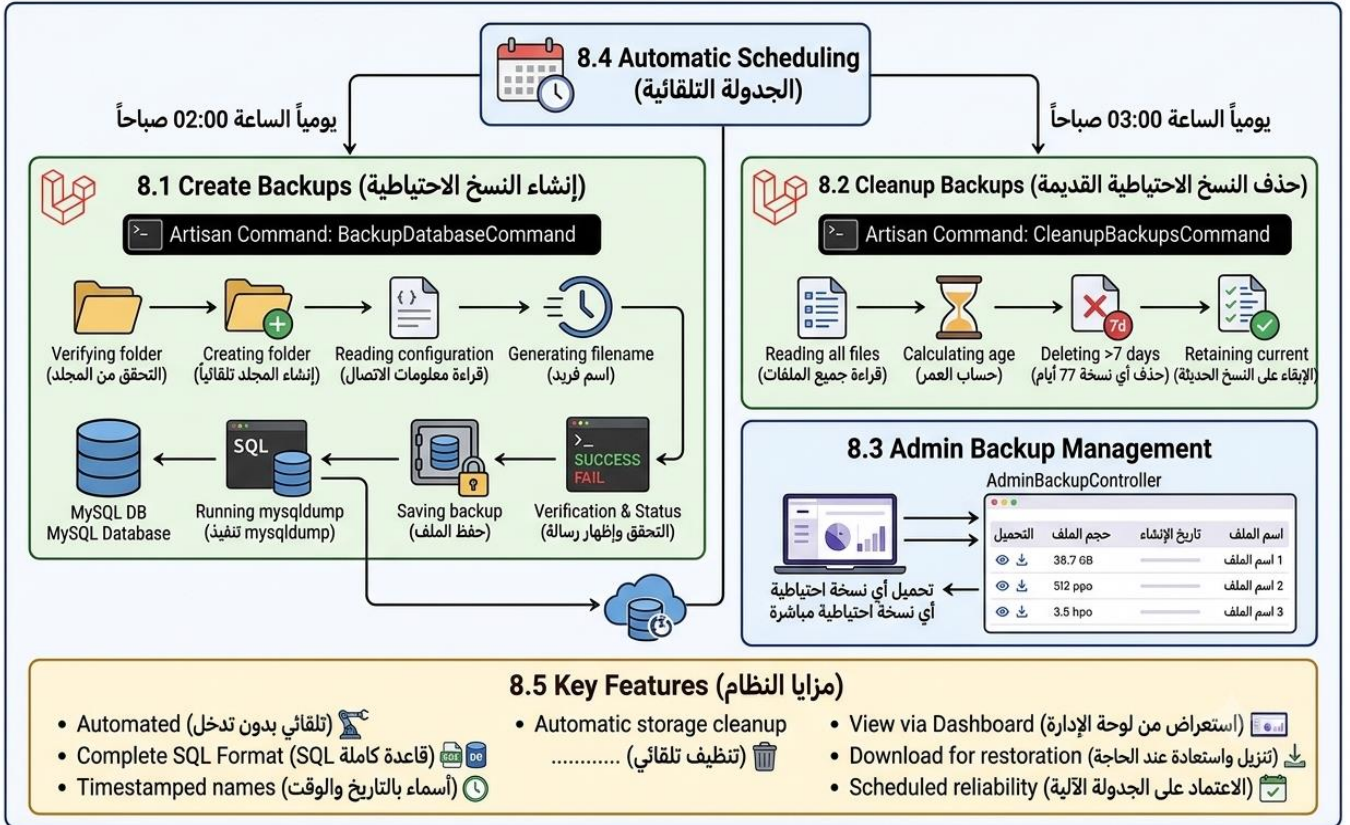
يوفر نظام المراقبة عدداً من المزايا المهمة، أبرزها:

- تتبع جميع العمليات التي تتم داخل النظام .
- تسجيل الأخطاء الحرجة تلقائياً .
- الاحتفاظ بتاريخ كامل للتعديلات على البيانات .
- تسهيل عمليات التدقيق الأمني .
- المساعدة في اكتشاف أسباب الأعطال والمشكلات .
- تحليل نشاط المستخدمين والإدارة .
- توفير سجل تاريخي يمكن الرجوع إليه عند الحاجة للتحقق من أي عملية تمت داخل النظام.

8. نظام النسخ الاحتياطي (Backup System)

تم تطوير نظام نسخ احتياطي تلقائي لقاعدة البيانات بهدف المحافظة على البيانات واستعادتها عند الحاجة في حال حدوث أعطال أو فقدان للبيانات. يعتمد النظام على أوامر (Artisan Commands) مجدولة تعمل بصورة دورية دون الحاجة إلى تدخل يدوي، كما يوفر واجهة إدارية لعرض النسخ الاحتياطية وتحميلها.

8. نظام النسخ الاحتياطي (Backup System)



8.1 إنشاء النسخ الاحتياطية

تم إنشاء الأمر `BackupDatabaseCommand` المسؤول عن إنشاء نسخة احتياطية كاملة من قاعدة البيانات. عند تنفيذ الأمر يقوم النظام بالخطوات التالية:

1. التحقق من وجود مجلد النسخ الاحتياطية داخل مجلد التخزين .
2. إنشاء المجلد تلقائياً في حال عدم وجوده .
3. قراءة معلومات الاتصال بقاعدة البيانات من ملف إعدادات `Laravel`.
4. إنشاء اسم فريد للنسخة الاحتياطية يعتمد على التاريخ والوقت الحالي .
5. تنفيذ أداة `mysqldump` لإنشاء ملف `SQL` يحتوي على جميع بيانات قاعدة البيانات .
6. حفظ ملف النسخة الاحتياطية داخل مجلد النسخ الاحتياطية .
7. التحقق من نجاح العملية وإظهار رسالة توضح نجاح أو فشل إنشاء النسخة .

يسمح هذا الأسلوب بالحصول على نسخة كاملة من قاعدة البيانات يمكن استخدامها لاحقاً لاستعادة النظام.

8.2 حذف النسخ الاحتياطية القديمة

لمنع تراكم الملفات واستهلاك مساحة التخزين، تم إنشاء الأمر `CleanupBackupsCommand` المسؤول عن حذف النسخ القديمة بصورة تلقائية.

يقوم الأمر بالخطوات التالية:

- قراءة جميع ملفات النسخ الاحتياطية .
- حساب عمر كل ملف اعتماداً على تاريخ آخر تعديل .
- حذف أي نسخة احتياطية يزيد عمرها عن سبعة أيام .
- الإبقاء على النسخ الحديثة فقط .

وبذلك يحافظ النظام على مساحة التخزين مع الاحتفاظ دائماً بأخر النسخ المطلوبة.

8.3 إدارة النسخ الاحتياطية

تم توفير متحكم إداري `AdminBackupController` لإدارة النسخ الاحتياطية من خلال واجهة برمجية.

يوفر المتحكم الوظائف التالية:

- عرض جميع ملفات النسخ الاحتياطية .
- عرض اسم كل ملف .
- عرض حجم الملف .
- عرض تاريخ إنشاء النسخة .
- تحميل أي نسخة احتياطية مباشرة عند الحاجة .

يساعد ذلك مسؤولي النظام على الوصول السريع إلى النسخ الاحتياطية دون الحاجة إلى الوصول المباشر إلى ملفات الخادم.

8.4 الجدولة التلقائية

اعتمد النظام على **Laravel Scheduler** لتنفيذ عمليات النسخ الاحتياطي والصيانة بصورة آلية.

تمت جدولة المهام كما يلي:

- إنشاء نسخة احتياطية يومياً عند الساعة **02:00 صباحاً**.
- حذف النسخ الاحتياطية القديمة يومياً عند الساعة **03:00 صباحاً**.

كما تم استخدام الخيار `withoutOverlapping` لمنع تشغيل أكثر من نسخة من المهمة نفسها في الوقت ذاته، مما يضمن عدم حدوث تعارض بين عمليات النسخ أو الحذف.

8.5 مزايا نظام النسخ الاحتياطي

يوفر النظام مجموعة من المزايا التي تزيد من موثوقية المنصة واستمرارية عملها، ومن أهمها:

- إنشاء نسخ احتياطية تلقائية دون تدخل المستخدم.
- حفظ قاعدة البيانات كاملة بصيغة `SQL` القياسية.
- تنظيم النسخ باستخدام أسماء تعتمد على التاريخ والوقت.
- حذف النسخ القديمة تلقائياً للحفاظ على مساحة التخزين.
- إمكانية استعراض جميع النسخ الاحتياطية من لوحة الإدارة.
- إمكانية تنزيل أي نسخة احتياطية واستعادتها عند الحاجة.
- الاعتماد على الجدولة الآلية لضمان تنفيذ عمليات النسخ بصورة منتظمة.

9 User Profile Management

تم تطوير نظام متكامل لإدارة الملف الشخصي (Profile Management) يتيح للمستخدمين تحديث بياناتهم الشخصية بصورة مرنة مع الحفاظ على سلامة البيانات وصحتها.

يدعم النظام إدارة مجموعة من المعلومات الأساسية، وتشمل:

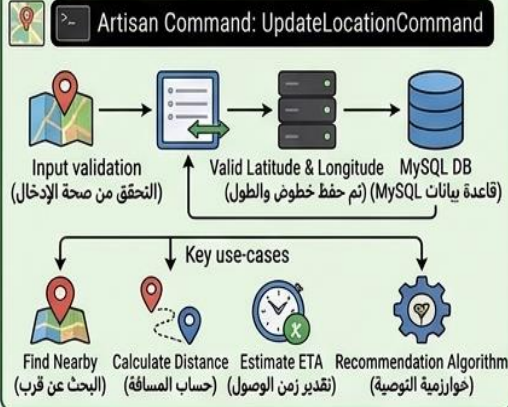
- الصورة الشخصية.
- المدينة.
- الموقع.
- نبذة تعريفية (Bio).
- رقم الهاتف.
- الموقع الجغرافي (Latitude & Longitude).

وقد تم تطبيق التحقق من صحة البيانات (Validation) قبل تنفيذ أي عملية تحديث، حيث يتم التحقق من نوع الملفات المرفوعة، وأحجام الصور، وصحة القيم الرقمية والنصية بما يضمن إدخال بيانات صحيحة ومتوافقة مع متطلبات النظام.

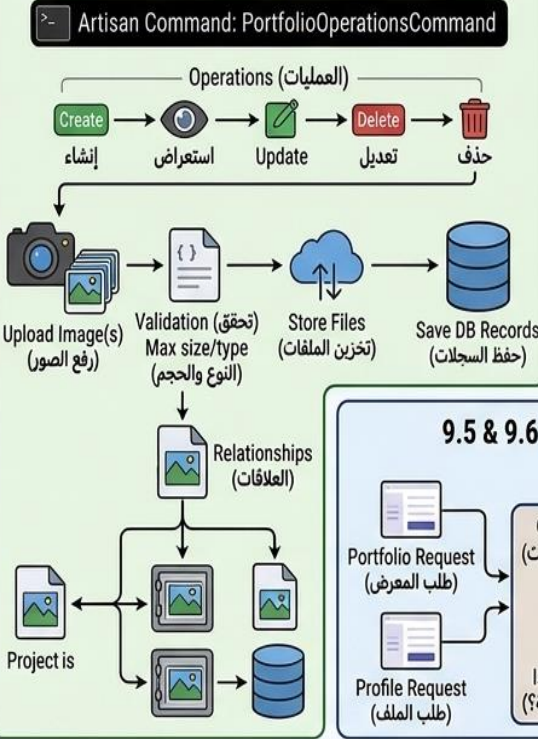
كما يعتمد النظام على مبدأ **Update or Create**، والذي يسمح بإنشاء الملف الشخصي لأول مرة أو تحديثه في حال وجوده دون الحاجة إلى تنفيذ عمليات تحقق إضافية.

9. نظام إدارة الملف الشخصي (User Profile Management)

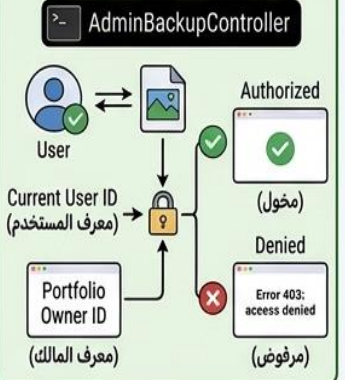
9.1 Geographical Location Management (الجغرافي)



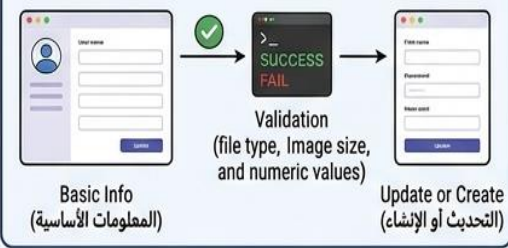
9.2 & 9.3 Portfolio & Image Management (دائرة الصور)



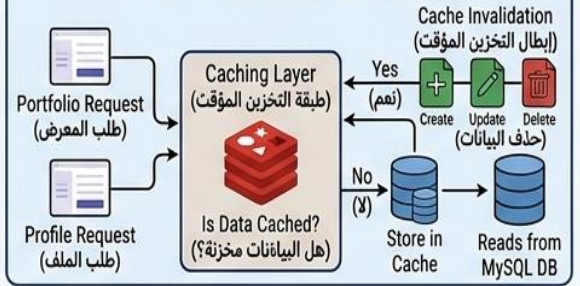
9.4 Portfolio Authorization



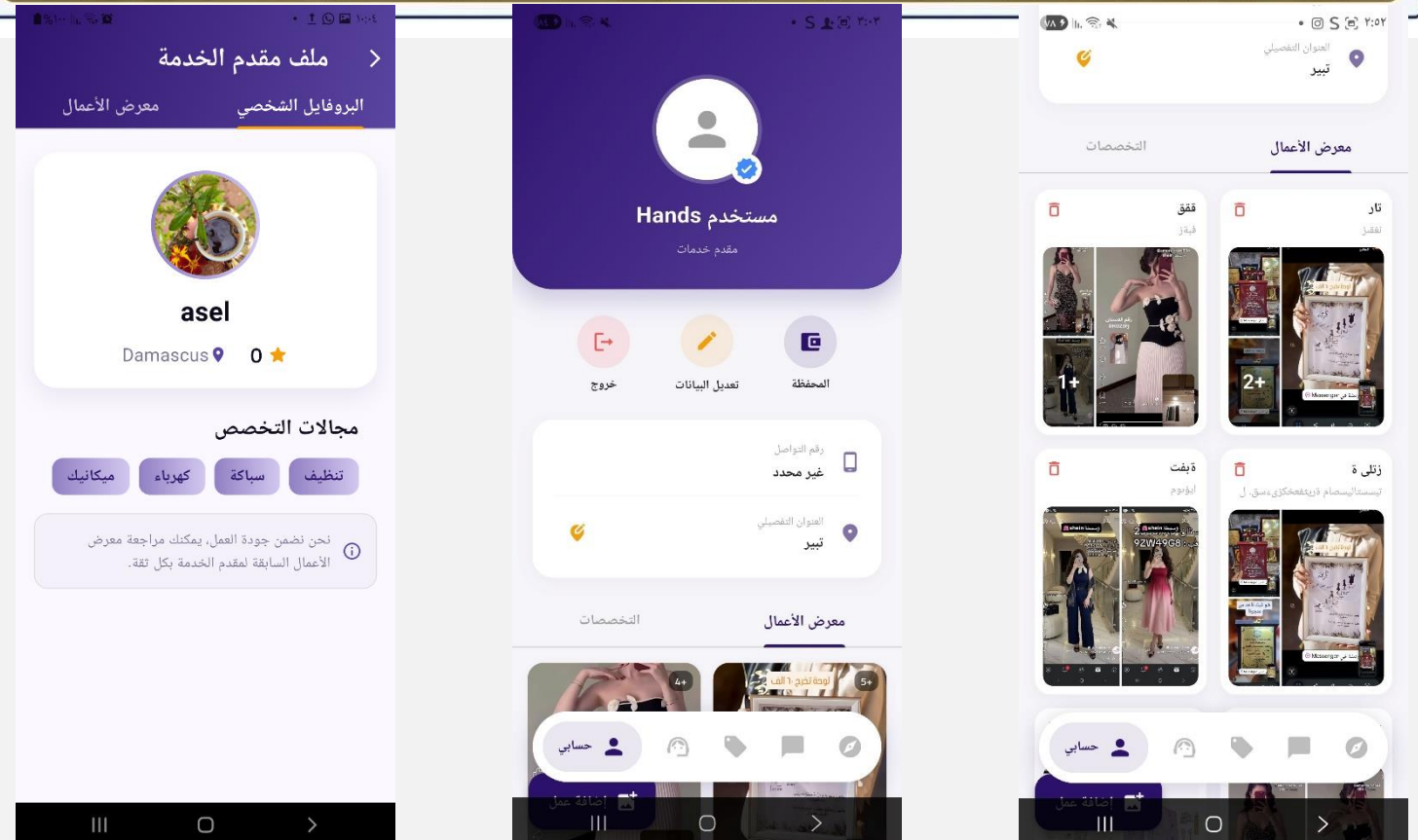
9.0 General Profile Management (العام)



9.5 & 9.6 Caching (التخزين المؤقت)



9.7 Key Profile Features (مزايا النظام)



9.1 Geographical Location Management

نظرًا لاعتماد التطبيق على الخدمات المحلية، تم تطوير نظام لإدارة الموقع الجغرافي لكل مستخدم.

يسمح النظام للمستخدم بتحديث إحداثياته الجغرافية المتمثلة بخطي العرض والطول (Latitude & Longitude)، ويتم استخدام هذه البيانات لاحقًا في العديد من وظائف النظام، مثل:

- البحث عن مقدمي الخدمات القريبين.
- حساب المسافات.
- تقدير زمن الوصول.
- تنفيذ خوارزمية التوصية الذكية.

ويضمن النظام التحقق من صحة الإحداثيات قبل تخزينها في قاعدة البيانات.

9.2 Portfolio Management

تم تطوير نظام متكامل لإدارة معرض الأعمال (Portfolio)، يتيح لمقدم الخدمة عرض أعماله السابقة بهدف زيادة موثوقيته ومساعدة المستخدمين على تقييم جودة الخدمات قبل اختيار مقدم الخدمة.

يوفر النظام العمليات الأساسية لإدارة معرض الأعمال، وتشمل:

- إنشاء معرض أعمال جديد.
- استعراض جميع الأعمال.
- تعديل بيانات العمل.
- حذف العمل.

ويحتوي كل عنصر داخل معرض الأعمال على:

- عنوان العمل.
- وصف اختياري.
- مجموعة من الصور.

كما يدعم النظام رفع أكثر من صورة لكل عمل، حيث يتم تخزين الصور داخل الخادم وربطها بسجل معرض الأعمال داخل قاعدة البيانات.

9.3 Portfolio Image Management

يدعم النظام رفع صور متعددة لكل عنصر داخل معرض الأعمال بدلاً من الاقتصار على صورة واحدة.

عند إنشاء أو تحديث معرض الأعمال، يتم تنفيذ الخطوات التالية:

1. التحقق من صحة جميع الصور المرفوعة.
2. التحقق من الحد الأقصى للحجم.
3. تخزين الصور داخل مجلد مخصص.

4. إنشاء سجل مستقل لكل صورة داخل قاعدة البيانات.
5. ربط الصور بمعرض الأعمال من خلال العلاقات (Relationships).

يسمح هذا التصميم بعرض عدد غير محدود من الصور لكل مشروع مع المحافظة على مرونة قاعدة البيانات.

9.4 Authorization for Portfolio Operations

تم تطبيق نظام صلاحيات (Authorization) يمنع أي مستخدم من تعديل أو حذف معرض أعمال لا يملكه. قبل تنفيذ عمليات التعديل أو الحذف، يقوم النظام بالتحقق من أن معرف المستخدم الحالي يطابق مالك معرض الأعمال، وفي حال عدم تحقق هذا الشرط يتم رفض الطلب وإرجاع رسالة خطأ مناسبة. يساعد ذلك في حماية بيانات المستخدمين ومنع الوصول غير المصرح به.

9.5 Portfolio Caching

نظرًا لأن معرض الأعمال يعد من أكثر البيانات التي يتم استعراضها داخل التطبيق، فقد تم استخدام آلية التخزين المؤقت (Caching) لتقليل عدد الاستعلامات المرسلة إلى قاعدة البيانات.

يقوم النظام بتخزين نتائج الاستعلام الخاصة بمعرض أعمال كل مستخدم لمدة زمنية محددة، ويتم حذف البيانات المخزنة تلقائيًا عند تنفيذ أي عملية من العمليات التالية:

- إنشاء معرض أعمال جديد.
- تعديل معرض أعمال.
- حذف معرض أعمال.

وبذلك يضمن النظام عرض أحدث البيانات مع المحافظة على الأداء العالي.

9.6 Profile Caching

تم تطبيق نفس آلية التخزين المؤقت على بيانات الملف الشخصي، حيث يتم حفظ بيانات المستخدم داخل الذاكرة المؤقتة لتقليل عمليات القراءة المتكررة من قاعدة البيانات.

وعند تحديث أي معلومة داخل الملف الشخصي، يتم حذف البيانات المخزنة مباشرة وإعادة بنائها عند أول طلب لاحق، مما يحقق التوازن بين الأداء ودقة البيانات.

10 Provider Verification System

نظرًا لأن التطبيق يعتمد على التعامل مع مقدمي خدمات حقيقيين، فقد تم تطوير نظام تحقق (Verification System) لزيادة مستوى الثقة بين المستخدمين ومقدمي الخدمات.

يعتمد النظام على قيام مقدم الخدمة بإرسال طلب تحقق يتضمن مستندين أساسيين:

- وثيقة إثبات الهوية.
- وثيقة تثبت مزاوله العمل أو الخبرة المهنية.

بعد رفع المستندات، يتم حفظها داخل الخادم وإنشاء طلب تحقق جديد بانتظار مراجعة الإدارة.

10. نظام التحقق من مقدمي الخدمات (Provider Verification System)

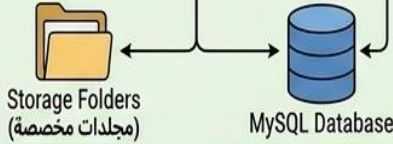
10.1 Verification Request Validation (التحقق من صحة الطلب)

Provider Profile

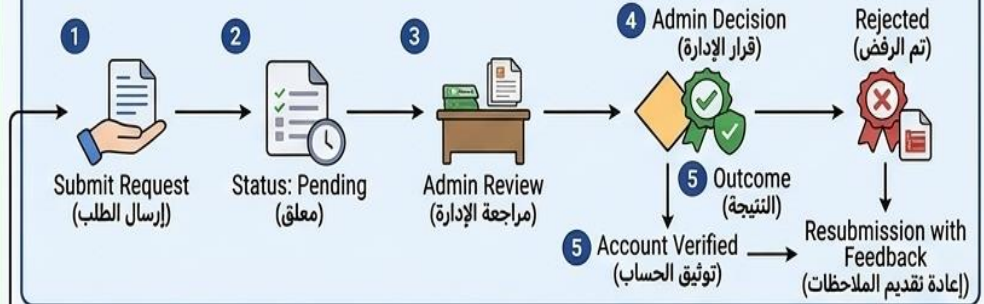
Verification Button

> _ VALIDATION RULES (قواعد العمل)

- ✓ User has Provider Role (المستخدم مقدم خدمة)
- ✓ Specializations Selected (تحديد مجالات)
- ✓ No Pending Request (لا يوجد طلب قيد المراجعة)
- ✓ File Types Valid (صحة أنواع الملفات)



10.2 Verification Workflow (دورة عمل التحقق)



10.3 Administrative Verification Management (إدارة التحقق الإدارية)



10.4 & 10.5 Impact & Notifications (الأثر)



10.6 Key Features (مزايا النظام)

- Boosted User Trust (زيادة ثقة المستخدمين)
- Complete Process Documentation (توثيق كامل للعملية)
- Automated Status Updates (تحديث تلقائي للحالة)
- Immediate User Notifications (إشعارات فورية للمستخدم)
- Structured Resubmission Process (عملية إعادة تقديم منظمة)

10.1 Verification Request Validation

قبل السماح بإرسال طلب التحقق، يقوم النظام بتنفيذ مجموعة من قواعد العمل لضمان صحة العملية، وتشمل:

- التأكد من أن المستخدم يمتلك دور مقدم خدمة (Provider).
- التأكد من اختيار مجال أو أكثر للتخصص.
- التأكد من عدم وجود طلب تحقق آخر قيد المراجعة.
- التحقق من نوع الملفات المرفوعة.
- تخزين المستندات داخل مجلدات مخصصة.

يساعد ذلك في منع الطلبات المكررة وضمان اكتمال البيانات المطلوبة قبل إرسالها للإدارة.

10.2 Verification Workflow

يتبع نظام التحقق دورة عمل واضحة تبدأ بطلب مقدم الخدمة وتنتهي بقرار الإدارة.

تمر العملية بالمراحل التالية:

1. يقوم مقدم الخدمة بإرسال طلب التحقق.
2. يتم حفظ الطلب بحالة (Pending).
3. يستطيع المسؤول استعراض جميع الطلبات المعلقة.
4. يقوم المسؤول بمراجعة الوثائق.
5. يتم اتخاذ أحد القرارين:
 - الموافقة (Approved).
 - الرفض (Rejected).
6. في حالة الموافقة يتم توثيق حساب مقدم الخدمة.
7. في حالة الرفض يتم إرسال سبب الرفض ليتمكن مقدم الخدمة من تصحيح البيانات وإعادة التقديم.

يعتمد هذا التسلسل على انتقالات واضحة بين حالات الطلب بما يضمن عدم حدوث تعارض في دورة حياة عملية التحقق.

10.3 Administrative Verification Management

تم تطوير واجهات خاصة بالإدارة لإدارة جميع طلبات التحقق الواردة من مقدمي الخدمات.

تتيح هذه الواجهات للمسؤول تنفيذ العمليات التالية:

- استعراض جميع الطلبات المعلقة.
- مراجعة الوثائق المرفوعة.
- قبول طلب التحقق.
- رفض الطلب مع تسجيل ملاحظات توضح سبب الرفض.

يساعد ذلك الإدارة على تنظيم عمليات المراجعة وضمان توثيق مقدمي الخدمات المؤهلين فقط.

10.4 Account Verification Status

عند الموافقة على طلب التحقق، يقوم النظام تلقائيًا بتحديث حالة الحساب من خلال تسجيل تاريخ التحقق داخل بيانات المستخدم.

ويتم الاعتماد على هذه الحالة لاحقًا في عدة أجزاء من النظام، مثل:

- تمييز مقدمي الخدمات الموثقين.
- دعم عمليات البحث والتصفية.
- زيادة موثوقية مقدم الخدمة أمام المستخدمين.
- استخدام حالة التحقق ضمن خوارزمية ترشيح مقدمي الخدمات.

10.5 Verification Notifications

بعد انتهاء مراجعة طلب التحقق، يقوم النظام بإرسال إشعار فوري إلى مقدم الخدمة لإعلامه بنتيجة الطلب.

في حالة الموافقة، يتم إشعاره بنجاح عملية التحقق وتفعيل حالة الحساب الموثق.

أما في حالة الرفض، فيتم حفظ سبب الرفض داخل الطلب، مما يسمح لمقدم الخدمة بمراجعة الأسباب وإعادة تقديم الطلب بعد معالجة الملاحظات.

يساهم هذا النظام في إبقاء المستخدم على اطلاع دائم بحالة طلبه دون الحاجة إلى مراجعة التطبيق بصورة مستمرة.

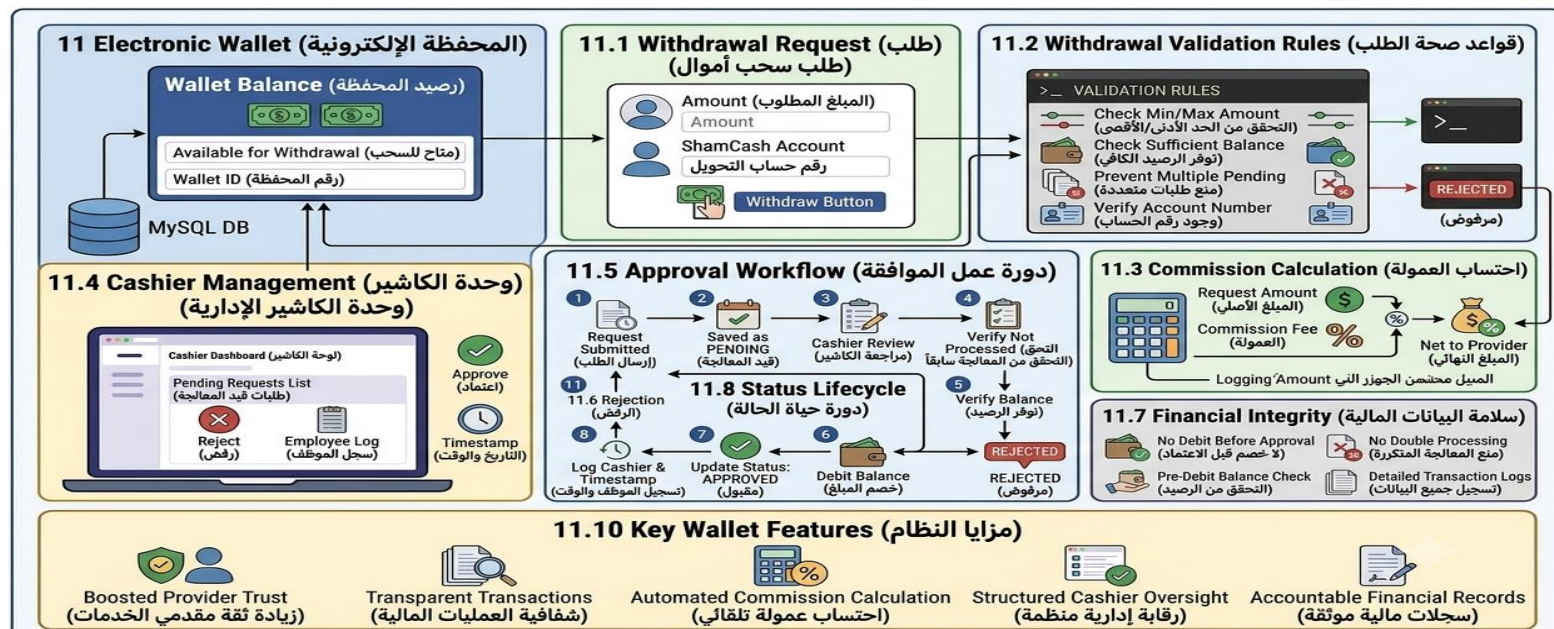
Electronic Wallet Management 11

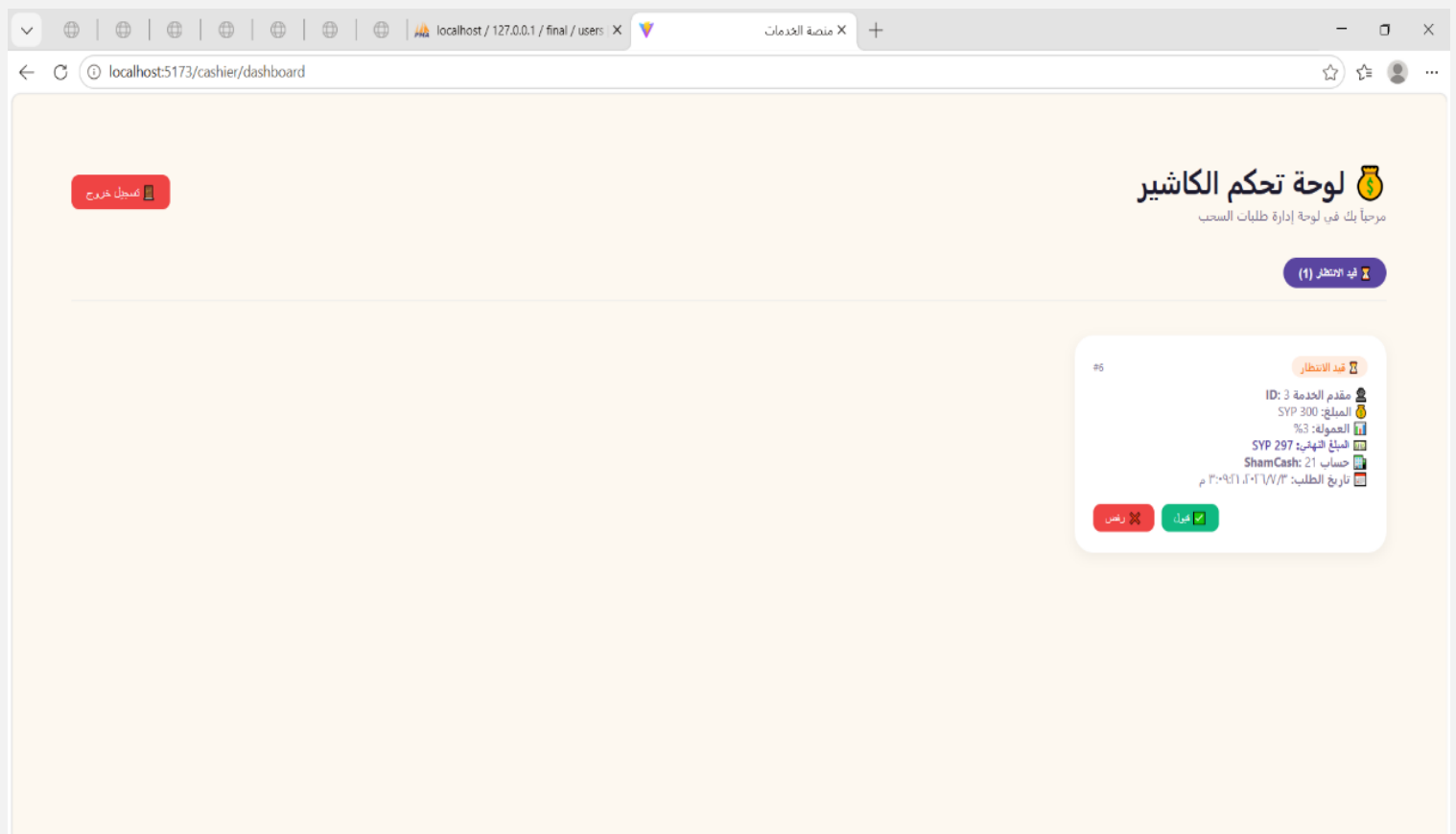
تم تطوير نظام محفظة إلكترونية (Electronic Wallet) خاص بمقدمي الخدمات، يهدف إلى إدارة المبالغ المالية الناتجة عن الخدمات المنجزة بصورة آمنة ومنظمة.

بعد إتمام الخدمة بنجاح، والتحقق من عملية الدفع، يقوم النظام بإضافة قيمة الخدمة إلى رصيد مقدم الخدمة داخل المحفظة الإلكترونية، مما يسمح له بالاحتفاظ بالأرصدة وسحبها في الوقت المناسب.

ويتيح النظام لمقدم الخدمة الاطلاع على الرصيد الحالي مباشرة من خلال واجهة مخصصة تعرض قيمة الرصيد المتاح للسحب.

11. نظام إدارة المحفظة الإلكترونية (Electronic Wallet Management System)





Withdrawal Request System 11.1

يوفر النظام آلية متكاملة لطلب سحب الأموال من المحفظة الإلكترونية إلى حساب مقدم الخدمة.

يقوم مقدم الخدمة بإدخال البيانات التالية:

- قيمة المبلغ المطلوب سحبه.
- رقم حساب ShamCash الذي سيتم التحويل إليه.

بعد التحقق من صحة البيانات، يتم إنشاء طلب سحب جديد بحالة (Pending) ، ليتم مراجعته لاحقاً من قبل موظف الكاشير.

ويهدف هذا التصميم إلى فصل عملية إنشاء الطلب عن عملية تنفيذ التحويل الفعلي، مما يمنح الإدارة القدرة على مراجعة جميع العمليات المالية قبل اعتمادها.

Withdrawal Validation Rules 11.2

قبل إنشاء طلب السحب، يقوم النظام بتنفيذ مجموعة من قواعد العمل (Business Rules) لضمان صحة العملية، وتشمل:

- التحقق من الحد الأدنى للمبلغ المسموح بسحبه.
- التحقق من الحد الأعلى للمبلغ.
- التأكد من توفر الرصيد الكافي داخل المحفظة.
- منع إنشاء أكثر من طلب سحب قيد المعالجة للمستخدم نفسه.
- التحقق من وجود رقم حساب التحويل.

وفي حال عدم تحقق أي من هذه الشروط، يتم رفض الطلب مع إرجاع رسالة توضح سبب الرفض.

Withdrawal Commission Calculation 11.3

يعتمد النظام سياسة احتساب عمولة تلقائية على عمليات السحب.

عند إنشاء الطلب، يقوم النظام بحساب:

- قيمة المبلغ المطلوب سحبه.
- قيمة العمولة المستحقة.
- المبلغ النهائي الذي سيصل إلى مقدم الخدمة بعد خصم العمولة.

ويتم حفظ جميع هذه القيم داخل قاعدة البيانات لضمان الشفافية وإمكانية مراجعة العمليات المالية لاحقاً.

يساهم هذا الأسلوب في فصل منطق احتساب العمولات عن واجهة المستخدم، بحيث تصبح جميع العمليات المالية موحدة وقابلة للتعديل مستقبلاً دون الحاجة إلى تغيير التطبيق.

Cashier Management Module 11.4

تم تطوير وحدة مستقلة خاصة بموظفي الكاشير (Cashier Module) ، تتولى إدارة جميع طلبات السحب الواردة من مقدمي الخدمات.

توفر هذه الوحدة مجموعة من الوظائف، أهمها:

- استعراض جميع طلبات السحب المعلقة.
- مراجعة بيانات الطلب.
- اعتماد الطلب.
- رفض الطلب.
- تسجيل الموظف المسؤول عن معالجة الطلب.
- تسجيل تاريخ ووقت تنفيذ العملية.

يسمح هذا التصميم بفصل العمليات المالية عن صلاحيات الإدارة العامة، بحيث يمتلك موظف الكاشير مسؤولية تنفيذ التحويلات المالية فقط.

Withdrawal Approval Workflow 11.5

تمر عملية السحب بعدة مراحل متتابعة تضمن سلامة الإجراءات المالية، ويمكن تلخيصها كما يلي:

1. يقوم مقدم الخدمة بإنشاء طلب سحب.
2. يتم حفظ الطلب بحالة (Pending).
3. يقوم موظف الكاشير بمراجعة الطلب.
4. يتم التأكد من أن الطلب لم تتم معالجته سابقاً.
5. يتم التحقق من توفر الرصيد داخل المحفظة.
6. يتم خصم قيمة السحب من رصيد مقدم الخدمة.
7. يتم تحديث حالة الطلب إلى (Approved).
8. يتم تسجيل الموظف الذي نفذ العملية وتاريخ تنفيذها.

يساعد هذا التسلسل في منع تنفيذ عمليات سحب مكررة أو غير صحيحة.

Withdrawal Rejection 11.6

في حال تعذر تنفيذ عملية السحب، يستطيع موظف الكاشير رفض الطلب.

عند تنفيذ عملية الرفض، يقوم النظام بما يلي:

- تغيير حالة الطلب إلى (Rejected).
- تسجيل الموظف المسؤول عن الرفض.
- تسجيل وقت معالجة الطلب.
- حفظ سبب الرفض (إن وجد).

وتسمح هذه المعلومات بالرجوع إلى سجل العمليات المالية ومعرفة أسباب رفض أي طلب مستقبلاً.

Financial Integrity 11.7

حرص النظام على المحافظة على سلامة البيانات المالية من خلال تطبيق مجموعة من الضوابط، من أهمها:

- منع خصم أي مبلغ قبل اعتماد الطلب رسميًا.
- منع معالجة الطلب أكثر من مرة.
- التحقق من الرصيد قبل تنفيذ أي عملية خصم.
- الاحتفاظ بجميع بيانات عملية السحب داخل قاعدة البيانات.
- تسجيل الموظف الذي قام بمعالجة كل عملية.
- تسجيل وقت تنفيذ كل عملية مالية.

تضمن هذه الإجراءات المحافظة على اتساق البيانات المالية وتقليل احتمالية حدوث أخطاء أو تكرار في عمليات السحب.

Withdrawal Status Management 11.8

يعتمد النظام دورة حياة واضحة لطلبات السحب (Withdrawal Lifecycle) ، حيث تنتقل الطلبات بين مجموعة من الحالات المحددة، وهي:

- **Pending:** تم إنشاء الطلب ويحتاج إلى مراجعة.
- **Approved:** تمت الموافقة على الطلب وتنفيذ عملية الخصم.
- **Rejected:** تم رفض الطلب مع تسجيل سبب الرفض عند الحاجة.

يساعد هذا النموذج في تتبع جميع العمليات المالية، كما يسهل على كل من مقدم الخدمة والإدارة معرفة الحالة الحالية لكل طلب بصورة دقيقة.

Wallet Inquiry Services 11.9

يوفر النظام مجموعة من الخدمات التي تمكن مقدم الخدمة من متابعة حالته المالية بشكل مستمر، وتشمل:

- استعراض الرصيد الحالي للمحفظة الإلكترونية.
- عرض جميع طلبات السحب السابقة.
- معرفة حالة كل طلب (قيد المعالجة، مقبول، مرفوض).
- متابعة العمليات المالية المنفذة على الحساب.

وتسهم هذه الخدمات في توفير مستوى عالٍ من الشفافية، وتمكين مقدم الخدمة من متابعة معاملاته المالية دون الحاجة إلى التواصل مع إدارة النظام.

نظام التقارير الإدارية وتصدير ملفات PDF 12

يوفر النظام وحدة متكاملة لإعداد التقارير الإدارية وتصديرها بصيغة PDF، بحيث يستطيع مدير النظام استخراج تقارير تفصيلية عن مختلف مكونات المنصة وفق فترة زمنية محددة. تم الاعتماد على مكتبة DomPDF لإنشاء التقارير بصيغة احترافية قابلة للطباعة أو الأرشفة.

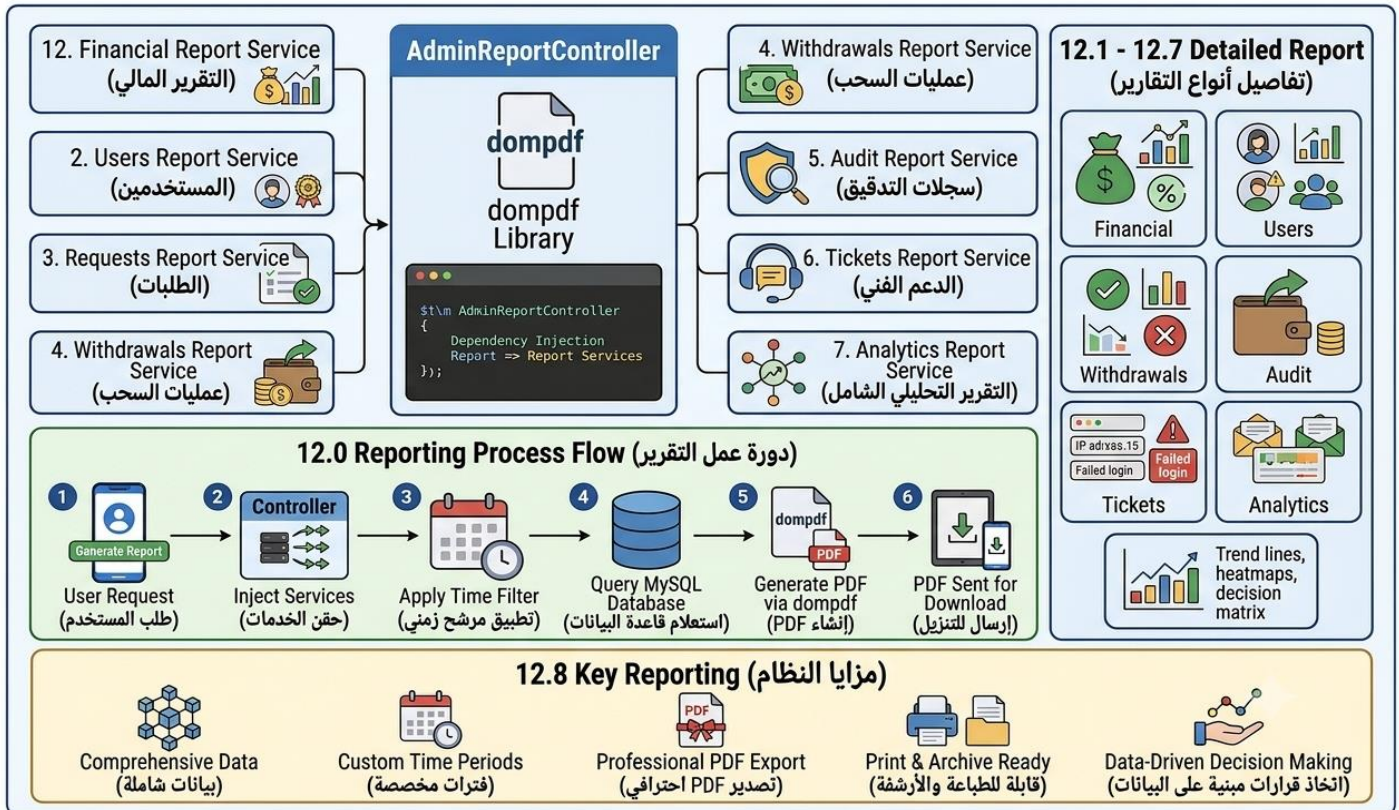
يعتمد AdminReportController على حقن مجموعة من خدمات التقارير (Report Services)، بحيث تكون كل خدمة مسؤولة عن نوع محدد من التقارير، بينما يقتصر دور المتحكم على استقبال طلب المستخدم واستدعاء الخدمة المناسبة ثم إنشاء ملف PDF وإرساله للتنزيل.

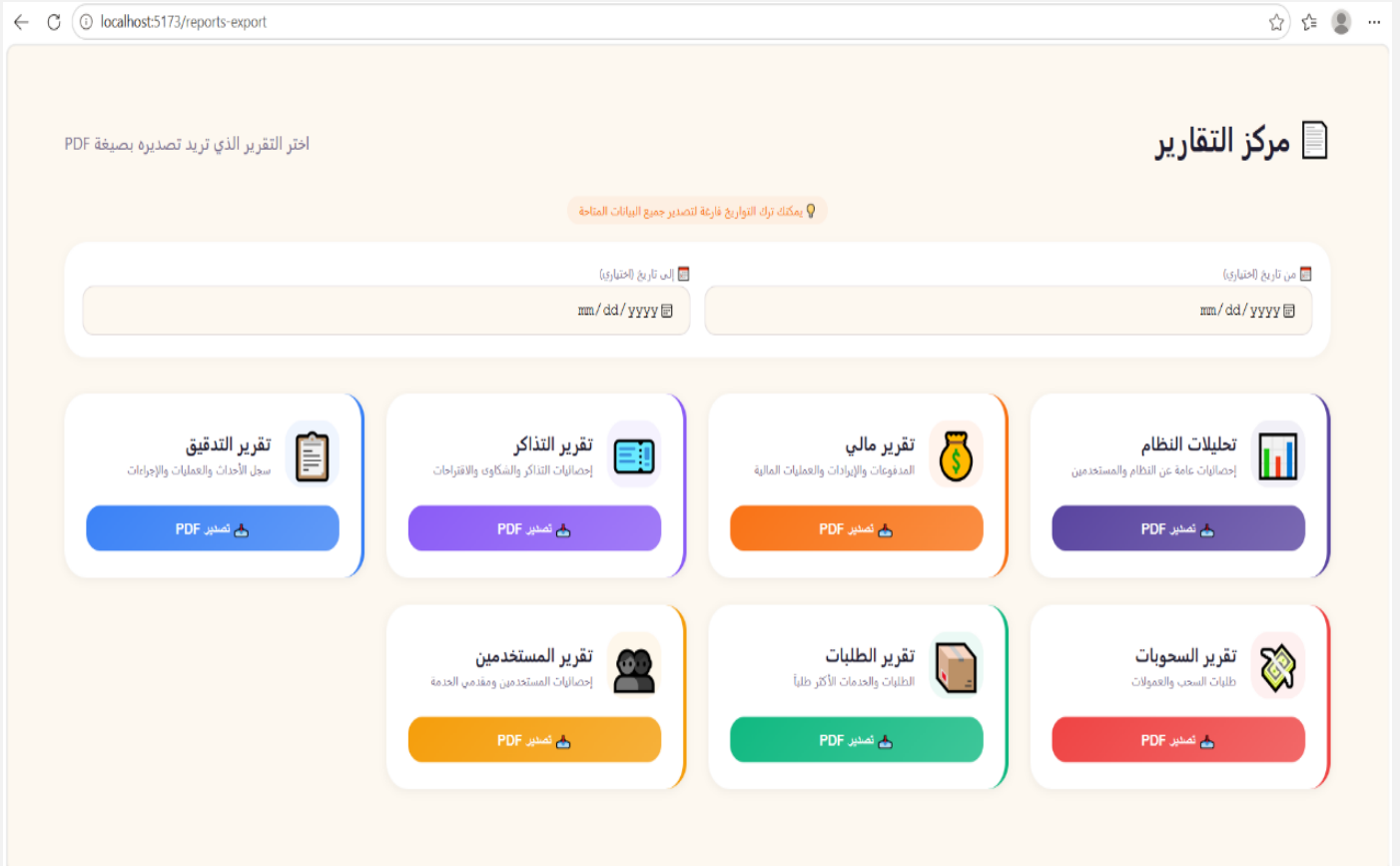
يدعم النظام الأنواع التالية من التقارير:

- التقرير المالي. (Financial Report).
- تقرير المستخدمين. (Users Report).
- تقرير الطلبات. (Requests Report).
- تقرير عمليات السحب. (Withdrawals Report).
- تقرير سجلات التدقيق. (Audit Report).
- تقرير التذاكر والدعم الفني. (Tickets Report).
- التقرير التحليلي الشامل. (Analytics Report).

كما تسمح جميع التقارير بتطبيق مرشحات زمنية (من تاريخ – إلى تاريخ)، مما يمنح الإدارة إمكانية استخراج تقارير خاصة بفترة معينة.

12. نظام التقارير الإدارية وتصدير ملفات PDF (Administrative Reporting)





التقرير المالي 12.1 (Financial Report)

يقوم التقرير المالي بتحليل جميع عمليات الدفع داخل النظام، ويعرض مؤشرات تساعد الإدارة على متابعة الأداء المالي، ومن أهم البيانات التي يحتويها:

- إجمالي المدفوعات الناجحة .
- إجمالي المدفوعات المعلقة .
- إجمالي المدفوعات الفاشلة .
- عدد عمليات الدفع .
- الإيرادات اليومية .
- أعلى عمليات الدفع .
- متوسط قيمة عملية الدفع .
- معدل نمو المدفوعات .
- نسبة نجاح عمليات الدفع .

تقرير المستخدمين 12.2 (Users Report)

يعرض هذا التقرير إحصاءات شاملة عن المستخدمين ومقدمي الخدمات داخل النظام، ويتضمن:

- إجمالي عدد المستخدمين .
- عدد مقدمي الخدمات .
- عدد المستخدمين العاديين .
- عدد المزودين الموثقين .
- عدد المزودين غير الموثقين .
- معدل نمو المستخدمين .
- المستخدمين المسجلين شهرياً .
- أكثر المستخدمين نشاطاً .
- أكثر مقدمي الخدمات نشاطاً .
- أفضل مقدمي الخدمات حسب التقييم .
- مقدمو الخدمات الأكثر إنجازاً للطلبات .
- أحدث المستخدمين المسجلين .
- المستخدمون المقيدون .
- المستخدمون الذين قاموا بتأكيد البريد الإلكتروني .
- أكثر مقدمي الخدمات حصولاً على التقييمات .

تقرير الطلبات 12.3 (Requests Report)

يوفر التقرير تحليلاً كاملاً لجميع طلبات الخدمات داخل المنصة، ويشمل:

- العدد الكلي للطلبات .
- الطلبات المكتملة .
- الطلبات المعلقة .
- الطلبات الملغاة .
- أكثر التصنيفات استخداماً .
- معدل نمو الطلبات .
- متوسط عدد الطلبات اليومية .

تقرير عمليات السحب 12.4 (Withdrawals Report)

يقدم هذا التقرير صورة مالية كاملة عن عمليات السحب التي ينفذها مقدمو الخدمات، ويتضمن:

- إجمالي طلبات السحب .
- الطلبات المقبولة .
- الطلبات المعلقة .
- الطلبات المرفوضة .
- إجمالي المبالغ المسحوبة .
- إجمالي العمولات المستقطعة .
- إجمالي المبالغ المحولة بعد خصم العمولة .
- متوسط قيمة السحب .
- أكبر عملية سحب .
- أصغر عملية سحب .

- توزيع الطلبات حسب الحالة .
- عمليات السحب اليومية .
- أعلى عمليات السحب .
- أحدث طلبات السحب .
- أكثر مقدمي الخدمات إجراءً لعمليات السحب .
- إحصاءات أداء أمناء الصندوق (Cashiers).

تقرير سجلات التدقيق 12.5 (Audit Report)

يستخدم هذا التقرير لمراقبة نشاط النظام وتحليل العمليات المنفذة داخله، ويتضمن:

- إجمالي السجلات .
- عدد السجلات الحرجة .
- سجلات التحذير .
- السجلات المعلوماتية .
- العمليات الفاشلة .
- العمليات الناجحة .
- أحدث سجلات النظام .
- أكثر العمليات استخداماً .
- أكثر المستخدمين نشاطاً .
- أكثر عناوين IP نشاطاً .
- توزيع الأحداث حسب النوع .
- توزيع الأحداث حسب مستوى الخطورة .
- محاولات تسجيل الدخول الفاشلة .
- النشاط اليومي للنظام .

تقرير التذاكر 12.6 (Tickets Report)

يوفر التقرير معلومات تفصيلية عن نظام الدعم الفني والتذاكر، ويشمل:

- إجمالي عدد التذاكر .
- التذاكر المفتوحة .
- التذاكر المعلقة .
- التذاكر المغلقة .
- نسبة حل التذاكر .
- متوسط عدد التذاكر اليومي .
- توزيع التذاكر حسب الحالة .
- معدل نمو التذاكر .
- أكثر المستخدمين إنشاءً للتذاكر .
- المستخدمون الذين لديهم أكبر عدد من التذاكر المفتوحة .
- أكثر الأيام استقبلاً للتذاكر .
- أحدث التذاكر .
- أقدم التذاكر المفتوحة .

التقرير التحليلي الشامل 12.7 (Analytics Report)

يعد هذا التقرير الأكثر شمولاً داخل النظام، حيث يجمع بيانات من عدة وحدات ويقدم مؤشرات تحليلية تساعد الإدارة في اتخاذ القرارات، ويتضمن:

- إحصاءات المستخدمين .
- نمو المستخدمين اليومي والشهري .
- نمو مقدمي الخدمات .
- الإيرادات ونمو المدفوعات .
- متوسط قيمة المدفوعات .
- إحصاءات الطلبات ونسبة الإنجاز .
- توزيع الطلبات حسب الحالة .
- أكثر التصنيفات استخداماً .
- أكثر الخدمات طلباً .
- أكثر المستخدمين نشاطاً .
- أكثر مقدمي الخدمات نشاطاً .
- أفضل مقدمي الخدمات حسب التقييم .
- متوسط التقييمات وعددها .
- متوسط عدد الطلبات اليومية .

13. متطلبات لوحة التحكم الإدارية (Dashboard Requirements)

13. ADMIN DASHBOARD REQUIREMENTS

13.1 Overview

SYSTEM PROVIDES INTEGRATED ADMIN DASHBOARD FOR REAL-TIME PLATFORM MONITORING VIA KPI CARDS & ANALYTICAL CHARTS.

13.2 KPI CARDS (Real-time Indicators)



TOTAL USERS

Total number of users registered.



SERVICE PROVIDERS

Number of registered and approved service providers.



TOTAL PAYMENTS

Total monetary value of successful payments.



TOTAL REQUESTS

Cumulative count of service requests submitted.



SUPPORT TICKETS

Total count of open and closed support tickets.



TOTAL WITHDRAWALS

Total monetary value of approved withdrawal operations.

13.3 ANALYTICAL CHARTS (Data Analysis & Trends)



USER GROWTH

Track daily/monthly rate of new user sign-ups.



TICKET DISTRIBUTION

Analyze support ticket percentages by status (Open, In Progress, Closed).



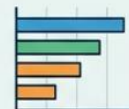
MOST ACTIVE USERS

Identify and rank top interacting users on the platform.



REQUEST GROWTH

Monitor service request volume trends over time.



MOST REQUESTED CATEGORIES

Compare popularity of service categories.



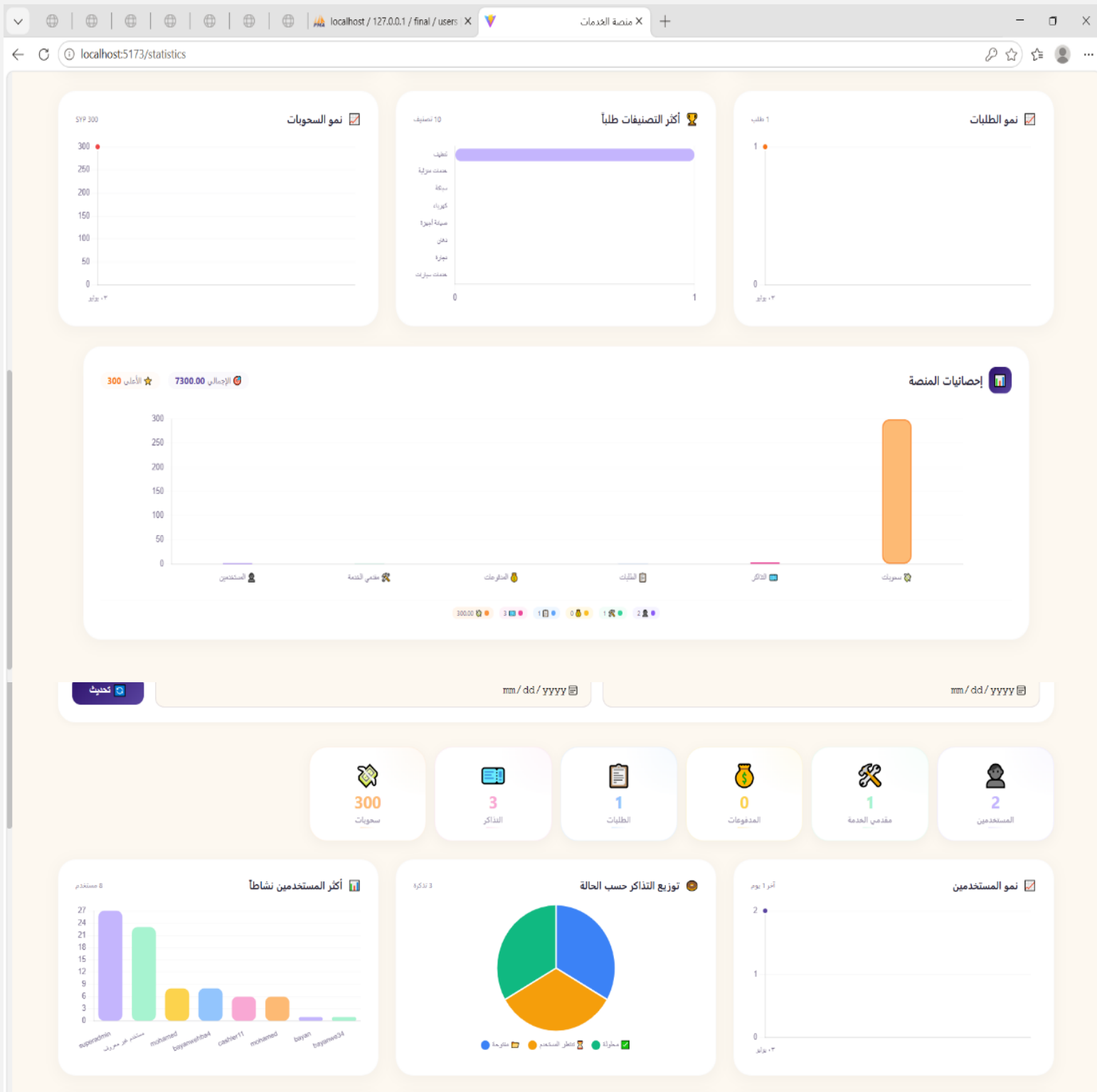
WITHDRAWAL GROWTH

Track total approved withdrawal values over time.



OVERALL PLATFORM STATISTICS

Consolidate general indicators for quick comparison.



13.1 نظرة عامة (Overview)

يجب أن يوفر النظام لوحة معلومات إدارية متكاملة (Admin Dashboard) تتيح للإدارة مراقبة أداء المنصة بشكل فوري، وذلك من خلال بطاقات مؤشرات الأداء الرئيسية (KPIs) ومجموعة من الرسوم البيانية المتنوعة لتحليل البيانات.

13.2 بطاقات مؤشرات الأداء الرئيسية (KPI Cards)

يجب أن تعرض لوحة التحكم المؤشرات اللحظية التالية:

الوصف	اسم المؤشر (KPI)	م
يعرض العدد الكلي للمستخدمين المسجلين في المنصة.	إجمالي المستخدمين	1
يعرض عدد مقدمي الخدمات المسجلين والمعتمدين.	مزودي الخدمة	2
يعرض مجموع المبالغ المالية للمدفوعات الناجحة.	إجمالي المدفوعات	3
يعرض العدد التراكمي لطلبات الخدمات المقدمة.	إجمالي الطلبات	4
يعرض العدد الإجمالي لتذاكر الدعم الفني المفتوحة والمغلقة.	تذاكر الدعم	5
يعرض مجموع المبالغ المالية لعمليات السحب التي تم اعتمادها.	إجمالي السحوبات	6

13.3 المخططات والرسوم البيانية (Analytical Charts)

يجب أن يدعم النظام 7 أنواع من الرسوم البيانية لتشريح البيانات وتطورها عبر الزمن كالتالي:

الغرض والوصف الوظيفي	نوع الرسم (Chart Type)	المخطط البياني	م
تتبع معدل انضمام المستخدمين الجدد بشكل يومي/شهري.	الخط الزمني (Line Chart)	نمو المستخدمين	1
تحليل نسب تذاكر الدعم الفني حسب حالتها (مفتوحة، قيد المعالجة، مغلقة).	الدائري المجوف (Pie/Donut Chart)	توزيع التذاكر	2
تحديد وتصنيف المستخدمين الأكثر تفاعلاً واستخداماً للمنصة.	الأعمدة (Bar Chart)	المستخدمين الأكثر نشاطاً	3

4	نمو الطلبات	الخط الزمني (Line Chart)	مراقبة حجم الطلبات على الخدمات وتطورها عبر الأيام.
5	التصنيفات الأكثر طلباً	الأعمدة الأفقية (Horizontal Bar)	مقارنة تصنيفات الخدمات الأكثر إقبالاً من قبل المستخدمين.
6	نمو السحوبات	المساحي المنحني (Area Chart)	تتبع حجم المبالغ المالية المسحوبة والمعتمدة وتطورها بمرور الوقت.
7	إحصائيات المنصة الشاملة	الأعمدة المركبة (Bar Chart)	تجميع ودمج المؤشرات العامة في مخطط واحد للمقارنة السريعة والمباشرة.

الفصل الخامس: دراسة تطبيقات مشابهة

SOFTWARE APPLICATION COMPARISON: SYSTEM DESIGN DOCUMENT APPENDIX

COMPARISON CRITERIA	 HANDS (Local Hybrid Platform)	 KHAMSAT / MOSTAQBAL (Regional Freelance)	 UPWORK (Global Freelance)	 TASKRABBIT (Global Local Service)
 SERVICE NATURE	Local & On-demand Hybrid	Digital Services & Freelancing	Digital Services & Long-term Projects	Physical Tasks & Local Help
 GEOGRAPHICAL SCOPE	Specific Local Area, e.g., City/Region	Middle East & North Africa (MENA) Region	Worldwide/Global	Selected Cities Worldwide
 DYNAMIC FORMS	Highly Customizable for Diverse Local Services	Mostly Standardized for Digital Categories	Project-Specific Proposals, Less Template Dynamic	Fixed Task Categories with Limited Custom Fields
 PAYMENT GATEWAY	Integrated Local & Global Gateways (e.g., KNET, Fawry, Cards)	Regional Partners & Payoneer/PayPal	Global Bank Transfers, PayPal, Cards	Integrated App Payments
 PRICING MECHANISM	Flexible: Fixed Price, Hourly, or Hybrid Estimates	Micro-tasks (\$5) & Larger Project Budgets	Hourly Rates, Fixed-Price Milestones, Bidding	Hourly Rates & Task-Based Estimates
 LOCATION ENGINE	Advanced Real-Time Geolocation for Service Matching	Limited: Regional Filters but not Real-Time	Very Limited: Mostly Virtual Collaboration	Strong Location-Based for Nearby Helpers
 VERIFICATION	Rigorous: National ID, Local Background Checks, Phone	Standard: Email & Sometimes Basic ID	Extensive: Video Verification, ID Checks	Extensive: Criminal Record Checks, ID Verification

APPENDIX D - COMPARATIVE ANALYSIS | SYSTEM DESIGN DOCUMENT V1.2

14.1 مقدمة (Introduction)

يهدف هذا القسم إلى استعراض وتحليل الأنظمة والمنصات الحالية الشبيهة بنظام Hands على المستويين الإقليمي والعالمي. يركز التحليل على تحديد نطاق عمل هذه المنصات، وبنيتها الوظيفية، بهدف تحديد الفجوات التقنية والتشغيلية في السوق الحالي، وتوضيح القيمة التنافسية والهندسية المضافة التي يقدمها نظام Hands للتلبية متطلبات البيئة المحلية المستهدفة.

14.2 توصيف المنصات المنافسة (Competitors Profiles)

1. منصتا خمسات ومستقل (Khamsat & Mostaqbal)

- طبيعة النظام: منصات إقليمية (تابعة لشركة حسوب) مخصصة لإدارة العمل الحر عن بُعد.
- نطاق العمل الوظيفي: تركز هذه المنصات كلياً على تسليم المنتجات والخدمات الرقمية (مثل: البرمجة، التصميم، صناعة المحتوى، الترجمة).
- القيود التقنية والتشغيلية: لا تدعم المنصتان أي نوع من الخدمات الفيزيائية أو اليدوية الميدانية التي تتطلب تواجداً جغرافياً على أرض الواقع، كما تعتمدان على آليات دفع دولية غير متوافقة مع القيود المالية المحلية.

2. منصة أب ورك (Upwork)

- طبيعة النظام: نظام عالمي لإدارة وتوظيف الكفاءات والمستقلين رقمياً (Enterprise Freelancing Platform).
- نطاق العمل الوظيفي: توفير بيئة عمل متكاملة للشركات والأفراد لتنفيذ مشاريع رقمية وتقنية وإدارية عن بُعد، مدعومة بأدوات مراقبة إنتاجية متطورة وسجلات زمنية (Time Tracking).
- القيود التقنية والتشغيلية: صرامة الإجراءات المالية والتعقيد البيروقراطي للنظام، بالإضافة إلى عدم دعمه للخدمات المهنية واليدوية الميدانية، فضلاً عن خروج السوق السوري المستهدف من نطاق تغطيتها المالي والقانوني بسبب الحظر الدولي.

3. منصة تاسك رابيت (TaskRabbit)

- طبيعة النظام: منصة عالمية رائدة للخدمات المحلية والفيزيائية الميدانية القائمة على الموقع (On-Demand Local Services Platform).
- نطاق العمل الوظيفي: تمكين المستخدمين من توظيف مقدمي خدمات لتنفيذ مهام مادية ملموسة (مثل: الصيانة المنزلية، التركيب، النقل، خدمات التنظيف).
- القيود التقنية والتشغيلية: على الرغم من كونها المنافس الأقرب من حيث المفهوم التشغيلي لنظام Hands، إلا أنها مصممة ببنية تحتية جغرافية ومالية تناسب الدول الغربية فقط (تعتمد الرموز البريدية Zip Codes وبوابات الدفع الدولية مثل Stripe)، مما يجعل تشغيلها أو الاستفادة منها في بيئة السوق المحلية أمراً مستحيلاً.

14.3 مصفوفة المقارنة الشاملة (Comprehensive Comparison Matrix)

يوضح الجدول التالي مقارنة تحليلية ومواصفائية بين نظام Hands والأنظمة المنافسة:

منصة TaskRabbit	منصة Upwork	منصة خمسات / مستقل	نظام Hands	معايير المقارنة / الخاصية
خدمات فيزيائية وميدانية فقط.	خدمات رقمية فقط (عن بُعد).	خدمات رقمية فقط (عن بُعد).	نموذج هجين (Hybrid): فيزيائية/ميدانية، تقنية، تجميلية، وتعليمية...	طبيعة ونوع الخدمات
نطاق عالمي (أمريكا الشمالية وأوروبا).	نطاق عالمي مفتوح.	العالم العربي والشرق الأوسط.	السوق المحلي (الجمهورية العربية السورية).	النطاق الجغرافي المستهدف
تصنيفات مسبقة وقوائم مهام محددة سلفاً.	وصف نصي مرن مع تصنيفات عامة ثابتة.	وصف نصي ثابت (Static Text) يحدده العميل.	نماذج ديناميكية (Dynamic Forms): الحقول والأسئلة برمجياً بحسب نوع الخدمة.	هيكلية جمع متطلبات الخدمة
بوابات دفع دولية مشفرة (Stripe).	قنوات مالية دولية معقدة بنوك عالمية، (PayPal PayPal).	بوابات دفع دولية وإقليمية (PayPal)، بطاقات اتتمان.	تكاملي محلي كامل : مدعوم ببوابة ShamCash ومحفظة ذكية داخلية ونظام كاشير.	بوابة ونظام الدفع (Payment Gateway)
نظام تسعير ثابت بالساعة يحدده مقدم الخدمة.	سعر ثابت للمشروع أو نظام تسعير بالساعة.	تسعير ثابت (خمسات) أو تقديم عروض مالية محددة للمشروع (مستقل).	مرونة عالية :تحديد نطاق سعري أولي وتفاوض نهائي بعد المعاينة أو التنفيذ الميداني.	آلية التسعير والتفاوض
مدعوم استاتيكياً بناءً على الرمز البريدي (Zip Code).	غير مدعوم (تصفية بناءً على البلد أو المنطقة فقط).	غير مدعوم (لا تتطلبه طبيعة الخدمات الرقمية).	متقدم ولحظي : الاعتماد على معادلة Haversine وحساب زمن الوصول التقريبي (ETA).	محددات البحث الجغرافي

فحص السجل الجنائي والخلفية الأمنية (Background Check) مزودين خارجيين.	توثيق صارم بالاعتماد على البيانات الحيوية (Biometrics) والوثائق الرسمية.	توثيق الهوية الرقمية (اختياري أو إلزامي عند سحب الأموال).	توثيق صارم ومحلي: رفع وثائق الهوية الشخصية، شهادات مزاولة المهنة، ومراجعة إدارية يدوية.	التحقق من الهوية والأمان (Verification)
--	--	--	--	---

14.4 الميزات التنافسية والقيمة الهندسية المضافة لنظام Hands (Competitive Advantages)

تكمّن القوة الهندسية والتشغيلية لنظام Hands في معالجة الفجوات التقنية التي تقع فيها الأنظمة العالمية، وملاءمته التامة مع البنية التحتية والخصوصية الثقافية للسوق المستهدف، وتتخلص العوامل التنافسية بما يلي:

1. معمارية الخدمات الهجينة (Hybrid Service Architecture)

يتميز النظام بقدرته على كسر القيود التقليدية للمنصات الرقمية؛ حيث يوفر بيئة موحدة تجمع بين المهنيين الميدانيين (كالنجارة، السباكة، الكهرباء) ومقدمي الخدمات الفكرية والتقنية (كالتدريس الخصوصي، التصميم، والخدمات التجميلية). هذا التكامل يحول النظام إلى منصة خدمات شاملة (Super App) تلبي الاحتياجات اليومية للمجتمع المحلي في مكان واحد.

2. التوافق المالي التام مع البنية التحتية المحلية (ShamCash Integration)

يتخطى النظام عقبة الحظر المالي وغياب وسائل الدفع العالمية في السوق السوري من خلال توفير تكامل برمجي (API Integration) مع بوابة الدفع المحلية ShamCash. يدعم هذا التكامل نظام محفظة رقمية ذكية (Internal Wallet) ونظام كاشير (Cashier Management System) يضمن حجز الأموال وضمان حقوق الطرفين (العميل ومقدم الخدمة) وحل النزاعات المالية محلياً دون تعقيدات.

3. المعمارية البرمجية المتقدمة (Architectural Supremacy)

تم بناء النظام وفق أعلى المعايير الهندسة البرمجية لضمان القابلية للتوسع (Scalability) والاستقرار:

- النماذج الديناميكية (Dynamic Forms) تنتج المعمارية إدخال وتعديل حقول متطلبات الخدمات من لوحة التحكم بشكل لحظي، وينعكس ذلك على واجهات المستخدم (UI) دون الحاجة لإعادة بناء البرمجية أو دفع تحديثات جديدة للمتاجر. (Over-the-Air Configurations)
- إدارة التزامن المتفائلة (Optimistic Locking) تضمن عدم تضارب البيانات أو حدوث (Race Conditions) أثناء معالجة تذاكر الدعم الفني والبلاغات من قبل المشرفين في نفس الوقت.
- البرمجة موجهة الجوانب (AOP) ونمط المراقب الشامل (Global Observer) توفر آلية مؤتمتة بالكامل لتسجيل عمليات التدقيق والعمليات الأمنية (Audit Logs) بشكل منفصل عن منطق العمل الأساسي (Business Logic)، مما يحسن الأداء ويسهل تتبع الاختراقات أو الأخطاء.

4. محرك البحث الجغرافي اللحظي (Real-Time Location Engine)

تطبيق خوارزميات رياضية متقدمة تعتمد على معادلة Haversine مدمجة مع نظام تحديد المواقع العالمي (GPS) لتصفية وترشيح مقدمي الخدمات الأقرب جغرافياً للعميل. لا يقتصر النظام على تحديد المسافة الاستاتيكية، بل يعمل على حساب زمن الوصول التقريبي (ETA) الفعلي، مما يمنحه كفاءة تشغيلية وميدانية تتفوق على الأنظمة العالمية التي تعتمد على الرموز البريدية الثابتة.

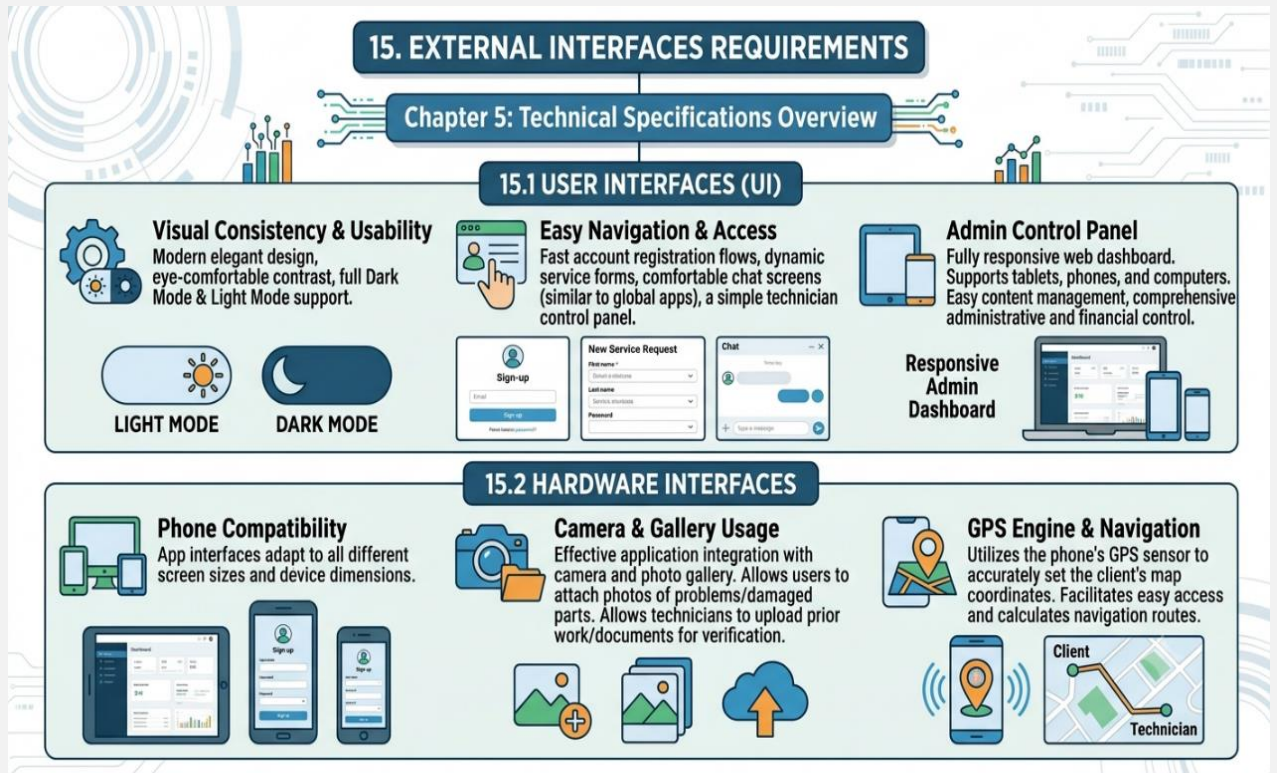
5. مرونة حوكمة الأسعار (Flexible Pricing Governance)

يتمشى النظام مع طبيعة الخدمات الفيزيائية الميدانية من خلال إتاحة "نظام التفاوض المرن". يسمح النظام لمقدم الخدمة بتحديد نطاق سعري (حد أدنى وأقصى)، مع إمكانية تعديل السعر النهائي وتأكيد به بعد المعاينة الفعلية على أرض الواقع، شريطة عدم تجاوز الحد الأعلى المتفق عليه إلا بعد إرسال طلب تحديث سعر رسمي وموافقة العميل برمجياً عليه، مما يضمن الشفافية ويمنع الاستغلال.

6. الإشراف الوقائي والمؤتمت (Automated Micro-Moderation)

يملك النظام نظام حماية استباقي يعتمد على قواعد برمجية ذكية (Rule-Based Engine) عند تجاوز مقدم الخدمة لعدد محدد من البلاغات أو التقييمات السلبية المقبولة، يقوم النظام باتخاذ إجراء "تقييد وتجميد تلقائي وحظر مؤقت" للحساب فوراً لحماية أمن المستخدمين، مع إحالة الملف إلى الإشراف البشري، على عكس الأنظمة التقليدية التي تنتظر المراجعات اليدوية الطويلة.

الفصل السادس: متطلبات الواجهات الخارجية



15.1. واجهات المستخدم الرسومية (UI):

- الاتساق البصري وسهولة الاستخدام: يجب أن يتمتع التطبيق بتصميم عصري وأنيق مستخدماً تبايناً لونياً مريحاً للعين مع دعم كامل للوضع الداكن (Dark Mode) والوضع الفاتح (Light Mode).

- **سهولة التصفح والوصول:** توفير واجهات وصول سريعة لتسجيل الحسابات، طلب الخدمات الديناميكية عبر النماذج التفاعلية، شاشة دردشة مريحة تشابه تطبيقات المحادثة العالمية، ولوحة تحكم مخصصة للفنيين لعرض الطلبات القريبة وإرسال العروض بضغط زر واحدة.
- **لوحة تحكم المشرفين:** واجهة ويب مستجيبة (Responsive Web Dashboard) بالكامل تدعم التصفح من الأجهزة اللوحية والهواتف وأجهزة الكمبيوتر لإدارة المحتوى والتحكم الإداري والمالي الشامل ببسر وسهولة.

15.2. واجهات العتاد والأجهزة (Hardware Interfaces):

- **التوافق الهاتفي:** تماشي واجهات المنصة مع أحجام الشاشات المختلفة وأبعاد الأجهزة الذكية المتوفرة بالأسواق.
- **استخدام الكاميرا والمعرض:** تكامل التطبيق الفعال مع نظام كاميرا الهاتف ومعرض الصور للسماح للمستخدمين بالتقاط الصور للمشاكل والقطع التالفة وإرفاقها بالطلبات، أو للسماح للفنيين برفع أعمالهم السابقة وصور الوثائق الثبوتية للتحقق من الحسابات.
- **محرك الموقع والملاحة (GPS):** الاعتماد على مستشعر تحديد المواقع بالهاتف لتعيين إحداثيات العميل بدقة على الخريطة لتسهيل الوصول وحساب مسارات الملاحة.

الفصل السابع: المتطلبات غير الوظيفية والأمان

CHAPTER SIX: NON-FUNCTIONAL REQUIREMENTS AND SECURITY



16.1. PERFORMANCE STANDARDS & SPEED

- **RESPONSE TIME:** Backend HTTP requests < 500ms under normal load.



- **CONCURRENCY:** Design for thousands of simultaneous users/requests without service degradation.



- **PAGINATION:** Efficiently fetch long data lists (offers, requests, messages) in batches.



16.2. CACHING MECHANISMS

- **REDIS CACHE:** Use Redis for frequently accessed and complex DB queries (categories, dynamic questions, near active requests, tech stats).



- **CACHE EVICTION:** Strict lifecycle to update/delete cache upon any data write/modify/delete to prevent stale data.



16.3. SYSTEM ARCHITECTURE



16.4. ASPECT-ORIENTED PROGRAMMING (AOP)

Manage concerns:

- Logging
- Security
- Transaction Management



16.5. STRONG INPUT VALIDATION

Prevent attacks:

- SQL Injection
- XSS



16.6. SECURITY & PRIVACY

- **PRIVACY & DATA ENCRYPTION:** Block sharing sensitive user info (exact phone, location) with technicians before offer acceptance.



- **FINANCIAL RECORD INTEGRITY:** Use Database Transactions in all financial operations and withdrawals, auto-rollback on connection/program errors.



16.7. SCALABILITY

Designed to support more users and data volume without impacting performance.



16.8. FLEXIBILITY & MAINTAINABILITY

High flexibility for adding new features or modifying current ones.

16.1. معايير الأداء والسرعة (Performance):

- زمن الاستجابة السريع (Response Time): يجب ألا يتجاوز زمن استجابة خوادم النظام الخلفية لطلبات الـ HTTP العادية 500 مللي ثانية في ظروف التشغيل والضغط الطبيعي.
- القدرة على تحمل الضغط المتزامن (Concurrency): تصميم المنصة والتحقق من قدرتها البرمجية والعنادية للتعامل مع آلاف المستخدمين والطلبات المتزامنة في نفس اللحظة دون هبوط في مستويات الخدمة أو تراجع الأداء.
- التقسيم الفعال للبيانات (Pagination): تطبيق معايير تفتيت وجلب القوائم الطويلة للبيانات (كالعروض، الطلبات، والرسائل) على دفعات مبرمجة لتقليل حجم الاستهلاك وضمان سرعة العرض.

16.2. آليات التخزين المؤقت وحفظ البيانات (Caching):

- Redis Cache: اعتماد محرك Redis لحفظ وتخزين البيانات التي يتكرر تصفحها والوصول إليها بشكل كبير وتتطلب استعلامات معقدة من قاعدة البيانات (مثل التصنيفات، الأسئلة الديناميكية للخدمات، الطلبات النشطة القريبة، الإحصائيات العامة للفنيين) وحفظها في الذاكرة المؤقتة السريعة.
- تحديث وتنظيف الذاكرة التلقائي (Cache Eviction): تصميم دورة حياة برمجية صارمة تقوم بحذف وتحديث التخزين المؤقت فور حدوث أي عملية كتابة أو تعديل أو حذف تطرأ على البيانات الأساسية لضمان عدم حصول المستخدمين على بيانات قديمة.

System Architecture. 16.3

يجب أن يتم تصميم النظام وفق بنية الطبقات الثلاث (Three-Tier Architecture) والتي تتكون من:

- طبقة العرض (Presentation Layer)
- طبقة منطق الأعمال (Business Logic Layer)
- طبقة البيانات (Data Layer)

16.4. Aspect Oriented Programming (AOP):

يجب أن يستخدم النظام مبدأ البرمجة المعتمدة على الجوانب (AOP) لإدارة الاهتمامات المشتركة مثل التتبع (Logging) والأمان وإدارة المعاملات.

Strong Input Validation. 16.5

يجب أن يطبق النظام تحقّقاً صارماً من المدخلات (Strong Validation) لجميع حالات الاستخدام لضمان سلامة البيانات ومنع الهجمات مثل:

- SQL Injection
- XSS

16.6. معايير الوثوقية والأمان والخصوصية (Security & Privacy):

- حماية الخصوصية وتعمية البيانات: منع مشاركة أي معلومات شخصية حساسة للعميل (مثل رقم هاتفه الدقيق وموقعه التفصيلي) مع الفنيين قبل قبول العرض والاتفاق الرسمي على التنفيذ، للحفاظ على أمان وخصوصية المستخدمين ومنع المضايقات.
- حفظ سلامة السجلات المالية: استخدام ميزة المعاملات البرمجية المتكاملة (Database Transactions) في جميع العمليات المالية وعمليات السحب؛ بحيث يتم التراجع تلقائياً عن العملية بالكامل في حال حدوث أي انقطاع بالاتصال أو خطأ برمجي طارئ أثناء التنفيذ، لمنع حدوث حالات نقص مالي أو فقدان للأرصدة ومحافظ

Scalability.16.7

يجب أن يتم تصميم النظام بحيث يكون قابلاً للتوسع (Scalable) لدعم زيادة عدد المستخدمين أو حجم البيانات دون التأثير على الأداء.

Flexibility and Maintainability. 16.8

يجب أن يتمتع النظام بدرجة عالية من المرونة وقابلية الصيانة لتسهيل إضافة ميزات جديدة أو تعديل الميزات الحالية.